

BitZ: Proofs in arbitrary domains with commitments in binary fields

immediate

August 21, 2026

Abstract

Contents

1	Introduction	3
1.1	[Notes/Draft] Contributions	3
1.2	Related work	4
1.3	Acknowledgements	4
1.4	Ramblings (warm-up for the true intro)	4
2	Technical overview	7
2.1	$\mathbb{F}_2\mathbb{Z}$ for R1CS over integers with virtual linear constraints over $\mathbb{F}_2$	7
2.2	Scratch notes	7
3	Preliminaries	7
3.1	Algebra	8
3.2	Linear codes and relations	8
3.3	Interactive Oracle Reductions (IOR) of proximity	9
4	$\mathbb{F}_2\mathbb{Z}$ IOP for Lin-BitsDual	11
4.1	The Lin-BitsDual relation	11
4.2	Tensor decomposition, interleaved codes, and products of constrained codes	12
4.3	An IOR for projecting the weights to a random prime field	14
4.4	The $\mathbb{F}_2\mathbb{Z}$ IOP for the core Lin-BitsDual relation	17
4.5	Soundness and completeness	19
4.6	$\mathbb{F}_2\mathbb{Z}$: an IOP for the general Lin-BitsDual relation	20
4.7	Instantiation	22
5	Virtualization of $\mathbb{F}_2$-linear combinations lifted to integers	24

6	A GKR protocol for low entropy batched grand products via lookup tables	26
6.1	Optimization via lookup tables	29
6.2	Other optimizations for Construction 4.8	34
6.3	Optimizing $\mathbb{F}_2\mathbb{Z}$ for virtualized inputs $\pi_2^{-1}(M \cdot \pi_2(\mathbf{f}))$	35
7	$\mathbb{F}_2\mathbb{Z}$ for PIMSAT	35
7.1	Polynomial Ideal Membership Satisfiability (PIMSAT)	35
7.2	An IOP for PIMSAT with oracles in binary fields	36
7.3	The CM constraint system: composition of $\mathbb{Z}$ -linear with $\mathbb{F}_2$ -linear constraints	40
8	Experiments	41
8.1	Instantiation	41
8.2	Experimental results	42
8.2.1	Raw performance of $\mathbb{F}_2\mathbb{Z}$	42
8.2.2	Integer and modular multiplication	42
8.2.3	SHA-256 hashing	42
8.2.4	Falcon signature verification	44
8.2.5	RSA and Poseidon	44
8.2.6	Hashing followed by Falcon signature verification	44
8.2.7	Comparison with Hyrax	44
	References	44
A	Deferred proofs	45
A.1	Completeness and soundness of Construction 4.12	45
A.2	Completeness and soundness of Construction 7.3	46
A.2.1	Proof of Lemma A.2	51
B	Ring switching via Galois orbits	52
C	Bilinear Embeddings	53
C.1	Background	53
C.1.1	Perfect pairings	53
C.1.2	Succinct vectors	54
C.2	Structure	55
C.3	Constructions	57
C.3.1	Trace embedding	57
C.3.2	Coefficient projection	58
C.4	Extension openings	59
C.4.1	Batching protocol	59
C.4.2	Folding protocol	60
D	Scratch	65
D.1	Adding Zero Knowledge	65
D.1.1	PIOP	65
D.1.2	F2Z-PCS	65
D.2	Open questions for us	66

1 Introduction

1.1 [Notes/Draft] Contributions

F2Z: Proving linear claims over any field (possibly an extension field) or integers, for a vector of bits committed on a binary field. [Albert: What happens if we replace FRI by F2Z in Plonky3?](#)

We can handle arbitrary linear weights in our claims, of arbitrary bit-size and in arbitrary extension degrees, and still use a smallish binary field like GHASH during our protocol (not obvious at all because “proving in the exponent in principle only allows for using $\lesssim 90$ -ish integer bits. [Albert: This is a good story for the technical overview](#)

Follow up on Zinc+: we adopt its constraint system and greatly boost its performance with the aforementioned F2Z

Results in proof system that works with highly expressive and “small” constraints, and is competitive or outperforms other systems on many tasks of interests. Removes all the complexity of arithmetization, and in exchange requires you to work on a gnarly grand product.

New optimizations for grand products in binary fields, arising from our “in the exponent claims” (Binius64 already works with such grand products – we provide new optimizations).

Virtualization of XOR (or, more generally, $\mathbb{F}_2$ -linear transformations) of our committed witness elements, without the need to arithmetize on binary fields whatsoever.

Generalization/improvements to ring-switch in binary fields?

Benchmarks: F2Z works with a highly expressible constraint system and outperforms in all axis on all measured tasks except in proving big quantities of hashes. For that task, prover time is 2-3x slower, while proof sizes are about 20-33% smaller, and memory usage is smaller too [Albert: how about the verifier?](#). For small number of hashes, say $< 2^{10}$, F2Z outperforms on all axis [Albert: or so it seems](#). This may be because, with F2Z arithmetization, witnesses are smaller, and so floor costs in F2Z may be smaller than in other schemes.

Our stuff is 128 bits of security (once we add grinding). Achieving it is less costly because lowering rate of code has lesser impact (though we get a big performance loss over $> 2^{34}$ -ish witness bits because we can’t use grinding to keep working with a binary field generator of only 128 bits).

Binius is 96 bits of security. With our change of field into a prime field of 105 bits in step 1 of F2Z, we can more easily achieve 128 bits? (The PIOP can be on a > 128 bit prime)

Ok I think I understood a big difference between F2Z and Binius, and a main reason F2Z may perform better in, say, integer mult:

The grand product in Binius arises when proving integer mult $a * b = c$. In the exponent they prove $g^{a*b} = g^c$. This means they have witness*witness claims on the exponent.

In F2Z we have witness*public_value in the exponent. This allows us to 1) tensor split the claim, 2) use a lot of lookup tricks (which require tensor splitting to begin with). OTOH a claim involving witness*witness terms cannot be easily tensor split, and so the lookup tricks cannot be applied easily.

The reason we only need witness*public_value in the exponent is that we run the PIOP on Z before calling our PCS

Applications In the F2Z world, when elements in the computation never exceed 100 bits, we can work on a fixed prime, instead of a random one, because F2Z enforces range checks automatically

[Albert: Motivate ideals somewhere \(intro?\)](#)

1.2 Related work

Integers and more: Zaratan, Zinc, Zinc+, Qaren, Limber

Binary fields. Binius papers and Binius64, Flock, BinarySpartan

1.3 Acknowledgements

Usage and non-usage of AI This document was essentially written without the assistance of AI. AI was only used to identify typos, and for the cost accounting in [Section 6](#).

We used AI in our implementation and experiments, as explained in ??.

1.4 Ramblings (warm-up for the true intro)

Why does this paper exist? F2Z has many similarities to Binius64, so why F2Z at all? First of all, Binius64 is an end-to-end proving system, while F2Z is a PCS. Let's consider F2Z-SNARK a SNARK for proving R1CS constraints over $\mathbb{Z}$ or $\mathbb{Z}[X]$ with ideals. Flock is also related to F2Z-SNARK, though less so, since Flock is a more use-case specific SNARK for proving bit-intensive computations. We should also keep in mind that Flock is, at the end of the day, Binius64 on steroid optimizations. In general, I think of Flock as subsumed by a Binius64 repo that adopts Flock's optimizations. Hence I will not discuss Flock too much here.

In what follows I understand F2Z-SNARK to be a proof system for constraints over $\mathbb{Z}, \mathbb{Z}[X]$ with ideals, binary fields, etc., like Zinc+ at the arithmetization and PIOP level (applied to R1CS-like constraints), but using F2Z as PCS.

Efficiency and costs I will now compare F2Z-SNARK with Binius64 on a operation-by-operation case:

- *Integer and modular addition.* F2Z-SNARK can handle addition or modular addition) of arbitrarily sized integers. Even if the integers are gigantic, the projection to random field trick compresses their size to 100-ish bits.

On the other hand, integer addition or modular addition (how does Binius handle modular addition when the modulus is not 2^n actually?) in Binius64 requires committing to the carry bits (so each single addition adds a new 32-64 bit word to the witness), and cannot handle large integer/modular addition without chunking.

Further, addition in Binius64 produces a non-linear constraint, because proving that the committed carry bits are indeed carry bits requires a Hadamard product. In F2Z-SNARK, additions are linear constraints.

- *Integer or modular multiplication.* This is a quadratic constraint in F2Z-SNARK. It does not introduce extra witness¹. Again, it supports arbitrary sizes and benefits from the random projection for large sizes.

Binius64 supports multiplication of elements of less than 32 bits. Larger sizes require chunking. Under 32 bits, it does not pay much for multiplication at the constraint level. The way it

¹Or it negligible does, because of the possible $+n \cdot w$ term in a modular equation, where n is the modulus and w is a committed integer –very small in bit size in practice

proves the multiplications is via a grand product similar to F2Z-SNARK grand product in MLE evaluations.

A computation where we are just making m multiplications of under 32 bit integers may actually end up costing almost the same between Binius64 and F2Z-SNARK. [Albert: Let's run the experiment](#)

It is interesting that in Binius64, multiplications don't go through the Spartan-like PIOP, and instead are proved directly with the grand product [Albert: At least that's my understanding](#). In F2Z-SNARK, multiplications would indeed go through the normal PIOP pipeline. Let's keep in mind as something F2Z-SNARK could mimic. [Albert: Interesting](#)

- *XOR*
- *AND*
- *Proof system costs*

Specific performance This is WIP, but when it comes to hash proving, F2Z-SNARK seems to outperform in all axes Binius/Flock when there are less than 2^8 hashes to prove. Beyond that, the F2Z-SNARK route points to 1.5-3x prover times, comparable proof sizes, significantly smaller RAM usage, and unknown verifier comparison for now [Albert: WIP](#). The practical max number of hashes that F2Z-SNARK can handle is higher than in Flock due to the lesser memory usage.

The fact that F2Z-SNARK performs better for smaller number of hashes may be due to F2Z-SNARK working with a smaller witness. At those sizes, there's likely floor costs (e.g. Merkle openings) that somewhat dominate costs, and having a 2x (roughly) smaller witness may start to be relevant.

Interesting to note that in many use cases (e.g. digital Id) one has less than 2^8 hashes to prove. Flock-like numbers come up in Ethereum signature aggregation and maybe proof of provenance where one has to prove a hash of a large file (and others?).

[Albert: Current WIP: hash + ECDSA sign comparison. Quantized ML linear layer](#)

For initial reference, Binius64 takes about 110ms to prove an ECDSA with a 200KB proof size.

Scientific contributions So far, we have compared Binius64 and F2Z-SNARK in terms of performance. But how about the scientific contributions of F2Z-SNARK?

F2Z-SNARK is a proof system designed to handle constraints over $\mathbb{Z}$, $\mathbb{Z}[X]$ with ideal membership, binary fields, and other fields, simultaneously (in v3 we are probably sticking to integers and $\mathbb{Z}[X]$ for simplicity). Arithmetization and PIOP-wise, it borrows from Zinc+, and the novelty compared to Zinc+ is that we are committing to the integer bits over a binary field. For bit-intensive computations, this is much better than committing over $\mathbb{Z}$. It remains to be seen what happens in integer-heavy computations [Albert: TODO! Edit: John Wu already compared it to Hyrax](#).

Potentially, we could commit to large integers with a different PCS over $\mathbb{Z}$, say with IPRS, and create a recursive proof of opening for the two PCS's, using F2Z since PCS opening proofs are hash heavy.

Thus some scientific contributions of F2Z-SNARK are

- Unlocks good scaling of bit intensive computations in Zinc+-like proof systems.

	F2Z-PCS	Zinc+-PIOP	Binius64	Flock	Plonky3	Zinc+
ECDSA			119/6.3/195/?	NA		
RSA				NA		
2^{17} int mul 16bits	48/2/80/62		217/9/313/?	NA	367/7/340/?	
2^{17} int mul 32bits	74/2.4/86/124		218/9/313/?	NA	571/7.6/373/?	
1M int mul 16bits	192/2.9/112/483		3600/61/440/?	NA	2920/9.8/492/?	
1M int mul 32bits	266/3.7/119/863		3800/62/440/?	NA	5050/10.2/520/?	
1M int mul 128bits	266/3.7/119/863		3800/62/440/?	NA	5050/10.2/520/?	
2^{17} BabyBear mults				NA	56/7/306/?	
1M BabyBear mults				NA	400/11/478/?	
2^7 SHA	12.7/1.75/71/14*	expected negl.	37/3.4/249/?	33/25/112/?		
2^{11} SHA	93.5/2.8/109/197*	expected negl.	126/27/381/?	47.2/27/172/?		
2^{15} SHA	1320/4.8/170/3080*	expected negl.	Stuck	425/25/233/?		
2^8 Blake3	22/2/71/23	expected negl.	31/2.6/249/?	256/254/111/?		
2^{11} Blake3	91/2.8/105/164	expected negl.		253/249/153/?		
2^{15} Blake3	1100/4.7/161/2550	expected negl.	Stuck	386/267/160/?		

Table 1: Albert: Our stuff is 128 bits of security? (Once we add grinding?) How many queries? *= we are not using virtualization of XOR. The committed and opened witness size is 4x larger than it could be. Albert: Missing costs for F2Z PIOP. Note these are negligible for the hash benches though, and likely quite less than the grand product cost in other benches. All these were obtained asking AI to run benches. For the final paper we should make sure we are doing fair and correct benching (e.g. I’m not sure memory consumption is measured the same way across benches, especially for Binius, which is max RSS), but this helps get a rough idea how F2Z performs relatively to other systems. Prover_time(ms)/Verifier_time(ms)/Proof_size(KB)/Peak_memory(MB). Integer mult measure one million $u16 \times u16 \rightarrow u32$ mults. F2Z-PCS does not include PIOP costs – these are negligible in the hash benches (linear constraints), but should be accounted for in the other benches (degree 2 constraints). Ran in MacBook Air M4 16GB.

Flock’s verifier seems a bit broken in general. Flock’s Blake3 prover for small number of hashes seems broken too. I modified Flock’s Ligerito config so that it produces smaller proofs at very little extra cost. For Blake3 we couldn’t do 2^7 hashes because Flock errors out on 2^7 hashes Albert: todo: make SHA also do 2^8 instead of 2^7

F2Z-PCS hash performance was “proxied” as follows: for SHA-256, 2^{14+k} witness corresponds to 2^k hashes. Better arithmetization may be possible. We *DID NOT* leverage virtualization of XOR, which allows to run Ligerito for 2^{12+k} witness size, rather than 2^{14+k} (The GKR grand product still would use 2^{14+k} factors). Albert: VERY IMPORTANT: It is possible that the GKR grand product can be optimized in this setting. Right now we are just proving a 2^{14+k} on unstructured bits, but in reality many of these bits are linear combinations in $\mathbb{F}_2$ of witness bits.

Binius64 achieves the highest number of mults per second at 2^{17} mults. Binius64 at multiplication of 16bit integers pays a penalty because the system uses 64-bit words by default, so it either uses a 64 bits per 16bit witness element and range check the word, or pack different 16 and 32 bit integers into the same 64 bit word, but then it has to relate the product of the 64 bit words to the products of the original words (or unpack them as 64 bit words before multiplying). OTOH doing more Binius-native mults ($32 \times 32 \rightarrow 64$ bits) does not perform amazingly either Albert: This most likely heavily disproves my belief that Binius64 and F2Z-SNARK are similar in integer multiplication. This is a good showcase of the flexibility of arithmetizing the way we do: we don’t need to use predefined word sizes: here our R1CS is just reconstruction of integers via linear combinations with powers of twos, and Hadamard product (note that we are not measuring the cost of the PIOP in F2Z! But it should be cheaper than the grand product, and amenable to some better optimization (univariate skip e.g.). But in any case, I’m not even sure the issue is the rigidity of the arithmetization in Binius64, since $32 \times 32 \rightarrow 64$ is Binius friendly as far as I understand, and performance is mid

- If we include Zinc+ in this line of work, F2Z-SNARK is a follow up for a proof system that uses a highly expressive constraint system. Due to the expressivity, the use-cases of F2Z-SNARK are potentially huge, and difficult to predict. For example, the F2Z-PCS could be used as a drop-in replacement for elliptic curve PCS’s (as shown by John Wu)

We (tentatively) choose F2Z-SNARK (likely with both WHIZ and F2Z) working exclusively on $\mathbb{Z}$ or $\mathbb{Z}[X]$ as our route for a general zkDSL, and the motivation was the expressivity of the constraints.

- F2Z-SNARK does not quite provide much novelty regarding SNARKs over binary fields, beyond possibly some optimizations in the grand product proof (e.g. lower levels of the grand product GKR are computed via a lookup table, exploiting the limited options we have in leaves due to the bit-ness of the witness).

That being said, some of these optimizations could be published as contributions, following Flock’s lead. [Albert: Sometimes I undersell my work, this is a memo to myself to be mindful with that](#)

There’s also the virtualization of XOR elements of F2Z-PCS. In a sense, a similar thing occurs in Binius64: their R1CS constraint system admits hadamard products, replacing the hadamard by integer multiplication. So such constraint is multiplying together linear combinations of the witness (over the binary field, so “XOR combinations”).

Our virtualization trick can be understood as if we had an R1CS over $\mathbb{Z}$, with witness $\mathbf{z}$, and an extra constraint $\mathbf{z} = \pi_2^{-1}(M\pi_2(\mathbf{z}))$ for some matrix M . What Binius is doing can be understood similarly, but with much more limited expressivity. Hence in this sense Binius64 arithmetization is a specific case of our arithmetization.

[Albert: Noting so I don’t forget: the idea of F2Z being a drop-in replacement alternative to Hyrax \(PQ secure and more efficient\) may be a good extra motivating point](#)

2 Technical overview

2.1 $\mathbb{F}_2\mathbb{Z}$ for R1CS over integers with virtual linear constraints over $\mathbb{F}_2$

[Albert: TODO](#)

2.2 Scratch notes

Thoughts: We can handle arbitrary linear weights in our claims, of arbitrary bit-size and in arbitrary extension degrees, and still use a smallish binary field like GHASH during our protocol (not obvious at all because “proving in the exponent in principle only allows for using $\lesssim 90$ -ish integer bits. [Albert: This is a good story for the technical overview](#)

3 Preliminaries

General notation Given an integer $n \geq 1$, we write $[n]$ to denote the set $\{1, \dots, n\}$. Any map $f : S \rightarrow T$ is extended entrywise to vectors and matrices with entries in S , and coefficient-wise to polynomials with coefficients in S .

We use $\circ$ to denote composition of maps, so that for maps $f : S_1 \rightarrow S_2$ and $g : S_2 \rightarrow S_3$, we have $f \circ g : S_1 \rightarrow S_3$ defined as $f \circ g(x) = f(g(x))$.

3.1 Algebra

We write $\mathbb{F}_q$ for the finite field with q elements (where q is a prime or a prime power), $\mathbb{Q}$ for the field of rational numbers, and $\mathbb{Z}$ for the ring of integers.

The *characteristic* of a ring R is the largest $c \geq 1$, if it exists, such that $1 + \dots + 1 = 0$ in R . Otherwise, if $1 + \dots + 1 \neq 0$ for all c , the characteristic is zero. By $[0..n]_R$ we denote all elements in R of the form $1 + \dots + 1$ for $i = 0, \dots, n$. For us, typically $R = \mathbb{Z}$ or R is a finite field $\mathbb{F}$.

We let $R[X]$ be the ring of univariate polynomials with coefficients in R , on the variable X . For a tuple of variables $\mathbf{Y} = (Y_1, \dots, Y_\mu)$, $R[\mathbf{Y}]$ denotes multivariate polynomials with coefficients in R . Given a degree bound $d \geq 1$, a bit-size bound $B \geq 1$, we define

$$R_{\leq B}^{\leq d}[X] := \left\{ a(X) \in R[X] \mid a(X) = \sum_{i=0}^d a_i \cdot X^i, \text{ for some } a_i \in [0..2^B - 1]_R, \text{ for all } i \right\},$$

When we place no degree or bit-size bound, we use the notation $R_{\leq B}[X]$ and $R^{<d}[X]$, respectively. Note that $R^{\leq d}[X]$ is an R -module of dimension $(d+1)$, admitting as base $1, X, \dots, X^d$.

Given a ring homomorphism $\psi : R_1 \rightarrow R_2$, a R_1 -module M_{R_1} , and a R_2 -module, a ψ -module homomorphism between M_{R_1} and M_{R_2} is a map ϕ such that $\phi(\alpha \cdot a + \beta \cdot b) = \psi(\alpha) \cdot \phi(a) + \psi(\beta) \cdot \phi(b)$ for all $\alpha, \beta \in R_1$ and $a, b \in M_{R_1}$.

Given a base $\alpha_1, \dots, \alpha_d$ for M_R , we let $M_{R_{\leq B}}$ be the subset of M_R with elements whose coordinates belong to $R_{\leq B}$ in this base.

A subset $S \subseteq R$ of a commutative ring R is called *exceptional* if for all two distinct $a, b \in R$, the element $a - b$ has a multiplicative inverse in R .

Lemma 3.1 (Generalized Alon–Füredi Theorem [BCPS18]). *Let R be a ring and let $S \subseteq R$ be a finite exceptional subset of R . Let $f(\mathbf{Y}) \in R[\mathbf{Y}]$ be a nonzero multilinear polynomial in μ variables. Then $\Pr_{\mathbf{r} \leftarrow S^\mu} [f(\mathbf{r}) = 0] \leq \mu/|S|$.*

3.2 Linear codes and relations

Linear codes Let $\mathcal{C} \subseteq \mathbb{F}^m$ be a linear code over $\mathbb{F}$ of dimension ℓ and blocklength m . For $h \geq 1$, the h -interleaving of $\mathcal{C}$ is the code $\mathcal{C}^{\equiv h} \subseteq (\mathbb{F}^h)^m$ of dimension $\ell \cdot h$ and blocklength $m \cdot h$, whose codewords have the form

$$((c_{i1}, \dots, c_{ih}))_{i \in [m]} \quad \text{for } (c_{1j}, \dots, c_{mj}) \in \mathcal{C} \quad \text{for all } j \in [h].$$

Given two words w_1, w_2 of length n , the relative Hamming distance between w_1 and w_2 is denoted

$$\Delta(w_1, w_2).$$

Relations A *relation* $\mathcal{R}$ consists of triples $(\mathbf{x}, \mathbf{y}; \mathbf{w})$, where $\mathbf{x}$ is the explicit instance, $\mathbf{y}$ the implicit instance or oracle, and $\mathbf{w}$ the witness. The corresponding language is $\text{LANG}(\mathcal{R}) := \{(\mathbf{x}, \mathbf{y}) \mid \exists \mathbf{w} \text{ such that } (\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}\}$.

Polynomial Equation Satisfiability (PESAT) over finite fields We recall a relation from [linear-accumulation] [Albert: Linear time accumulation paper, or earlier?](#) over finite fields, which captures popular constraints such as R1CS, CCS, AIR (cf. [linear accumulation]), and lookup and permutation constraints (cf. [zinc]).

Fix a finite field $\mathbb{F}$, a witness length ℓ , an input length k , a number of rows n , a collection of n polynomials $\mathbf{p} = (\mathbf{p}_1, \dots, \mathbf{p}_n)$ on $k + \ell$ variables, for each $j \in [n]$.

Definition 3.2. The PESAT relation with respect to these parameters is defined as

$$\mathcal{R}_{\text{PESAT}}(\mathbb{F}, \mathbf{p}) = \left\{ \begin{array}{l} \mathbf{x} = \mathbf{x} \in \mathbb{F}^k, \\ \mathbf{y} = \perp, \\ \mathbf{w} = \mathbf{f} \in \mathbb{F}^\ell \end{array} \middle| \mathbf{p}_i(\mathbf{x}, \mathbf{f}) = 0 \text{ for all } i \in [n] \right\}$$

Let $\mathcal{C} \subseteq \mathbb{F}^m$ be a linear code of dimension ℓ' and blocklength m over $\mathbb{F}$. Let $T : \mathbb{F}^\ell \rightarrow \mathbb{F}^{\ell'}$ be an arbitrary map (looking ahead, for us T will be a packing map that packs elements from $\mathbb{F}_2$ into elements of a larger binary field). We define the *Encoded PESAT relation* as:

$$\mathcal{R}_{\text{PESAT}}(\mathbb{F}, \mathbf{p}, \mathcal{C}, T) := \left\{ \begin{array}{l} \left(\begin{array}{l} \mathbf{x} = \mathbf{x} \in \mathbb{F}^k, \\ \mathbf{y} = f \in \mathbb{F}^m, \\ \mathbf{w} = \mathbf{f} \in (\mathbb{F}^\ell) \end{array} \right) \middle| \begin{array}{l} \text{Enc}_{\mathcal{C}}(T(\mathbf{f})) = f \\ \mathbf{p}_i(\mathbf{x}, \mathbf{f}) = 0 \text{ for all } i \in [n] \end{array} \right\}$$

We will drop the reference to T whenever it is understood from the context. When $n = 1$ and $\mathbf{x}$ are such that $\mathbf{x} = (\mathbf{v}, \mu) \in \mathbb{F}^\ell \times \mathbb{F}$; $\mathbf{p}_1(\mathbf{x}, \mathbf{f}) = \langle \mathbf{f}, \mathbf{v} \rangle - \mu$ (and $\ell = k$), the relation $\mathcal{R}_{\text{PESAT}}(\mathbb{F}, \mathbf{p}, \mathcal{C}, T)$ is called a *constrained code* [WHIR?] [Albert: WHIR?](#). We denote it

$$\mathcal{R}_{\text{Lin}}(\mathbb{F}, \mathcal{C}, T).$$

Definition 3.3 (Relaxation of $\mathcal{R}_{\text{PESAT}}$ and $\mathcal{R}_{\text{Lin}}$). Given $\delta \in (0, 1)$, the *relaxation* of $\mathcal{R}_{\text{PESAT}}$ and $\mathcal{R}_{\text{Lin}}$, denoted

$$\mathcal{R}_{\text{PESAT}}^\delta, \quad \mathcal{R}_{\text{Lin}}^\delta$$

are the relations defined exactly as $\mathcal{R}_{\text{PESAT}}$ and $\mathcal{R}_{\text{Lin}}$, respectively, except that the constraint $\text{Enc}_{\mathcal{C}}(T(\mathbf{f})) = f$ is replaced by

$$\Delta(\text{Enc}_{\mathcal{C}}(T(\mathbf{f})), f) < \delta.$$

3.3 Interactive Oracle Reductions (IOR) of proximity

A public-coin *Interactive Oracle Reduction (IOR) of proximity* $\Pi = (\mathbf{P}, \mathbf{V})$ from a relation $\mathcal{R}$ to a relation $\mathcal{R}'$ is specified as follows. The prover $\mathbf{P}$ receives $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}$. The verifier $\mathbf{V}$ receives $\mathbf{x}$ and query access to $\mathbf{y}$. They interact for k rounds. In round $j \in [k]$, the prover sends an oracle π_j and the verifier sends a uniformly random message $\rho_j \in \{0, 1\}^{r_j}$. After the k rounds, the verifier makes queries to $\mathbf{y}$ and to $\pi_1, \dots, \pi_k$. Then:

- $\mathbf{V}$ either rejects or outputs a new pair $(\mathbf{x}', \mathbf{y}')$, where $\mathbf{y}'$ is specified implicitly in terms of the oracles $\mathbf{y}$ and $\pi_1, \dots, \pi_k$.
- $\mathbf{P}$ outputs a witness $\mathbf{w}'$ such that $(\mathbf{x}', \mathbf{y}', \mathbf{w}') \in \mathcal{R}'$.

We write $\langle \mathbf{P}, \mathbf{V} \rangle(\mathbf{x}, \mathbf{y}, \mathbf{w})$ for the interaction of prover and verifier during the IOR.

Definition 3.4 (Completeness). Π has (*perfect*) *completeness* if for every $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}$,

$$\Pr_{\rho_1, \dots, \rho_k} \left[(\mathbf{x}', \mathbf{y}'; \mathbf{w}') \in \mathcal{R}' \mid (\mathbf{x}', \mathbf{y}'; \mathbf{w}') \leftarrow \langle \mathbf{P}, \mathbf{V} \rangle(\mathbf{x}, \mathbf{y}, \mathbf{w}) \right] = 1.$$

Remark 3.5. An IOP for a relation $\mathcal{R}$ is an IOR from $\mathcal{R}$ to the trivial relation $\mathcal{R}_{\text{triv}} = \{(\mathbf{x} = \perp, \mathbf{y} = \perp, \mathbf{w} = \perp)\}$.

We proceed to define round-by-round knowledge soundness of an IOR, as presented in [TCC:BCFW25].

Definition 3.6 (Knowledge state function with relaxation). Let $\Pi = (\mathbf{P}, \mathbf{V})$ be an IOR from a relation $\mathcal{R}$ to $\mathcal{R}'$. A *knowledge state function with relaxation* $(\widetilde{\mathcal{R}}, \widetilde{\mathcal{R}'})$ for Π is a (possibly inefficient) function KState that, on input $(\mathbf{x}, \mathbf{y})$, interaction transcript tr , and knowledge state witness $\mathbf{w}$, outputs a bit and has the following properties.

- **Empty transcript.** If $\text{tr} = \emptyset$ is the empty transcript, then $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if $(\mathbf{x}, \mathbf{y}; \mathbf{w}) \in \widetilde{\mathcal{R}}$.
- **Prover moves.** If tr is a transcript where the prover is about to move and $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 0$, then $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \pi, \mathbf{w}) = 0$ for every prover message π .
- **Full transcript.** If $\text{tr} = (\pi_1, \rho_1, \dots, \pi_k, \rho_k)$ is a full transcript, then $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if $\mathbf{V}$ on tr outputs $(\mathbf{x}', \mathbf{y}')$ such that $(\mathbf{x}', \mathbf{y}'; \mathbf{w}) \in \widetilde{\mathcal{R}'}$.

Definition 3.7 (Round-by-round knowledge soundness). A k -round IOR Π has *round-by-round knowledge soundness with relaxation* with errors $(\kappa_1, \dots, \kappa_k)$ if there exists a knowledge state function KState as in Definition 3.6 and a deterministic extractor $\mathcal{E}_{\text{rbr}}$ such that for every $(\mathbf{x}, \mathbf{y})$, every $j \in [k]$, and every transcript tr where the verifier is about to send ρ_j ,

$$\Pr_{\rho_j} \left[\exists \mathbf{w} \text{ such that } \begin{array}{l} \wedge \text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \rho_j, \mathbf{w})) = 0 \\ \wedge \text{KState}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \rho_j, \mathbf{w}) = 1 \end{array} \right] \leq \kappa_j(\mathbf{x}, \mathbf{y}).$$

If $(\widetilde{\mathcal{R}}, \widetilde{\mathcal{R}'}) = (\mathcal{R}, \mathcal{R}')$ then we say the round-by-round knowledge soundness of Π is *without relaxation*.

Remark 3.8 (Composition of IORs). *Albert: Turn into a theorem/lemma. What can I cite for this? TensorSwitch, Arc, or ZOOK?* IORs can be composed together and inherit completeness and round-by-round knowledge soundness errors. More precisely, let Π_1, Π_2 be perfectly complete IORs from $\mathcal{R}_1$ to $\mathcal{R}_2$ and $\mathcal{R}_3$, respectively, with round-by-round knowledge errors with relaxation $(\widetilde{\mathcal{R}}_1, \widetilde{\mathcal{R}}_2)$ and $(\widetilde{\mathcal{R}}_2, \widetilde{\mathcal{R}}_3)$, respectively; and with errors $\kappa_{11}, \dots, \kappa_{1k_1}$ and $\kappa_{21}, \dots, \kappa_{2k_2}$, respectively. Consider the IOR Π' from $\mathcal{R}_1$ to $\mathcal{R}_3$ consisting of applying IOR Π_1 first, and then Π_2 taking as input for Π_2 the output of Π_1 . Then Π' is a perfectly complete $(k_1 + k_2)$ -round IOR from $\mathcal{R}_1$ to $\mathcal{R}_3$ with round-by-round knowledge soundness with relaxation $(\widetilde{\mathcal{R}}_1, \widetilde{\mathcal{R}}_3)$ and errors $\kappa_{11}, \dots, \kappa_{1k_1}, \kappa_{21}, \dots, \kappa_{2k_2}$.

Grinding *Albert: Review* Grinding [?] is a technique where, given an IOP Π for a relation $\mathcal{R}$ with relaxation $\widetilde{\mathcal{R}}$ with round-by-round knowledge errors $\kappa_1, \dots, \kappa_k$, produces a new IOP Π' for $\mathcal{R}$ with round-by-round knowledge with relaxation $\widetilde{\mathcal{R}}$ with errors

$$2^{-\alpha_1} \cdot \kappa_1, \dots, 2^{-\alpha_k} \cdot \kappa_k.$$

The new IOP Π' works exactly as Π , except that, at each round $i \in [t]$, the verifier samples a random string of α_i bits, called *grinding string*, which sends alongside its message, and rejects unless all bits are 0.

Clearly Π' is not necessarily complete, even if Π is. The goal is to use the BCS compilation of IOP's, and not the IOP's themselves (i.e. the compilation where oracles are replaced by Merkle roots and where interactivenss is removed through the Fiat-Shamir transform) To this end, one chooses $\alpha_1, \dots, \alpha_k$ small enough that, the prover of the BCS transform of Π' can *grind* transcripts until all of the grinding strings in the transcript are the zero string. This results in the prover having to compute per round $i \in [t]$, an average of 2^{α_i} hashes of its i -th message followed by a small string of bits (in practice, one has the prover send a small string of bits next to its message, so that during grinding, it does not need to produce fresh Merkle roots at each grinding iteration). In Table 2, we provide example grinding times for different values of α_i .

The round-by-round knowledge errors of Π' are obtained by defining a state function which is exactly as the state function of Π , except that the function outputs zero if some of the aforementioned strings of bits are not the zero string. The extractor for Π' is defined as the extractor for Π , ignoring the additional strings of bits in the transcript.

Table 2: Cost of finding a 64-bit nonce such that $\text{BLAKE3}(\text{msg}, \text{nonce})$ has at least α leading zero bits, with msg of 256 bits. Single and multi-threaded on a MacBook Air M4 16GB. *Albert: As implemented in Flock?*

α	18	19	20	21	22	23	24	25
Mean, 1 thread (ms)	12.3	19.5	42.8	82.2	203.8	464.4	808.5	1291.4
Median, 1 thread (ms)	9.2	16.3	27.8	55.2	140.6	358.1	608.5	852.0
Mean, 10 threads (ms)	2.3	3.5	8.9	14.3	35.5	84.6	148.6	240.8
Median, 10 threads (ms)	1.8	2.5	6.8	9.6	26.9	68.0	111.3	161.8

4 $\mathbb{F}_2\mathbb{Z}$ IOP for Lin-BitsDual

Albert: Need to discuss somewhere that range checks are free

4.1 The Lin-BitsDual relation

In this section we define the main linear relations we work with. We start by fixing notation:

Notation and terminology

- Let $\mathbb{F}_{2^\nu}$ be a field with 2^ν elements. Let $\beta_1, \dots, \beta_\nu \in \mathbb{F}_{2^\nu}$ be an $\mathbb{F}_2$ -base of $\mathbb{F}_{2^\nu}$ seen as an $\mathbb{F}_2$ -vector space of dimension n . Given a vector $\mathbf{u} \in \mathbb{F}_2^m$, with 2^ν dividing m *Albert: add in hypotheses*, we define

$$\text{Pack}_\nu(\mathbf{u}) \in \mathbb{F}_{2^\nu}^{m/\nu}$$

as the vector whose j -th entry is

$$\text{Pack}_\nu(\mathbf{u})_j = \mathbf{u}_{(j-1) \cdot \nu + 1} \cdot \beta_1 + \dots + \mathbf{u}_{(j-1) \cdot \nu + \nu - 1} \cdot \beta_{\nu-1} + \mathbf{u}_{j \cdot \nu} \cdot \beta_\nu \in \mathbb{F}_{2^\nu}.$$

- Let $\ell \geq 1$ be a dimension parameter, $\mathcal{C}$ a code over $\mathbb{F}_{2^\nu}$ of dimension ℓ/ν and block length m .
- Let R be a ring (typically R is either $\mathbb{Z}$ or a finite field, not necessarily prime) and let $\pi : \mathbb{Z} \rightarrow R$ be a ring homomorphism (possibly the identity if $R = \mathbb{Z}$).

- Let R_M be a ring, possibly different than R , let $\pi_q : R_M \rightarrow R$ be a ring homomorphism. Let M be a R_M -module and let $\psi : M \rightarrow R$ be a module homomorphism (with underlying ring homomorphism π_q).
- Fix a vector length ℓ_0 . Let $\mathbf{f} \in M_{R_{\leq B}}^{\ell_0}$ and write $\mathbf{f} = \sum_{j \in [d]} \mathbf{f}_{*j} \cdot \gamma_j$ with $\mathbf{f}_{*j} \in R_{\leq B}^{\ell_0}$ for all $j \in [d]$. Then write

$$\mathbf{f}_{*j} = \sum_{k=1}^B \mathbf{f}_{*jk} \cdot 2^{k-1}$$

for some vectors

$$\mathbf{f}_{*jk} \in \{0, 1\}^{\ell_0} \subseteq R^{\ell_0},$$

for all j, k . We define $\text{bits}(\mathbf{f}) \in \{0, 1\}^{\ell_0 \cdot d \cdot B} \subseteq \mathbb{Z}^{\ell_0 \cdot d \cdot B}$ as the vector with integer entries obtained by concatenating the vectors $\mathbf{f}_{*jk}$ and, if $R = \mathbb{F}$, lifting their entries to $\{0, 1\} \subseteq \mathbb{Z}$ via π_q^{-1} , i.e.

$$\text{bits}(\mathbf{f}) := (\text{lift}(\mathbf{f}_{*jk}) \mid j \in [d], k \in [B]) \in \{0, 1\}^{\ell_0 \cdot d \cdot B} \subseteq \mathbb{Z}^{\ell_0 \cdot d \cdot B}, \quad (1)$$

where lift is the identity function if $R = \mathbb{Z}$, and $\text{lift} = \pi_q^{-1}$ if $R = \mathbb{F}$.

Definition 4.1. The *Core Lin-BitsDual* relation is defined as

$$\mathcal{R}_{\text{Lin-BitsDual}}(\{0, 1\}, \pi, R, \mathcal{C}) := \left\{ \left(\begin{array}{l} \mathbf{x} = (\mu, \mathbf{v}) \in R \times R^\ell, \\ \mathbf{y} = f \in \mathbb{F}_{2^\nu}^m, \\ \mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell \end{array} \right) \mid \begin{array}{l} f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f}) \\ \langle \pi(\mathbf{f}), \mathbf{v} \rangle = \mu \end{array} \right\}$$

For $\delta \in (0, 1)$, we let $\mathcal{R}_{\text{Lin-BitsDual}}^\delta$ be the relation $\mathcal{R}_{\text{Lin-BitsDual}}$ after replacing the constraint $f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f})$ by

$$\Delta(f, \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f})) < \delta.$$

Assume $\mathcal{C}$ is a linear code over $\mathbb{F}_{2^\nu}$ of dimension $(\ell_0 \cdot B \cdot d)/\nu$ and blocklength m .

Definition 4.2. The *general Lin-BitsDual* relation is defined as

$$\begin{aligned} \mathcal{R}_{\text{Lin-BitsDual}}(M_{\leq B}, \psi, R, \mathcal{C}) \\ = \left\{ \left(\begin{array}{l} \mathbf{x} = (\mathbf{v}, \mu) \in R^{\ell_0} \times R, \\ \mathbf{y} = f \in \mathbb{F}_{2^\nu}^m, \\ \mathbf{w} = \mathbf{f} \in M_{\leq B}^{\ell_0} \end{array} \right) \mid \begin{array}{l} f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu \circ \pi_2(\text{bits}(\mathbf{f})), \\ \langle \psi(\mathbf{f}), \mathbf{v} \rangle = \mu \end{array} \right\}. \end{aligned}$$

When there is no risk of confusion, we denote $\mathcal{R}_{\text{Lin-BitsDual}}(M_{\leq B}, \psi, R, \mathcal{C})$ simply by $\mathcal{R}_{\text{Lin-BitsDual}}$. The Core Lin-BitsDual relation is precisely $\mathcal{R}_{\text{Lin-BitsDual}}(M_{\leq B}, \psi, R, \mathcal{C})$ after taking $M = \mathbb{Z}$, $B = 1$, and ψ a morphism from $\mathbb{Z}$ to R .

4.2 Tensor decomposition, interleaved codes, and products of constrained codes

We start with some groundwork for our IOP for the Core Lin-BitsDual relation. Let $(\mathbf{x}, \mathbf{y}, \mathbf{w}) = ((\mu, \mathbf{v}), f, \mathbf{f}) \in \mathcal{R}_{\text{Lin-BitsDual}}(\{0, 1\}, \pi, R, \mathcal{C})$. We fix a tensor decomposition of vectors $\mathbf{v} \in R^\ell$ as

$$\mathbf{v} = \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)} = \left(\mathbf{v}_i^{(1)} \cdot \mathbf{v}_j^{(2)} \right)_{i \in [\ell_1], j \in [\ell_2]}$$

for some $\mathbf{v}^{(1)} \in R^{\ell_1}, \mathbf{v}^{(2)} \in R^{\ell_2}$, with $\ell_1 \cdot \ell_2 = \ell$. Note that the trivial tensor decomposition is admitted, i.e. we may effectively not decompose $\mathbf{v}$ by choosing $\ell_1 = \ell, \ell_2 = 1, \mathbf{v}^{(1)} = \mathbf{v}$, and $\mathbf{v}^{(2)} = (1)$.

We index the entries of $\mathbf{f}$ as $\mathbf{f} = (\mathbf{f}_{ij})_{i \in [\ell_1], j \in [\ell_2]}$. Note that

$$\langle \pi(\mathbf{f}), \mathbf{v} \rangle = \sum_{j \in [\ell_2]} \left(\sum_{i \in [\ell_1]} \pi(\mathbf{f}_{ij}) \cdot \mathbf{v}_i^{(1)} \right) \cdot \mathbf{v}_j^{(2)} = \sum_{j \in [\ell_2]} \langle \pi(\mathbf{f}_{*j}), \mathbf{v}^{(1)} \rangle \cdot \mathbf{v}_j^{(2)},$$

where $\mathbf{f}_{*j} = (\mathbf{f}_{ij})_{i \in [\ell_1]}$. Consider the relation (cf. [Definition 3.2](#)) $\mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{Bit}\mathbb{Z}}, \mathcal{C}, \text{Pack}_\nu)$ for ℓ_2 constraint polynomials $Q_{\text{Bit}\mathbb{Z}}$ so that

$$\mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{Bit}\mathbb{Z}}, \mathcal{C}, \text{Pack}_\nu) = \left\{ \left(\begin{array}{l} \mathbf{x} = (\mathbf{y}, \boldsymbol{\mu}) \in \mathbb{F}_{2^\nu}^{\ell_1} \times \mathbb{F}_{2^\nu}^{\ell_2}, \\ \mathbf{y} = f \in \mathbb{F}_{2^\nu}^m, \\ \mathbf{w} = \mathbf{f} = (\mathbf{f}_{*j})_{j \in [\ell_2]} \in \mathbb{F}_2^{\ell_1 \times \ell_2} \end{array} \right) \mid \begin{array}{l} f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f}) \\ \wedge \Pi_{i=1}^{\ell_1} ((\mathbf{y}_i - 1) \cdot \mathbf{f}_{ij} + 1) = \boldsymbol{\mu}_j \text{ for all } j \in [\ell_2] \end{array} \right\}.$$

We omit an explicit description of the polynomials $Q_{\text{Bit}\mathbb{Z}}$, since the fact that the above relation is a particular instance of the PESAT relation is not relevant in this paper.

Remark 4.3 (Using interleaved codes). [Albert: What is this remark for?](#) In practice, one may use as code $\mathcal{C}$ an ℓ_2 interleaving $\mathcal{C}_1^{\equiv \ell_2}$ of a code $\mathcal{C}_1$ of dimension ℓ_1/ν . Then, if ν divides ℓ_1 [Albert: add as requirement somewhere](#), then

$$\text{Pack}_\nu \circ \pi_2(\mathbf{f}) = (\text{Pack}_\nu \circ \pi_2(\mathbf{f}_{*j}))_{j \in [\ell_2]}$$

and

$$f = \text{Enc}_{\mathcal{C}_1^{\equiv \ell_2}} \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f}) = (\text{Enc}_{\mathcal{C}_1} \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f}_{*j}))_{j \in [\ell_2]} \in \mathbb{F}_{2^\nu}^m.$$

[Albert: If we do interleaving, the reason why we work with an interleaved \$\mathcal{R}_{\text{PESAT}}\(\mathbb{F}_{2^\nu}, Q_{\text{Bit}\mathbb{Z}}, \mathcal{C}_1^{\equiv \ell_2}, \text{Pack}_\nu\)\$ rather than \$\ell_2\$ copies of a non-interleaved PESAT is rather technical. It has to do with keeping the same distance parameter when doing the reduction from \$\mathcal{R}_{\text{Lin-BitsDual}}\$ to \$\mathcal{R}_{\text{Lin}}\$. If we simply considered separate encodings of \$\text{Pack}_\nu\(\mathbf{f}_{*j}\)\$, the proximity parameter in from \$\mathcal{R}_{\text{Lin}}\$ to \$\mathcal{R}_{\text{Lin-BitsDual}}\$ would grow by a factor of \$\ell_2\$.](#)

We assume prover and verifier have access to an IOR Π_{red} that reduces $\mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{Bit}\mathbb{Z}}, \mathcal{C}, \text{Pack}_\nu)$ to $\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu)$, where, recall, this latter relation has predicates of the form $\langle \mathbf{g}, \mathbf{u} \rangle = \mu'$ with $\mathbf{g}, \mathbf{u}$ vectors over $\mathbb{F}_{2^\nu}$, and $g = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu(\mathbf{g})$. In our instantiation Π_{red} is an optimized GKR protocol for grand products ??, which reduces the initial grand product claims to MLE evaluation claims of $\mathbf{y}$ and $\mathbf{f}$ over $\mathbb{F}_{2^\nu}$. As we explain in [Section 6.3](#), we design several optimizations for the specific shape of our grand products.

We assume Π_{red} works so that the verifier never queries $\mathbf{y}$; and that, given an input

$$(\mathbf{x} = (\mathbf{y}, \boldsymbol{\mu}), \mathbf{y} = f, \mathbf{w} = \pi_2(\mathbf{f})) \in \mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{Bit}\mathbb{Z}}, \mathcal{C}, \text{Pack}_\nu),$$

Π_{red} outputs

$$((\mathbf{v}, \mu'), f, \pi_2(\mathbf{f})) \in \mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu)$$

for some vector $\mathbf{v}$ and field element μ' . In particular, Π_{red} outputs the same oracle f and witness $\pi_2(\mathbf{f})$ as the oracle and witness it is given as inputs.

This is indeed the case with GKR: indeed, GKR reduces the initial circuit evaluation claim with a witness input, into an MLE evaluation claim of the same witness, without querying any oracle in-between.

4.3 An IOR for projecting the weights to a random prime field

The IOP for $\mathcal{R}_{\text{Lin-BitsDual}}$ that we present in Section 4.4 certifies linear claims by encoding the weights $\mathbf{v}$ as integer exponents of a generator g of the multiplicative group of $\mathbb{F}_{2^\nu}$. As we will see (cf. Theorem 4.10), this strategy requires the weights to be, or to lift to, integers in $[0..q-1]$ with $(\ell_1 + 1) \cdot (q - 1)$ smaller than the multiplicative order of g . In this section we give a one-round IOR that reduces $\mathcal{R}_{\text{Lin-BitsDual}}(\{0, 1\}, \pi, R, \mathcal{C})$, where R is an arbitrary finite field or $R = \mathbb{Z}$ with arbitrarily large weights, to $\mathcal{R}_{\text{Lin-BitsDual}}(\{0, 1\}, \pi_{q'}, \mathbb{F}_{q'}, \mathcal{C})$ for a randomly sampled prime q' fitting the aforementioned budget. The IOR leaves the oracle and the witness untouched: only the instance is updated. We believe this reduction is of independent interest, as it can be composed with any IOP for $\mathcal{R}_{\text{Lin-BitsDual}}$ over prime fields of bounded size, and not only ours.

We assume the notation from the previous two Sections 4.1 and 4.2. Given a projection $\pi_p : \mathbb{Z} \rightarrow \mathbb{F}_p$, we fix an inverse map of the form $\pi_p^{-1} : \mathbb{F}_p \rightarrow [0..p-1] \subseteq \mathbb{Z}$. Given a finite field $\mathbb{K}$ which is an extension of a prime field $\mathbb{F}_p$ of degree $\deg(\mathbb{K})$, we let $\pi_{\text{canon}}^{\mathbb{K}} : \mathbb{Z}[X] \rightarrow \mathbb{K}$ be the canonical homomorphism from $\mathbb{Z}[X]$ to $\mathbb{K}$: namely, if $\mathbb{K} = \mathbb{Z}[X]/(p, h(X))$ for a polynomial $h(X)$ irreducible over $\mathbb{F}_p[X]$, then $\pi_{\text{canon}}^{\mathbb{K}}$ consists in modding p out, and taking the quotient by $(h(X))$ (if $\mathbb{K}$ is the prime field $\mathbb{F}_p$, then $\pi_{\text{canon}}^{\mathbb{K}}$ is just π_p). We fix an inverse for $\pi_{\text{canon}}^{\mathbb{K}}$ of the form

$$\pi_{\text{canon}}^{\mathbb{K}}{}^{-1} : \mathbb{K} \rightarrow [0..p-1]^{\leq \deg(\mathbb{K})}[X] \subseteq \mathbb{Z}.$$

In words, $\pi_{\text{canon}}^{\mathbb{K}}{}^{-1}$ simply takes an element from $\mathbb{K}$ represented as a degree $< \deg(h(X))$ polynomial over $\mathbb{F}_p$, and applies π_p^{-1} to it. We remark that one could choose any other such “inverse” maps, e.g. for π_p^{-1} one may prefer to use centered representatives.

We define $\pi_{\text{canon}}^{\mathbb{Z}} : \mathbb{Z} \rightarrow \mathbb{Z}$ as the identity map. If $R = \mathbb{Z}$, we let $\deg(R) = 0$.

Throughout this and the next subsection, given a $\mathcal{R}_{\text{Lin-BitsDual}}(\{0, 1\}, \pi, R, \mathcal{C})$ instance with weight vector $\mathbf{v}$, the integer q denotes the characteristic of R if R is a finite field, or, if $R = \mathbb{Z}$, then

$$q = \ell_1 \cdot \|\mathbf{v}\|_{\infty} + 1. \quad (2)$$

Construction 4.4 (IOR for projecting $\mathcal{R}_{\text{Lin-BitsDual}}$ to a random prime field). **Parameters.** $(R, \pi, \mathbb{F}_{2^\nu}, \ell, \ell_1, \ell_2, \mathcal{C}, \mathcal{P})$ comprised of: a ring R (either $\mathbb{Z}$ or a finite field) and a ring homomorphism $\pi : \mathbb{Z} \rightarrow R$; a binary field $\mathbb{F}_{2^\nu}$; a witness length ℓ ; tensor decomposition dimensions ℓ_1, ℓ_2 satisfying $\ell_1 \cdot \ell_2 = \ell$; a linear code $\mathcal{C}$ over $\mathbb{F}_{2^\nu}$ of dimension ℓ/ν and block length m ; a set of primes $\mathcal{P}$.

Inputs. The prover and verifier receive $\mathbf{x} = (\mu, \mathbf{v}) \in R \times R^\ell$ and $\mathbf{y} = f \in \mathbb{F}_{2^\nu}^m$. The prover receives $\mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ such that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{Lin-BitsDual}}(\{0, 1\}, \pi, R, \mathcal{C})$. We assume the tensor split from Section 4.2, and adopt its notation.

1. **Round 1: linking the claim to a bounded integer claim, and projecting the latter to a random prime field.**

- If $R = \mathbb{Z}$, the verifier rejects unless $|\mu| \leq \ell \cdot \|\mathbf{v}\|_\infty$. Prover and verifier set $L^{(1)} := \mathbf{v}^{(1)}$, $L^{(2)} := \mathbf{v}^{(2)}$, and $\mu_{\mathbb{Z}} := \mu$; the prover sends an empty message.
- If R is a finite field, prover and verifier set $L^{(k)} := \pi_{\text{canon}}^R(\mathbf{v}^{(k)}) \in [0..q-1]^{\leq \deg(R)}[X]^{\ell_k}$ for $k \in \{1, 2\}$, and the prover sends $\mu_{\mathbb{Z}} \in [0.. \ell \cdot (\deg(R) + 1) \cdot (q-1)^2]^{\leq 2 \deg(R)}[X]$ such that, over $\mathbb{Z}[X]$,

$$\langle \mathbf{f}, L^{(1)} \otimes L^{(2)} \rangle = \mu_{\mathbb{Z}}. \quad (3)$$

The verifier checks that $\pi_{\text{canon}}^R(\mu_{\mathbb{Z}}) = \mu$ over R .

- The verifier uniformly samples and sends a prime q' from $\mathcal{P}$, and, if $\deg(R) > 0$, an element $\alpha \in \mathbb{F}_{q'}$ (if $\deg(R) = 0$, no α is sampled, and we understand $\pi_{q', \alpha}$ below as $\pi_{q'}$). **Albert: We should allow for sampling a vector of α 's rather than using powers of α . This would make one of the error terms of κ_0^* be $1_{\deg(R) > 0}/q_{\min}$ rather than a multiple of $\deg(R)$.**
- Prover and verifier output the instance and oracle $\mathbf{x}' := (\mu', \mathbf{v}'^{(1)} \otimes \mathbf{v}'^{(2)})$, $\mathbf{y}' := f$, where

$$\begin{aligned} \mathbf{v}'^{(k)} &:= \pi_{q', \alpha}(L^{(k)}) \in \mathbb{F}_{q'}^{\ell_k}, \quad k \in \{1, 2\}, \\ \mu' &:= \pi_{q', \alpha}(\mu_{\mathbb{Z}}) \in \mathbb{F}_{q'}. \end{aligned}$$

The oracle and the witness are left untouched: the honest prover outputs $\mathbf{w}' := \mathbf{f}$.

Note that the output instance of [Construction 4.4](#) depends on the sampled prime q' . Formally, the target relation of the IOR is $\bigcup_{q' \in \mathcal{P}} \mathcal{R}_{\text{Lin-BitsDual}}(\{0, 1\}, \pi_{q'}, \mathbb{F}_{q'}, \mathcal{C})$, with the sampled prime q' included in the output instance; for readability, we simply write $\mathcal{R}_{\text{Lin-BitsDual}}(\{0, 1\}, \pi_{q'}, \mathbb{F}_{q'}, \mathcal{C})$.

Theorem 4.5 (Completeness and round-by-round knowledge soundness of [Construction 4.4](#)). *Let $\delta \in (0, 1)$ be a proximity parameter. Then [Construction 4.4](#) is a perfectly complete 1-round IOR from $\mathcal{R}_{\text{Lin-BitsDual}}(\{0, 1\}, \pi, R, \mathcal{C})$ to $\mathcal{R}_{\text{Lin-BitsDual}}(\{0, 1\}, \pi_{q'}, \mathbb{F}_{q'}, \mathcal{C})$ with round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{Lin-BitsDual}}^\delta, \mathcal{R}_{\text{Lin-BitsDual}}^\delta)$, with error*

$$\kappa_0^* = \frac{2 \deg(R)}{q_{\min}} + \frac{\log(\ell + 1) + \log(\deg(R) + 1) + 2 \log(q - 1)}{|\mathcal{P}| \cdot \lfloor \log(q_{\min}) \rfloor},$$

where $q_{\min}$ is the smallest prime in $\mathcal{P}$.

To prove [Theorem 4.5](#) we begin with the following lemma, which is implicit in [\[Abd+26\]](#).

Lemma 4.6. *Let $h(X) \in \mathbb{Z}[X]$ be a nonzero polynomial of degree $\leq d$ and with coefficients in the range $[-2^B + 1..2^B - 1]$. Let $\mathcal{P}$ be a set of primes, and let $q_{\min}$ be the smallest prime in $\mathcal{P}$. Then*

$$\Pr_{\substack{q \leftarrow \mathcal{P}, \\ \alpha \leftarrow \mathbb{F}_q}} [\pi_{q, \alpha}(h(X)) = 0] \leq \frac{B}{\lfloor \log(q_{\min}) \rfloor \cdot |\mathcal{P}|} + \frac{d}{q_{\min}}.$$

Proof. Let $\mathcal{E}_{\text{prime}}$ be the event that $h(X)$ is a multiple of q . Then, by Schwartz-Zippel lemma,

$$\Pr_{\substack{q \leftarrow \mathcal{P}, \\ \alpha \leftarrow \mathbb{F}_q}} [\pi_{q, \alpha}(h(X)) = 0 \mid \neg \mathcal{E}_{\text{prime}}] \leq \frac{d}{q_{\min}}.$$

Now, every nonzero coefficient of $h(X)$ is divisible by at most $B/\lfloor \log(q_{\min}) \rfloor$ primes from $\mathcal{P}$. Hence, the probability that $\mathcal{E}_{\text{prime}}$ occurs is at most $(B/\lfloor \log(q_{\min}) \rfloor)/|\mathcal{P}|$, and the lemma follows. $\square$

Proof of Theorem 4.5. Perfect completeness. Assume $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{Lin-BitsDual}}(\{0, 1\}, \pi, R, \mathcal{C})$. If $R = \mathbb{Z}$, then $|\mu| = |\langle \mathbf{f}, \mathbf{v} \rangle| \leq \ell \cdot \|\mathbf{v}\|_\infty$ since $\mathbf{f}$ contains bits, so the verifier does not reject; and, since $\pi_{q'} : \mathbb{Z} \rightarrow \mathbb{F}_{q'}$ is a ring homomorphism, $\langle \pi_{q'}(\mathbf{f}), \mathbf{v}'^{(1)} \otimes \mathbf{v}'^{(2)} \rangle = \pi_{q'}(\langle \mathbf{f}, \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)} \rangle) = \pi_{q'}(\mu) = \mu'$. If R is a finite field, the honest $\mu_{\mathbb{Z}}$ from (3) has degree $\leq 2 \deg(R)$ and coefficients in $[0 \dots \ell \cdot (\deg(R) + 1) \cdot (q - 1)^2]$: indeed, each entry of $L^{(1)} \otimes L^{(2)}$ is a product of two polynomials of degree $\leq \deg(R)$ with coefficients in $[0 \dots q - 1]$, and $\mathbf{f}$ has at most ℓ nonzero (bit) entries. Moreover, since π_{canon}^R is a ring homomorphism satisfying $\pi_{\text{canon}}^R \circ \pi_{\text{canon}}^{R^{-1}} = \text{id}$, applying π_{canon}^R to (3) yields

$$\pi_{\text{canon}}^R(\mu_{\mathbb{Z}}) = \langle \pi(\mathbf{f}), \pi_{\text{canon}}^R(L^{(1)}) \otimes \pi_{\text{canon}}^R(L^{(2)}) \rangle = \langle \pi(\mathbf{f}), \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)} \rangle = \mu,$$

so the verifier's check passes. Similarly, since $\pi_{q', \alpha} : \mathbb{Z}[X] \rightarrow \mathbb{F}_{q'}$ is a ring homomorphism, applying $\pi_{q', \alpha}$ to (3) yields $\langle \pi_{q'}(\mathbf{f}), \mathbf{v}'^{(1)} \otimes \mathbf{v}'^{(2)} \rangle = \pi_{q', \alpha}(\mu_{\mathbb{Z}}) = \mu'$. In both cases $(\mathbf{x}', \mathbf{y}', \mathbf{f}) \in \mathcal{R}_{\text{Lin-BitsDual}}(\{0, 1\}, \pi_{q'}, \mathbb{F}_{q'}, \mathcal{C})$, as needed.

Round-by-round knowledge soundness. We describe a knowledge state function KState . We start by setting KState so that the first two conditions in Definition 3.6 are met. Precisely, if $\text{tr} = \perp$, we set $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{Lin-BitsDual}}(\{0, 1\}, \pi, R, \mathcal{C})^\delta$; and we have KState output zero on inputs $(\mathbf{x}, \mathbf{y}, \text{tr}, m, \mathbf{w})$ where m is a prover message and $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 0$. After the verifier's message, $\text{tr} = (\mu_{\mathbb{Z}}, q', \alpha)$ is a full transcript, and we define $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if the verifier does not reject and $(\mathbf{x}', \mathbf{y}', \mathbf{w}) \in \mathcal{R}_{\text{Lin-BitsDual}}(\{0, 1\}, \pi_{q'}, \mathbb{F}_{q'}, \mathcal{C})^\delta$, where $(\mathbf{x}', \mathbf{y}')$ is the output of the verifier; this way the condition “Full transcript” in Definition 3.6 is met as well. We define $\mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \mu_{\mathbb{Z}}, q', \alpha, \mathbf{w}) = \mathbf{w}$. We next show that

$$\Pr_{q', \alpha} \left[\exists \mathbf{w} \mid \begin{array}{l} \wedge \text{KState}(\mathbf{x}, \mathbf{y}, \mu_{\mathbb{Z}}, \mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \mu_{\mathbb{Z}}, q', \alpha, \mathbf{w})) = 0 \\ \wedge \text{KState}(\mathbf{x}, \mathbf{y}, \mu_{\mathbb{Z}}, q', \alpha, \mathbf{w}) = 1 \end{array} \right] \leq \kappa_0^*.$$

Assume

$$\text{KState}(\mathbf{x}, \mathbf{y}, \mu_{\mathbb{Z}}, q', \alpha, \mathbf{w}) = 1 \quad \text{and} \quad \text{KState}(\mathbf{x}, \mathbf{y}, \mu_{\mathbb{Z}}, \mathbf{w}) = 0.$$

The latter occurs if and only if $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \notin \mathcal{R}_{\text{Lin-BitsDual}}(\{0, 1\}, \pi, R, \mathcal{C})^\delta$. The former gives $\mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell$ with $\Delta(f, \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f})) < \delta$ and

$$\langle \pi_{q'}(\mathbf{f}), \mathbf{v}'^{(1)} \otimes \mathbf{v}'^{(2)} \rangle = \mu' \quad \text{over } \mathbb{F}_{q'}. \quad (4)$$

Hence necessarily $\langle \pi(\mathbf{f}), \mathbf{v} \rangle \neq \mu$ over R . Define

$$P(X) := \langle \mathbf{f}, L^{(1)} \otimes L^{(2)} \rangle - \mu_{\mathbb{Z}} \in \mathbb{Z}[X].$$

We claim that $P \neq 0$. Indeed, otherwise, applying the ring homomorphism π_{canon}^R to $\langle \mathbf{f}, L^{(1)} \otimes L^{(2)} \rangle = \mu_{\mathbb{Z}}$, as in the completeness argument, yields $\langle \pi(\mathbf{f}), \mathbf{v} \rangle = \pi_{\text{canon}}^R(\mu_{\mathbb{Z}}) = \mu$, where the latter equality is the verifier's check (for $R = \mathbb{Z}$, note directly $P = \langle \mathbf{f}, \mathbf{v} \rangle - \mu \neq 0$), a contradiction. Now, P has degree $\leq 2 \deg(R)$, and its coefficients have absolute value at most $\ell \cdot (\deg(R) + 1) \cdot (q - 1)^2$ (at most $2\ell \cdot \|\mathbf{v}\|_\infty$ if $R = \mathbb{Z}$, by the verifier's bound check on μ); in both cases, at most 2^B for $B := \log(\ell + 1) + \log(\deg(R) + 1) + 2 \log(q - 1)$, using (2) in the latter case. Finally, applying the ring homomorphism $\pi_{q', \alpha}$ to P and using (4),

$$\pi_{q', \alpha}(P) = \langle \pi_{q'}(\mathbf{f}), \mathbf{v}'^{(1)} \otimes \mathbf{v}'^{(2)} \rangle - \mu' = 0.$$

By Lemma 4.6,

$$\Pr_{q', \alpha}[\pi_{q', \alpha}(P(X)) = 0] \leq \frac{\log(\ell + 1) + \log(\deg(R) + 1) + 2\log(q - 1)}{\lfloor \log(q_{\min}) \rfloor \cdot |\mathcal{P}|} + \frac{2\deg(R)}{q_{\min}} = \kappa_0^*,$$

as needed. $\square$

Remark 4.7 (Using any module homomorphism instead of $\pi_{q', \alpha}$). [Albert: todo](#)

4.4 The $\mathbb{F}_2\mathbb{Z}$ IOP for the core Lin-BitsDual relation

We provide an IOP for $\mathcal{R}_{\text{Lin-BitsDual}}$ next. We let Π_{iopp} be an IOP for $\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu)$. In our instantiation, Π_{iopp} is the WHIR protocol [ACFY25] over binary fields, with ring-switching [?] as described in Section B [Albert: proper reference?](#), and with proximity parameter set to the Johnson bound.

Construction 4.8 (IOP for the core $\mathcal{R}_{\text{Lin-BitsDual}}$ relation). **Parameters.** $(R, \pi, \mathbb{F}_{2^\nu}, \ell, \ell_1, \ell_2, \mathcal{C})$ comprised of: a ring R and a ring homomorphism $\pi : \mathbb{Z} \rightarrow R$; a binary field $\mathbb{F}_{2^\nu}$ a witness length ℓ ; tensor decomposition dimensions ℓ_1, ℓ_2 for [Construction 4.8](#) satisfying $\ell_1 \cdot \ell_2 = \ell$; a linear code $\mathcal{C}$ over $\mathbb{F}_{2^\nu}$ of dimension ℓ/ν and block length m .

Let $\mathcal{P}$ be a set of primes with $\max\{\mathcal{P}\} \leq (|\mathbb{F}_{2^\nu}| - 1)/(\ell_1 + 1)$.

Inputs. The prover and verifier receive $\mathbf{x} = (\mu, \mathbf{v}) \in R \times R^\ell$ and $\mathbf{y} = f \in \mathbb{F}_{2^\nu}^m$. The prover receives $\mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ such that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{Lin-BitsDual}}(\{0, 1\}, \pi, R, \mathcal{C})$. We assume the tensor split from Section 4.2, and adopt its notation.

Let $g \in \mathbb{F}_{2^\nu}$ be the generator of the multiplicative subgroup of $\mathbb{F}_{2^\nu}$, and let $\text{ord}(g)$ denote its multiplicative order. Recall the integer q associated with R and $\mathbf{v}$, defined at the beginning of [Section 4.3](#).

1. **Round 1: transforming the constraint over R to an integer constraint with bounded integers.**

- **Optional step: projecting the claim to a random prime field.** The following step triggers in the following scenarios:

- $\deg(R) > 0$.
- $\deg(R) = 0$ and $(\ell_1 + 1) \cdot (q - 1) \geq \text{ord}(g)$.

Prover and verifier run the IOR from [Construction 4.4](#) with parameters $(R, \pi, \mathbb{F}_{2^\nu}, \ell, \ell_1, \ell_2, \mathcal{C}, \mathcal{P})$ on the input $(\mathbf{x}, \mathbf{y})$ (the prover additionally uses $\mathbf{w} = \mathbf{f}$), and continue the present protocol on the output of the IOR. That is, with $q', \alpha, \mathbf{v}'^{(1)}, \mathbf{v}'^{(2)}, \mu'$ as in [Construction 4.4](#), update

$$\begin{aligned} \mathbf{v}^{(k)} &\leftarrow \mathbf{v}'^{(k)} \in \mathbb{F}_{q'}^{\ell_k}, & k \in \{1, 2\}, \\ \mu &\leftarrow \mu' \in \mathbb{F}_{q'}, \\ R &\leftarrow \mathbb{F}_{q'}, & \pi &\leftarrow \pi_{q'}, & q &\leftarrow q'. \end{aligned}$$

Note that after this step $\deg(R) = 0$ and $q = q' \leq \max \mathcal{P}$.

- The prover sends $\mu_1, \dots, \mu_{\ell_2} \in [0.. \ell_1 \cdot (q - 1)] \subseteq \mathbb{Z}$ such that, over $\mathbb{Z}$,

$$\langle \mathbf{f}_{*j}, \pi_{\text{canon}}^R{}^{-1}(\mathbf{v}^{(1)}) \rangle = \mu_j \quad \text{for all } j \in [\ell_2]. \quad (5)$$

- The verifier checks that

$$\sum_{j \in [\ell_2]} \pi_{\text{canon}}^R(\mu_j) \cdot \mathbf{v}_j^{(2)} = \mu \quad \text{over } R. \quad (6)$$

Next, prover and verifier will focus on certifying that (5) holds for all $j \in [\ell_2]$.

In Remark 4.9 we describe a variation in which, instead of sending the elements $\mu_1, \dots, \mu_{\ell_2}$, the prover commits to their bit-decomposition over $\mathbb{F}_{2^\nu}$, and proves (6) using the present IOR. This saves on proof size and verifier time when ℓ is large.

2. Phase 2: Transforming the claim into a PESAT claim over $\mathbb{F}_{2^\nu}$, and invoking the IOR Π_{red} .

- Note that, at this point of the protocol, $\deg(R) = 0$, i.e. R is either $\mathbb{Z}$ or a prime field $\mathbb{F}_q$. Prover and verifier define

$$\gamma = \pi_q^{-1}(\mathbf{v}^{(1)}) \in [0..q-1]^{\ell_1} \subseteq \mathbb{Z}^{\ell_1},$$

and set to prove the following claim:

$$\prod_{i \in [\ell_1]} g^{\gamma_i \cdot \mathbf{f}_{ij}} = g^{\mu_j} \in \mathbb{F}_{2^\nu}, \quad \text{for all } j \in [\ell_2].$$

This claim is equivalent to

$$\prod_{i \in [\ell_1]} ((g^{\gamma_i} - 1) \cdot \pi_2(\mathbf{f}_{ij}) + 1) = g^{\mu_j} \in \mathbb{F}_{2^\nu}, \quad \text{for all } j \in [\ell_2].$$

Notice that

$$(\mathbf{x} = ((g^{\gamma_i})_{i \in [\ell_1]}, (g^{\mu_j})_{j \in [\ell_2]}), \mathbf{y} = f, \mathbf{w} = \pi_2(\mathbf{f})) \in \mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{BitZ}}, \mathcal{C}, \text{Pack}_\nu). \quad (7)$$

- To prove (7), prover and verifier engage in the IOR Π_{red} ² with input

$$(\mathbf{x} = ((g^{\gamma_i})_{i \in [\ell_1]}, (g^{\mu_j})_{j \in [\ell_2]}), \mathbf{y} = f, \mathbf{w} = \pi_2(\mathbf{f})).$$

At the end of this IOR, they are left with the following claim to prove:

$$\begin{aligned} & ((\mathbf{v}, \mu'), f, \pi_2(\mathbf{f})) \in \mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu) \\ &= \left\{ \left(\begin{array}{l} \mathbf{x} = (\mathbf{u}, \mu) \in \mathbb{F}_{2^\nu}^k, \\ \mathbf{y} = g \in \mathbb{F}_{2^\nu}^m, \\ \mathbf{w} = \mathbf{g} \in (\mathbb{F}_2)^\ell \end{array} \right) \middle| \begin{array}{l} g = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu(\mathbf{g}) \\ \langle \mathbf{u}, \mathbf{g} \rangle = \mu \end{array} \right\} \end{aligned}$$

for some vector $\mathbf{v}$ and field element μ' output during the IOR Π_{red} .

3. Phase 3: Using an IOP over $\mathbb{F}_{2^\nu}$. Prover and verifier engage in an IOP to certify the claims (??).

²Recall that, in our instantiation, Π_{red} is the GKR protocol applied to multiple grand products, cf. ??

Remark 4.9 (Reducing proof size by committing to the elements $\mu_1, \dots, \mu_{\ell_2}$). When $\ell = \ell_1 \cdot \ell_2$ is such that ℓ_2 is large, say $\ell \geq 2^{11}$, the elements $\mu_1, \dots, \mu_{\ell_2} \in \mathbb{Z}$ sent at Round 1 of [Construction 4.8](#) may start to dominate the proof size. For example, for $\ell = 2^{13}$ and μ_i of 128 bits, the elements $\mu_1, \dots, \mu_{\ell_2}$ amount to 131 KB. To mitigate this cost, one can have the prover commit to the vector $(\mu_1, \dots, \mu_{\ell_2})$ and then prove (6), i.e.

$$\langle \mathbf{f}_{*j}, \pi_{\text{canon}}^R \cdot \mathbf{v}^{(1)} \rangle = \mu_j \quad \text{for all } j \in [\ell_2].$$

using an IOP for constrained codes. Concretely, the prover commits to the vector $\text{bits}(\mu)$ of bits forming the elements $(\mu_1, \dots, \mu_{\ell_2})$ using $\mathbb{F}_2\mathbb{Z}$, and then proves (6) using [Construction 4.8](#).

4.5 Soundness and completeness

For readability, we temporarily denote

$$\mathcal{R}_{\text{PESAT}} = \mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{Bit}\mathbb{Z}}, \mathcal{C}, \text{Pack}_\nu), \quad \mathcal{R}_{\text{Lin}} = \mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu).$$

Theorem 4.10 (Completeness and round-by-round knowledge soundness of [Construction 4.8](#)). *Albert: Completeness missing* Let $\delta \in (0, 1)$ be a proximity parameter. Assume g has multiplicative order $\text{ord}(g)$ with

$$\text{ord}(g) > (\ell_1 + 1) \cdot (\max \mathcal{P} - 1).$$

Assume Π_{red} is a perfectly complete t_{red} -round IOR from $\mathcal{R}_{\text{PESAT}}$ to $\mathcal{R}_{\text{Lin}}$ with round-by-round knowledge soundness with relaxation $\mathcal{R}_{\text{PESAT}}^\delta, \mathcal{R}_{\text{Lin}}^\delta$, with errors $(\kappa_{\text{red}1}, \dots, \kappa_{\text{red}t_{\text{red}}})$. Assume Π_{iopp} is a perfectly complete t_{iopp} -round IOP from $\mathcal{R}_{\text{Lin}}$ (i.e. an IOR from $\mathcal{R}_{\text{Lin}}$ to the trivial relation $\perp$) with round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{Lin}}^\delta, \perp)$, and errors

$$\kappa_{\text{iopp}1}, \dots, \kappa_{\text{iopp}t_{\text{iopp}}}.$$

Then, if Round 1 of [Construction 4.8](#) invokes the IOR from [Construction 4.4](#) (i.e. if $\deg(R) > 0$, or $(\ell_1 + 1) \cdot (q - 1) \geq \text{ord}(g)$), [Construction 4.8](#) is a perfectly complete $(1 + t_{\text{red}} + t_{\text{iopp}})$ -round IOP for $\mathcal{R}_{\text{Lin-BitsDual}}$ with round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{Lin-BitsDual}}^\delta, \perp)$, with errors

$$\kappa_0^*, (\kappa_{\text{red}i})_{i \in [t_{\text{red}}]}, (\kappa_{\text{iopp}i})_{i \in [t_{\text{iopp}}]},$$

where κ_0^* is the error from [Theorem 4.5](#). Recall that q is either the characteristic of R if R is a finite field, or $q = \ell_1 \cdot \|\mathbf{v}\|_\infty + 1$ if $R = \mathbb{Z}$.

Otherwise, i.e. if $\deg(R) = 0$ and $(\ell_1 + 1) \cdot (q - 1) < \text{ord}(g)$, [Construction 4.8](#) is a perfectly complete $(t_{\text{red}} + t_{\text{iopp}})$ -round IOP with round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{Lin-BitsDual}}^\delta, \perp)$, with errors $(\kappa_{\text{red}i})_{i \in [t_{\text{red}}]}, (\kappa_{\text{iopp}i})_{i \in [t_{\text{iopp}}]}$.

Proof of Theorem 4.10. We decompose [Construction 4.8](#) into IORs. When the optional step of Round 1 triggers, the first IOR, Π_0 , is the IOR from [Construction 4.4](#); by [Theorem 4.5](#), it is perfectly complete and has round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{Lin-BitsDual}}^\delta, \mathcal{R}_{\text{Lin-BitsDual}}^\delta)$ with error κ_0^* . The next IOR, Π_1 , consists of the remainder of Round 1 of [Construction 4.8](#), and outputs

$$(\mathbf{x}' = ((g^{\gamma_i})_{i \in [\ell_1]}, (g^{\mu_j})_{j \in [\ell_2]}), \mathbf{y}' = f, \mathbf{w}' = \pi_2(\mathbf{f})),$$

as described at the beginning of Phase 2 of [Construction 4.8](#). The next IOR, Π_2 , is simply Π_{red} , reducing $\mathcal{R}_{\text{PESAT}}$ to $(\mathcal{R}_{\text{Lin}})$. Finally, the last IOR, Π_3 , is simply the IOP Π_{iopp} for $\mathcal{R}_{\text{Lin}}$.

By [Theorem 4.5](#) and the composition of IORs result (cf. [Remark 3.8](#)), it suffices to show that Π_1 is a perfectly complete IOR from $\mathcal{R}_{\text{Lin-BitsDual}}(\{0, 1\}, \pi, R, \mathcal{C})$ to $\mathcal{R}_{\text{PESAT}}$ with round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{Lin-BitsDual}}^\delta, \mathcal{R}_{\text{PESAT}}^\delta)$ and an empty sequence of errors. Here and below, $R, \pi, q, \mathbf{v}^{(1)}, \mathbf{v}^{(2)}$ and μ denote the (possibly updated) objects after the optional step of Round 1. In all cases, R is $\mathbb{Z}$ or a prime field, and, by the trigger condition of the optional step and the hypothesis $\text{ord}(g) > (\ell_1 + 1) \cdot (\max \mathcal{P} - 1)$,

$$(\ell_1 + 1) \cdot (q - 1) < \text{ord}(g). \quad (8)$$

Completeness and soundness of Π_1 The IOR Π_1 contains no verifier messages: the prover sends $\mu_1, \dots, \mu_{\ell_2}$, and the verifier performs the check (6) and outputs the $\mathcal{R}_{\text{PESAT}}$ instance above. Hence its sequence of round-by-round errors is empty, and it suffices to exhibit a knowledge state function KState and an extractor satisfying [Definition 3.6](#). We take the extractor to be the identity map on the knowledge state witness, and define KState as follows. If $\text{tr} = \perp$, then $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{Lin-BitsDual}}^\delta$. If $\text{tr} = (\mu_1, \dots, \mu_{\ell_2})$ (a full transcript of Π_1), then $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if the verifier does not reject and $(\mathbf{x}', \mathbf{y}', \mathbf{w}) \in \mathcal{R}_{\text{PESAT}}^\delta$, where $(\mathbf{x}', \mathbf{y}')$ is the output of the verifier. This way, the conditions “Empty transcript” and “Full transcript” of [Definition 3.6](#) hold by definition. It remains to verify the condition “Prover moves”: namely, that if $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \notin \mathcal{R}_{\text{Lin-BitsDual}}^\delta$, then, for every prover message $\boldsymbol{\mu} = (\mu_1, \dots, \mu_{\ell_2})$, either the verifier rejects, or $(\mathbf{x}', \mathbf{y}', \mathbf{w}) \notin \mathcal{R}_{\text{PESAT}}^\delta$.

Assume that the check (6) passes and that $(\mathbf{x}', \mathbf{y}', \mathbf{w}) \in \mathcal{R}_{\text{PESAT}}^\delta$, with $\mathbf{w} = \mathbf{f} = (\mathbf{f}_{*j})_{j \in [\ell_2]} \in \{0, 1\}^\ell$; we show that then $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{Lin-BitsDual}}^\delta$. First, membership in $\mathcal{R}_{\text{PESAT}}^\delta$ gives $\Delta(f, \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f})) < \delta$. Second, for every $j \in [\ell_2]$,

$$g^{\langle \mathbf{f}_{*j}, \boldsymbol{\gamma} \rangle} = g^{\mu_j},$$

which holds if and only if $\langle \mathbf{f}_{*j}, \boldsymbol{\gamma} \rangle \equiv \mu_j \pmod{\text{ord}(g)}$. Now, since $\gamma_i \in [0..q-1]$ and $\mathbf{f}_{*j}$ contains bits, we have $\langle \mathbf{f}_{*j}, \boldsymbol{\gamma} \rangle \in [0.. \ell_1 \cdot (q-1)]$, and also $\mu_j \in [0.. \ell_1 \cdot (q-1)]$, so that $|\langle \mathbf{f}_{*j}, \boldsymbol{\gamma} \rangle - \mu_j| \leq (\ell_1 + 1) \cdot (q-1)$. Hence, by (8),

$$\langle \mathbf{f}_{*j}, \boldsymbol{\gamma} \rangle = \mu_j \quad \text{over } \mathbb{Z}, \quad \text{for all } j \in [\ell_2].$$

Applying π on both sides, and using that $\pi(\boldsymbol{\gamma}) = \mathbf{v}^{(1)}$ and that π and π_{canon}^R agree on $\mathbb{Z}$, we obtain $\langle \pi(\mathbf{f}_{*j}), \mathbf{v}^{(1)} \rangle = \pi_{\text{canon}}^R(\mu_j)$ over R for all $j \in [\ell_2]$. Combining with the check (6),

$$\langle \pi(\mathbf{f}), \mathbf{v} \rangle = \sum_{j \in [\ell_2]} \langle \pi(\mathbf{f}_{*j}), \mathbf{v}^{(1)} \rangle \cdot \mathbf{v}_j^{(2)} = \sum_{j \in [\ell_2]} \pi_{\text{canon}}^R(\mu_j) \cdot \mathbf{v}_j^{(2)} = \mu \quad \text{over } R.$$

Hence $(\mathbf{x}, \mathbf{y}, \mathbf{f}) \in \mathcal{R}_{\text{Lin-BitsDual}}^\delta$, as needed. Perfect completeness of Π_1 follows from the same equivalences: for the honest μ_j from (5), the check (6) passes, and the output triple lies in $\mathcal{R}_{\text{PESAT}}$. $\square$

4.6 $\mathbb{F}2\mathbb{Z}$: an IOP for the general Lin-BitsDual relation

Next we describe our IOP for $\mathcal{R}_{\text{Lin-BitsDual}}$. We assume the notation of ???. Namely, M denotes an R_M -module; R is a ring; and $\psi : M \rightarrow R$ is a module homomorphism with underlying ring homomorphism $\pi : R_M \rightarrow R$. Let $\gamma_1, \dots, \gamma_d$ be a R_M -base of M .

We let ℓ_1, ℓ_2 be tensor decomposition parameters (to be used in [Construction 4.8](#)) be such that $\ell_1 \cdot \ell_2 = \ell_0 \cdot d \cdot B$. We assume that $\mathcal{C}$ is a linear code over $\mathbb{F}_{2^\nu}$ of dimension $\ell_0 \cdot d \cdot B/\nu$.

We will need the following result.

Lemma 4.11. Let $\mathbf{f} \in M_{\leq B}^{\ell_0}$, and let $\mathbf{v} \in R^{\ell_0}$. Then

$$\langle \psi(\mathbf{f}), \mathbf{v} \rangle = \langle \pi(\text{bits}(\mathbf{f})), \mathbf{u} \rangle$$

where

$$\mathbf{u} := (\mathbf{v} \otimes (1, \pi(2), \dots, \pi(2^{B-1})) \otimes (\psi(\gamma_1), \dots, \psi(\gamma_d))) \in R^{\ell_0 \cdot d \cdot B} \quad (9)$$

Proof. Rewriting $\langle \psi(\mathbf{f}), \mathbf{v} \rangle$, we obtain [Albert: add more detail?](#)

$$\langle \psi(\mathbf{f}), \mathbf{v} \rangle = \langle \psi \left(\sum_{\substack{k \in [B], \\ j \in [d]}} \mathbf{f}_{*jk} \cdot 2^{k-1} \cdot \gamma_j \right), \mathbf{v} \rangle \quad (10)$$

$$= \left\langle \sum_{\substack{k \in [B], \\ j \in [d]}} \pi(\mathbf{f}_{*jk}) \cdot \pi(2^{k-1}) \cdot \psi(\gamma_j), \mathbf{v} \right\rangle, \quad (11)$$

$$= \sum_{\substack{k \in [B], \\ j \in [d]}} \pi(2^{k-1}) \cdot \psi(\gamma_j) \cdot \langle \pi(\mathbf{f}_{*jk}), \mathbf{v} \rangle$$

$$= \sum_{\substack{k \in [B], \\ j \in [d]}} \sum_{i \in [\ell_0]} \pi(2^{k-1}) \cdot \psi(\gamma_j) \cdot \pi(\mathbf{f}_{*jk}), \mathbf{v}_i = \langle \pi(\text{bits}(\mathbf{f})), \mathbf{u} \rangle,$$

as needed. $\square$

Construction 4.12 (IOP for $\mathcal{R}_{\text{Lin-BitsDual}}$). **Parameters.**

$$(R_M, R, M, d, \pi, \psi, (\gamma_1, \dots, \gamma_d), \ell_0, B, \ell_1, \ell_2, \mathbb{F}_{2^\nu}, \mathcal{C})$$

comprised of: two rings R_M, R ; a ring morphism $\pi : R_M \rightarrow R$; a R_M -module M of dimension d ; a module homomorphism $\psi : M \rightarrow R$; a R_M -base $\gamma_1, \dots, \gamma_d$ of M_R ; a witness length ℓ_0 ; a bit-size bound B ; tensor decomposition dimensions ℓ_1, ℓ_2 for [Construction 4.8](#) satisfying $\ell_1 \cdot \ell_2 = \ell_0 \cdot d \cdot B / \nu$; a binary field $\mathbb{F}_{2^\nu}$; a linear code $\mathcal{C}$ over $\mathbb{F}_{2^\nu}$ of dimension $\ell_0 \cdot d \cdot B / \nu$ and block length m .

Inputs. The prover and verifier receive $\mathbf{x} = (\mathbf{v}, \mu) \in R^{\ell_0} \times R$ and $\mathbf{y} = f \in \mathbb{F}_{2^\nu}^m$. The prover receives $\mathbf{w} = \mathbf{f} \in M_{\leq B}^{\ell_0}$ such that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{Lin-BitsDual}}(M_{\leq B}, \psi, R, \mathcal{C})$, i.e.,

$$f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\text{bits}(\mathbf{f})), \quad \langle \psi(\mathbf{f}), \mathbf{v} \rangle = \mu.$$

1. By [Lemma 4.11](#), the claim $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{Lin-BitsDual}}(M_{\leq B}, \psi, R, \mathcal{C})$ is equivalent to the claim

$$(\mathbf{x}' = (\mathbf{u}, \mu), \mathbf{y}' = f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\text{bits}(\mathbf{f})), \mathbf{w}' = \text{bits}(\mathbf{f})) \in \mathcal{R}_{\text{Lin-BitsDual}}(\{0, 1\}, \pi, R, \mathcal{C}). \quad (12)$$

where $\mathbf{u} = (\mathbf{v} \otimes (1, \pi(2), \dots, \pi(2^{B-1})) \otimes (\psi(\gamma_1), \dots, \psi(\gamma_d)))$. Accordingly, prover and verifier use [Construction 4.8](#) to prove (12). They use parameters $(\mathbb{K}, q, \mathbb{F}_{2^\nu}, \ell_0 \cdot d \cdot B, \ell_1, \ell_2, \mathcal{C})$. See [Remark 4.13](#) for comments on how to use tensor decompositions of $\mathbf{u}$ in [Construction 4.8](#).

Remark 4.13 (Tensor decompositions of $\mathbf{u}$). Recall that [Construction 4.8](#) considers a (possibly trivial) tensor decomposition of its public input vector $\mathbf{u}$ (denoted $\mathbf{v}$ in [Construction 4.8](#)). This decomposition need not be given by some of the tensor factors in [Eq. \(9\)](#). For example, in many applications, the input vector $\mathbf{v}$ for $\mathcal{R}_{\text{Lin-BitsDual}}(M_{\leq B}, \psi, R, \mathcal{C})$ may admit further tensor decompositions, say $\mathbf{v} = \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)}$ (e.g. if $\mathbf{v}$ is of the form $(\text{eq}(\mathbf{b}, \mathbf{r}))_{\mathbf{b} \in \{0,1\}^{\log \ell_0}}$). In that case, we may use a tensor decomposition for $\mathbf{u}$ in [Construction 4.8](#) of the form, say, $\mathbf{u} = \mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)}$, where

$$\mathbf{u}^{(1)} := \mathbf{v}^{(1)}, \quad \mathbf{u}^{(2)} := \mathbf{v}^{(2)} \otimes (1, \pi(2), \dots, \pi(2^{B-1})) \otimes (\psi(\gamma_1), \dots, \psi(\gamma_d)).$$

See ?? for comments on how to choose close-to-optimal tensor split dimensions for $\mathbf{u}^{(1)}$ and $\mathbf{u}^{(2)}$.

Theorem 4.14. *Under the same hypotheses and notation as in [Theorem 4.10](#), if Round 1 of [Construction 4.8](#) invokes the IOR from [Construction 4.4](#) (i.e. if $\deg(R) > 0$, or $(\ell_1 + 1) \cdot (q - 1) \geq \text{ord}(g)$), [Construction 4.12](#) is a perfectly complete $1 + t_{\text{red}} + t_{\text{iopp}}$ IOP for $\mathcal{R}_{\text{Lin-BitsDual}}(M_{\leq B}, \psi, \mathbb{K}, \mathcal{C})$ with round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{Lin-BitsDual}}^\delta, \perp)$ and errors*

$$\kappa_0^*, (\kappa_{\text{red}i})_{i \in [t_{\text{red}}]}, (\kappa_{\text{iopp}i})_{i \in [t_{\text{iopp}}]},$$

where κ_0^* is the error from [Theorem 4.5](#). Otherwise, [Construction 4.12](#) is a $t_{\text{red}} + t_{\text{iopp}}$ IOP with round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{Lin-BitsDual}}^\delta, \perp)$, with errors $(\kappa_{\text{red}i})_{i \in [t_{\text{red}}]}, (\kappa_{\text{iopp}i})_{i \in [t_{\text{iopp}}]}$.

Proof. Deferred to [Section A.1](#). In short, the result is a consequence of [Lemma 4.11](#), which establishes an equivalence between triples for $\mathcal{R}_{\text{Lin-BitsDual}}(M_{R \leq B}, \psi, R, \mathcal{C})$ and $\mathcal{R}_{\text{Lin-BitsDual}}(\{0, 1\}, \pi, R, \mathcal{C})$. [Lemma 4.11](#) further provides a method for efficiently computing one triple from the other. This allows to build a round-by-round knowledge soundness state and an extractor for [Construction 4.12](#) from those of [Construction 4.8](#). $\square$

4.7 Instantiation

We instantiate [Construction 4.8](#) by letting Π_{red} be an optimized GKR protocol for multiple grand product claims of certain form. The IOP Π_{iopp} is taken to be WHIR [\[ACFY25\]](#) with the ring switching step from ??.

We next describe how to instantiate [Constructions 4.8](#) and [4.12](#) so as to achieve $\lambda = 100$ or $\lambda = 128$ bits of security, while keeping the protocols as efficient as possible.

In terms of security, we need to ensure the following:

1. $|\mathbb{F}_{2^\nu}| - 1 > (\ell_1 + 1) \cdot (\max \mathcal{P} - 1)$. A requisite in our [Theorems 4.10](#) and [4.14](#), to, intuitively, ensure that the inner product in the exponent of $g \in \mathbb{F}_{2^\nu}$ does not overflow.
2. The round-by-round knowledge soundness errors of [Construction 4.8](#) $\kappa_0^*, (\kappa_{i,\text{red}})_i, (\kappa_{i,\text{iopp}})_i$ are all at most $2^{-\lambda}$.

In terms of efficiency, the most important step of [Construction 4.8](#) to be aware of is Phase 2, where one uses an IOR Π_{red} to reduce the claim

$$\prod_{i \in [\ell_1]} ((g^{\gamma_i} - 1) \cdot \pi_2(\mathbf{f}_{ij}) + 1) = g^{\mu_j} \quad \in \mathbb{F}_{2^\nu}, \quad \text{for all } j \in [\ell_2],$$

into a linear claim on $\pi(\mathbf{f})$. As mentioned, in our instantiation, Π_{red} is the GKR protocol for multiple grand product claims, equipped with the optimizations discussed in ??. This is the most expensive

prover part of [Constructions 4.8](#) and [4.12](#), constituting between 50% to 66% of prover costs, cf. [??](#). Because of this, it is important for efficiency that operating in $\mathbb{F}_{2^\nu}$ is as efficient as possible.

With this in mind, we target using $\mathbb{F}_{2^\nu}$ with $\nu \approx 128$. A natural candidate is the 2^{128} sized binary field in GHASH representation [??](#)

$$\mathbb{F}_{2^\nu} = \mathbb{F}_2[X]/(X^{128} + X^7 + X^2 + X + 1).$$

Albert: Explain why it is performant. Discuss b127 too.

Before anything else, we discuss the constraint $|\mathbb{F}_{2^\nu}| - 1 > (\ell_1 + 1) \cdot (\max \mathcal{P} - 1)$. First of all, we set ℓ_1, ℓ_2 with $\ell_1 \cdot \ell_2 = \ell$ according to the following rule of thumb:

$$\log(\ell_1) \approx 0.6 \cdot \log(\ell), \quad \log(\ell_2) \approx 0.4 \cdot \log(\ell)$$

This split was chosen through experimental observation, being the split that consistently leads to roughly the best balance in prover and verifier time, and proof size.

To ground our analysis, we assume $\ell \leq 2^{35}$ (so the witness consists of $\leq 2^{35}$ bits), say, and $\ell_1 \leq 2^{14}$ following our rule of thumb. This leaves us with

$$\max \mathcal{P} \leq 2^{114.0001} = 2^{114+\varepsilon}.$$

We now discuss how to make the errors $\kappa_0^*, (\kappa_{i,\text{red}})_i, (\kappa_{i,\text{iopp}})_i$ be all at most $2^{-\lambda}$.

The errors $\kappa_{i,\text{iopp}}$ Regarding Phase 3, we instantiate our IOP for $\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \text{Pack}_\nu, \mathcal{C})$ as the WHIR protocol, with the ring switching from [??](#) **Albert:** does ring switching add new soundness errors?

Albert: We need a more formal description of the combination of the ring switch and WHIR? (I.e. inside a construction environment). We assume it is configured at λ bits of security, and so $\kappa_{i,\text{iopp}} < 2^{-\lambda}$. We do not discuss this further since WHIR is a well-known IOP **Albert:** should we, because we run it on 128 bit field, so 128-bits of security there is not common practice (i.e. people use larger fields?).

The errors $\kappa_{i,\text{red}}$. As mentioned, our Π_{red} IOR is the GKR protocol for multiple grand product claims from [??](#). Accordingly we have $\varepsilon_{i,\text{red}} \leq 3/|\mathbb{F}_{2^\nu}|$. This immediately guarantees $\lambda = 100$ bits of security. To achieve $\lambda = 128$ bits of security, one can do two bits of grinding ([Section 3.3](#)) per round.

The error κ_0^* . The error κ_0^* is perhaps the most delicate, as it (sometimes) demands at the same time that $\min \mathcal{P}$ and $|\mathcal{P}|$ are large, while still satisfying $\max \mathcal{P} \leq (|\mathbb{F}_{2^\nu}| - 1)/(\ell_1 + 1) + 1$. Recall that in many scenarios κ_0^* does not exist in the IOP (i.e. sometimes our IOP does not invoke [Construction 4.4](#)). These scenarios are described in the optional step of Round 1 of [Construction 4.8](#).

Recall ([Theorem 4.5](#))

$$\kappa_0^* = \frac{2 \deg(R)}{q_{\min}} + \frac{\log(\ell + 1) + \log(\deg(R) + 1) + 2 \log(q - 1)}{|\mathcal{P}| \cdot \lfloor \log(q_{\min}) \rfloor},$$

where q is either the characteristic of R or $\ell_1 \cdot \|\mathbf{v}^{(1)}\| + 1$ if $R = \mathbb{Z}$ in [Construction 4.8](#). In the latter case, in [Construction 4.12](#), $q = \ell_1 \cdot \|\mathbf{u}^{(1)}\| + 1$ where $\mathbf{u}$ is defined in [??](#).

The adjustment for κ_0^* (if needed at all) is highly dependent on $\deg(R)$. Concretely, if $\deg(R) = 0$, then the term $2\deg(R)/q_{\min}$ is zero. This allows to take $\mathcal{P}$ as all primes smaller than $2^{114+\varepsilon}$. There are approximately $2^{114}/114$.

We select $\mathcal{P}$ to be the set of all primes in a range $[2^a, 2^b]$. There are approximately

$$|\mathcal{P}| \approx (2^b/b - 2^a/a)/\log_e(2)$$

such primes. Using $b \leq 114$, $\ell \leq 2^{35}$, and plugging these values into κ_0^* we obtain

$$\kappa_0^* \lesssim f(a) := \frac{2\deg(R)}{2^a} + \frac{35 + \log(\deg(R) + 1) + 2\log(q - 1)}{\frac{a}{114} \cdot 2^{114} - 2^a}.$$

We now calculate minima for different values of $\deg(R)$ and q .

To fix intuition, $\deg(R)$ and q are determined by the field or ring a PIOP runs on. For example, in our full proving system from ??, constraints can be over $\mathbb{Z}$ or $\mathbb{Z}[X]$, then they are projected to a random prime field $\mathbb{F}_q$, a PIOP is ran on them, and an MLE claim over $\mathbb{F}_p$ is produced. Then [Construction 4.12](#) is used to prove those claims. The size of q is set so that the PIOP achieves λ bits of security. A very conservative choice in this setting is $q \leq 2^{256}$, but also $q \leq 2^{128}$ is feasible if $\lambda = 100$ or one is willing to use grinding during the PIOP.

Another setting is to have constraints over a finite field $\mathbb{K}$ such as a degree 4 or 5 extension of the Babybear field, or a degree 2 or 3 extension of the Goldilocks field.

For these choices of $(\deg(R), \log_2(q))$, the corresponding minima for f is as follows:

d	$\log q$	a^*	$\log f(a^*)$	Bit-deficit to 128	Grinding time (ms)
0	256	108	-105.3	23	68.0
0	128	108	-106.2	22	26.9
3	64	112	-106.5	22	26.9
5	32	112	-106.9	22	26.9

Table 3: Minima a^* of $f(a)$ for different values of $\deg(R)$ and $\log(q)$. The first error term κ_0^* satisfies $\kappa_0^* = 1/2^{-\log f(a^*)}$. We include the bits left required to achieve $\kappa_0^* < 1/2^{128}$ and the cost of grinding such amount of bits, in a MacBook Air M4 16GB, multi-threaded, using Blake3 as hash function.

5 Virtualization of $\mathbb{F}_2$ -linear combinations lifted to integers

Albert: Add a result about expressiveness of $A\pi^{-1}M\pi(\mathbf{f})$: arbitrary circuits. Minimality o

In this section we expand the core relation $\mathcal{R}_{\text{Lin-BitsDual}}$ to impose, instead of constraints of the form $f = \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f})$ and $\langle \pi_q(\mathbf{f}), \mathbf{v} \rangle = \mu$, where $\mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$, constraints as $\text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f}) = f$ and

$$\langle \pi_q \circ \pi_2^{-1}(A \cdot \pi_2(\mathbf{f})), \mathbf{v} \rangle = \mu$$

for a matrix $A \in \mathbb{F}_2^{\ell' \times \ell}$. Precisely, given a new parameter ℓ' , we define

$$\mathcal{R}_{\text{Lin-BitsDual}}^{\text{virtual}}(\{0, 1\}, \pi_q, R, \mathcal{C}) := \left\{ \left(\begin{array}{l} \mathbf{x} = (\mu, \mathbf{v}, A) \in R \times R^{\ell'} \times \mathbb{F}_2^{\ell' \times \ell}, \\ \mathbf{y} = f \in \mathbb{F}_2^m, \\ \mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell \end{array} \right) \middle| \begin{array}{l} f = \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f}) \\ \wedge \langle \pi_q \circ \pi_2^{-1}(A \cdot \pi_2(\mathbf{f})), \mathbf{v} \rangle = \mu \end{array} \right\}$$

Albert: We could replace $M \cdot \pi_2(\mathbf{f})$ by the output of a GKR circuit on $\pi_2(\mathbf{f})$. The relation has the tag “virtual” to reference the intuition that $\mathcal{R}_{\text{Lin-BitsDual}}^{\text{virtual}}(\{0, 1\}, \pi_q, R, \mathcal{C})$ expresses a linear constraint on the vector $\mathbf{h} = \pi_q \circ \pi_2^{-1}(A \cdot \pi_2(\mathbf{f})) \in R^{\ell'}$, which is not committed directly: only $\pi_2(\mathbf{f})$ is. Note that $\mathbf{h}$ is a linear expression over $\mathbb{F}_{2^\nu}$ of the elements in $\pi_2(\mathbf{f})$, lifted to the integers, and then projected to R . In ?? we arithmetize SHA-256 as an instance of $\mathcal{R}_{\text{Lin-BitsDual}}^{\text{virtual}}(\{0, 1\}, \text{id}, \mathbb{Z}, \mathcal{C})$.

Construction 5.1 (IOP for $\mathcal{R}_{\text{Lin-BitsDual}}^{\text{virtual}}$). **Parameters.** $(R, \pi, \mathbb{F}_{2^\nu}, \ell', \ell_1, \ell_2, \ell, \mathcal{C})$ comprised of: a ring R (either $\mathbb{Z}$ or a finite field); a ring homomorphism $\pi : \mathbb{Z} \rightarrow R$; a binary field $\mathbb{F}_{2^\nu}$; a virtual witness length ℓ' ; tensor decomposition dimensions ℓ_1, ℓ_2 satisfying $\ell_1 \cdot \ell_2 = \ell'$; a non-virtual witness length ℓ ; a linear code $\mathcal{C}$ over $\mathbb{F}_{2^\nu}$ of dimension ℓ/ν and block length m .

Inputs. The prover and verifier receive $\mathbf{x} = (\mu, \mathbf{v}, A) \in R \times R^{\ell'} \times \mathbb{F}_{2^\nu}^{\ell' \times \ell}$ and $\mathbf{y} = f \in \mathbb{F}_{2^\nu}^m$. The prover receives $\mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ such that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{Lin-BitsDual}}^{\text{virtual}}(\{0, 1\}, \pi_q, R, \mathcal{C})$.

1. Define $\mathbf{h} = \pi_2^{-1}(A \cdot \pi_2(\mathbf{f})) \in \{0, 1\}^{\ell'} \subseteq \mathbb{Z}^{\ell'}$. Prover and verifier execute [Construction 4.8](#) until before Phase 3, with parameters³

$$(R, \pi, \mathbb{F}_{2^\nu}, \ell', \ell_1, \ell_2, \mathcal{C})$$

and inputs

$$\mathbf{x}' = (\mu, \mathbf{v}), \quad \mathbf{y}' = f, \quad \mathbf{w}' = \mathbf{h} \in \{0, 1\}^{\ell'} \subseteq \mathbb{Z}^{\ell'}.$$

Note that until that point, the verifier has not made any queries to $\mathbf{y}'$. So, in terms of completeness, it does not matter, yet, that f is not the encoding of $\pi_2(\mathbf{h}) = M \cdot \pi_2(\mathbf{f})$.

2. At the end of Phase 2 of [Construction 4.8](#), prover and verifier are left with the claims

$$\begin{aligned} & ((\mathbf{v}, \mu'), f, \pi_2(\mathbf{h})) \in \mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu) \\ &= \left\{ \left(\begin{array}{l} \mathbf{x} = (\mathbf{u}, \mu) \in \mathbb{F}_{2^\nu}^k, \\ \mathbf{y} = g \in \mathbb{F}_{2^\nu}^m, \\ \mathbf{w} = \mathbf{g} \in (\mathbb{F}_{2^\nu}^\ell) \end{array} \right) \middle| \begin{array}{l} g = \text{Enc}_\mathcal{C} \circ \text{Pack}_\nu(\mathbf{g}) \\ \langle \mathbf{u}, \mathbf{g} \rangle = \mu \end{array} \right\} \end{aligned}$$

for some vector $\mathbf{v} \in \mathbb{F}_{2^\nu}^{\ell'}$ and field element $\mu' \in \mathbb{F}_{2^\nu}$ output during the IOR Π_{red} .

As we mentioned, this claim as is is false, because $f \neq \text{Enc}_\mathcal{C} \circ \text{Pack}_\nu \circ \pi_2(\mathbf{h})$ —indeed, the dimension of $\mathcal{C}$ does not match that of $\text{Pack}_\nu \circ \pi_2(\mathbf{h})$. Instead of, as done in [Construction 4.8](#), using an IOP for $\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu)$ to attempt to prove the above claim, prover and verifier proceed as follows.

Observe that, if the prover is honest,

$$\mu' = \langle \mathbf{v}, \pi_2(\mathbf{h}) \rangle = \langle \mathbf{v}, A \cdot \pi_2(\mathbf{f}) \rangle = \langle A^\top \cdot \mathbf{v}, \pi_2(\mathbf{f}) \rangle,$$

where $\top$ denotes transposition. Hence the following claim is true:

$$((A^\top \cdot \mathbf{v}, \mu'), f, \pi_2(\mathbf{f})) \in \mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu). \quad (13)$$

Now prover and verifier use an IOP for $\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu)$ to prove (13) [Albert: comments on verifier?](#).

³[Construction 4.8](#) would require a code $\mathcal{C}$ as parameter whose dimension matches ℓ' . This is not the case in our current setting, but since [Construction 4.8](#) is run only up to before Phase 3 (and so the code $\mathcal{C}$ is not used in this partial run), this is not an issue.

Remark 5.2. *Albert: Comments on how the verifier handles $A^\top \cdot \mathbf{v}$*

Theorem 5.3 (Completeness and round-by-round knowledge soundness of [Construction 5.1](#)). *Albert: Should we be more formal with this theorem? Albert: Completeness missing* Under the same hypothesis as [Theorem 4.10](#) for $\delta, g, \ell', \ell_1, \ell_2, \Pi_{\text{red}}, \Pi_{\text{iopp}}$, [Construction 5.1](#) is a perfectly complete IOP for $\mathcal{R}_{\text{Lin-BitsDual}}^{\text{virtual}}$ and has round-by-round knowledge soundness with relaxation $((\mathcal{R}_{\text{Lin-BitsDual}}^{\text{virtual}})^\delta, \perp)$, with errors

$$\kappa_0^*, (\kappa_{\text{red},i})_{i \in [t_{\text{red}}]}, (\kappa_{\text{iopp},i})_{i \in [t_{\text{iopp}}]}$$

as defined in [Theorem 4.10](#). The errors $\kappa_{\text{red},i}$ correspond to the errors of the IOR Π_{red} when used on triples of the form

$$(\mathbf{x}_{\text{red}} = ((g^{\gamma_i})_{i \in [\ell_1]}, (g^{\mu_j})_{j \in [\ell_2]}), \mathbf{y}_{\text{red}} = f, \mathbf{w}_{\text{red}} = \pi_2(\mathbf{h})) \in \mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{BitZ}}, \text{Pack}_\nu)$$

for certain γ_i and μ_j (so, the witness has dimension ℓ' and we use the decomposition $\ell' = \ell_1 \cdot \ell_2$). The errors $\kappa_{\text{iopp},i}$ correspond to the errors of the IOP Π_{iopp} when used on triples of the form

$$(\mathbf{x}_{\text{iopp}} = (A^\top \cdot \mathbf{v}, \mu'), \mathbf{y}_{\text{iopp}} = f, \mathbf{w}_{\text{iopp}} = \pi_2(\mathbf{f})) \in \mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu),$$

so that the witness has dimension ℓ .

The error κ_0^* is only present if $\deg(R) > 0$. Otherwise, [Construction 4.8](#) (and thus [Construction 5.1](#) has one round less) and κ_0^* is not part of the soundness errors.

Proof. Albert: Too vague? The proof results from observing that [Construction 5.1](#) is the composition of an IOR Π_1 from $\mathcal{R}_{\text{Lin-BitsDual}}^{\text{virtual}}(\{0, 1\}, \pi, R, \mathcal{C})$ to $\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu)$, with an IOP Π_2 for $\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu)$. In Π_1 the input is $(\mathbf{x} = (\mu, \mathbf{v}, A), \mathbf{y} = f, \mathbf{w} = \mathbf{f})$. The output is $((A^\top \cdot \mathbf{v}, \mu'), f, \pi_2(\mathbf{f}))$. The IOR Π_1 is exactly the same as [Construction 4.8](#) with input $((\mu, \mathbf{v}), f, \mathbf{h})$, until Phase 3. Hence by the same arguments as in the proof of [Theorem 4.10](#), it is perfectly complete and has errors $\kappa_0^*, (\kappa_{\text{red},i})_i$ with parameters set as $(R, \pi, \mathbb{F}_{2^\nu}, \ell', \ell_1, \ell_2, \mathcal{C})$. The IOP Π_2 is perfectly complete and has round-by-round knowledge errors $\kappa_{\text{iopp},i}$ by assumption. $\square$

Remark 5.4 (Instantiating [Construction 5.1](#)). [Construction 5.1](#) works in the exact same way as [Construction 4.8](#) until Phase 3, where an IOP for $\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu)$ is invoked on a slightly different instance-oracle-triple. The latter IOP is a standard combination of ring switching and WHIR ??, plus perhaps additional processes to make verification time cheaper ?. Assuming this is instantiated correctly, the whole [Construction 5.1](#) can be instantiated as discussed in [Section 4.7](#).

6 A GKR protocol for low entropy batched grand products via lookup tables

Albert: Explain somewhere: why we do ℓ_2 grand products instead of just one: sublinear verifier, lookup tables enabling the optimizations here, –something else?

The bulk of the work in $\mathbb{F}2\mathbb{Z}$ ([Construction 4.8](#)) is to reduce the claim over the binary field $\mathbb{F}_{2^\nu}$

$$\prod_{i \in [\ell_1]} ((g^{\gamma_i} - 1) \cdot \pi_2(\mathbf{f}_{ij}) + 1) = g^{\mu_j} \in \mathbb{F}_{2^\nu}, \quad \text{for all } j \in [\ell_2] \quad (14)$$

to an inner product claim of the form $\langle \mathbf{v}, \pi_2(\mathbf{f}) \rangle = \mu$ over $\mathbb{F}_{2^\nu}$, for some $\mathbf{v} \in \mathbb{F}_{2^\nu}^\ell$, where here $\mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$. Recall that

$$\ell = \ell_1 \cdot \ell_2.$$

To fix intuition, in our instantiation (cf. [Section 4.7](#)) we set $\ell_1 \approx \ell^{0.6}, \ell_2 \approx \ell^{0.4}$.

Cryptographically speaking, our protocol is simply a standard batched version of the GKR protocol applied to grand products from [?]. However, our implementation uses lookup tables (i.e. preprocessed data) to optimize the prover time of such protocol. In particular, our techniques leverage the fact that the factors in (14) have *low entropy*, in the sense that for every $i \in [\ell_1]$ and $j \in [\ell_2]$, the i -th factor in the j -th grand product of (14) can take only two values, depending on whether $\mathbf{f}_{ij} = 0$ or $\mathbf{f}_{ij} = 1$.

The optimizations we introduce apply to any (batched) grand product with low entropy factors, and not only to the grand product in (14). They also work on any field. Because of this,

We describe a general protocol IOR that, given a finite field $\mathbb{K}$; a subset $S \subseteq \mathbb{K}$; and dimensions $\ell = \ell_1 \cdot \ell_2$, reduces triples from

$$\mathcal{R}_{\text{PESAT}}(\mathbb{K}, p) = \left\{ \left(\begin{array}{l} \mathbf{x} = (\mathbf{z}, \boldsymbol{\mu}) \in \mathbb{K}^{\ell_1} \times \mathbb{K}^{\ell_2}, \\ \mathbf{y} = \perp, \\ \mathbf{w} = \mathbf{f} = (\mathbf{f}_{*j})_{j \in [\ell_2]} \in S^{\ell_1 \times \ell_2} \subseteq \mathbb{K}^{\ell_1 \times \ell_2} \end{array} \right) \left| \prod_{i \in [\ell_1]} p(\mathbf{z}_i, \mathbf{f}_{ij}) = \boldsymbol{\mu}_j \text{ for all } j \in [\ell_2] \right. \right\},$$

to triples in

$$\mathcal{R}_{\text{Lin}}(\mathbb{K}) = \left\{ \left(\begin{array}{l} \mathbf{x} = (\mathbf{u}, \mu) \in \mathbb{K}^\ell \times \mathbb{K}, \\ \mathbf{y} = \perp, \\ \mathbf{w} = \mathbf{f} \in S^{\ell_1 \times \ell_2} \subseteq \mathbb{K}^{\ell_1 \times \ell_2} \end{array} \right) \left| \langle \mathbf{u}, \mathbf{f} \rangle = \mu \right. \right\}.$$

One can additionally add an encoding $f = \text{Enc}_C \circ T(\mathbf{f})$ as an oracle $\mathbf{y}$ in the above relations (matching the context in which we use the IOR), but our IOR does not make any use of it whatsoever, hence we omit it.

We describe Π_{red} now. We will describe our implementation-level optimizations afterwards. As we mentioned, cryptographically/protocol-wise, our protocol is a standard batched GKR for grand products. We assume ℓ_1, ℓ_2 are powers of two:

$$\ell_1 = 2^{r_1}, \quad \ell_2 = 2^{r_2}.$$

We index $[\ell_1], [\ell_2]$ via elements of the hypercubes $\{0, 1\}^{\log \ell_1}, \{0, 1\}^{\log \ell_2}$, respectively.

Construction 6.1 (An IOR Π_{red} from $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, p)$ to $\mathcal{R}_{\text{Lin}}(\mathbb{K})$). **Input.** Prover and verifier receive $(\mathbf{y}, \boldsymbol{\mu}) \in \mathbb{K}^{\ell_1} \times \mathbb{K}^{\ell_2}$. Prover receives also $\mathbf{f} \in S^{\ell_1 \times \ell_2} \subseteq \mathbb{K}^{\ell_1 \times \ell_2}$.

Let L_{r_1} be a *multilinear* polynomial over $\mathbb{K}$ on $r_1 + r_2$ variables such that

$$L_{r_1}(\mathbf{b}, \mathbf{c}) = p(\mathbf{z}_{\mathbf{b}}, \mathbf{f}_{\mathbf{b}, \mathbf{c}}) \quad \text{for all } (\mathbf{b}, \mathbf{c}) \in \{0, 1\}^{r_1} \times \{0, 1\}^{r_2}.$$

(note that p may not be multilinear). For $i = 0, \dots, r_1 - 1$, let L_i be a multilinear polynomial on $r_2 + i$ variables such that

$$L_i(\mathbf{b}, \mathbf{c}) = L_{i+1}(\mathbf{b}, 0, \mathbf{c}) \cdot L_{i+1}(\mathbf{b}, 1, \mathbf{c}) \quad \text{for all } \mathbf{b} \in \{0, 1\}^i \times \{0, 1\}^{r_2}.$$

1. The verifier sends $\alpha^{(0)} \in \mathbb{K}^{r_2}$. Both prover and verifier compute $C_0 := \widetilde{L}_0(\alpha^{(0)})$ (note that L_0 is the multilinear extension of μ over $\mathbb{K}$).
2. For $i = 0, \dots, r_1 - 1$:

- (a) The prover and verifier run the sumcheck protocol for the following claim: John: FYI: The current implementation constructs two sumchecks. One for $\mathbf{b}$ variables. A subsequent one for the $\mathbf{c}$ variables. It might be good to explore the difference between the two Albert: I think it's the same. Any sumcheck with at least two rounds is a composition of two smaller sumchecks, that's what's happening in the implementation essentially. Maybe we should modify the implementation to avoid confusion

$$C^{(i)} = \sum_{(\mathbf{b}, \mathbf{c}) \in \{0,1\}^i \times \{0,1\}^{r_2}} L_{i+1}(\mathbf{b}, 0, \mathbf{c}) \cdot L_{i+1}(\mathbf{b}, 1, \mathbf{c}) \cdot \text{eq}(\mathbf{b}, \mathbf{c}; \alpha^{(i)}).$$

At the end of the sumcheck protocol, the prover and verifier are left with a claim of the form

$$L_{i+1}(\mathbf{r}_{\mathbf{b}}^{(i)}, 0, \mathbf{r}_{\mathbf{c}}^{(i)}) = a_0^{(i)}, \quad L_{i+1}(\mathbf{r}_{\mathbf{b}}^{(i)}, 1, \mathbf{r}_{\mathbf{c}}^{(i)}) = a_1^{(i)} \quad (15)$$

for some $\mathbf{r}_{\mathbf{b}}^{(i)} \in \mathbb{K}^i$, $\mathbf{r}_{\mathbf{c}}^{(i)} \in \mathbb{K}^{r_2}$ and $a_0^{(i)}, a_1^{(i)} \in \mathbb{K}$.

- (b) The prover and verifier unify the two claims from (15) into a single one via the following standard procedure:
 - i. The verifier sends a random element $\beta^{(i)}$.
 - ii. The claim (15) is replaced by

$$L_{i+1}(\mathbf{r}_{\mathbf{b}}^{(i)}, \beta^{(i)}, \mathbf{r}_{\mathbf{c}}^{(i)}) = C^{(i+1)} := a_0^{(i)} \cdot (1 - \beta^{(i)}) + a_1^{(i)} \cdot \beta^{(i)}.$$

Let

$$\alpha^{(i+1)} := (\mathbf{r}_{\mathbf{b}}^{(i)}, \beta^{(i)}, \mathbf{r}_{\mathbf{c}}^{(i)}).$$

3. At this stage, the prover and verifier are left with the claim

$$L_{r_1}(\alpha^{(r_1)}) = C^{(r_1)}.$$

This claim is equivalent to

$$\sum_{(\mathbf{b}, \mathbf{c}) \in \{0,1\}^{r_1} \times \{0,1\}^{r_2}} p(\mathbf{z}_{\mathbf{b}}, \mathbf{f}_{\mathbf{b}, \mathbf{c}}) \cdot \text{eq}((\mathbf{b}, \mathbf{c}), \alpha^{(r_1)}) = C^{(r_1)}, \quad (16)$$

If p is multilinear, then (16) is a linear claim on $\mathbf{f}$. Namely, it is equivalent to

$$\langle \mathbf{u}, \mathbf{f} \rangle = C^{(r_1)} + \mu$$

for some $\mathbf{u} \in \mathbb{K}^\ell, \mu \in \mathbb{K}$. In this case, prover and verifier output

$$(\mathbf{x}' = (\mathbf{u}, C^{(r_1)} + \mu), \perp, \mathbf{w}' = \mathbf{f}) \in \mathcal{R}_{\text{Lin}}(\mathbb{K}).$$

Otherwise, i.e. if p is not multilinear, prover and verifier run a sumcheck for the claim (16), which results in MLE claims for $\mathbf{y}$ and $\mathbf{f}$. The verifier checks the claim for $\mathbf{y}$ itself, and prover and verifier output the MLE claim for $\mathbf{f}$.

6.1 Optimization via lookup tables

We optimize [Construction 6.1](#) as follows. For simplicity we assume

$$S = \{0, 1\} \subseteq \mathbb{K},$$

but any small subset of $\mathbb{K}$ is amenable to the techniques discussed here.

John: Typing is a bit inconsistent here for c and $f_{b,c}$. Namely, we say c seems to be a boolean vector but $f_{b,c}$ is not. *Albert:* We mentioned above that we are indexing $[\ell_1]$ and $[\ell_2]$ with hypercube elements. Does this resolve the question?

Setup We start with the following observation: for every $(\mathbf{b}, \mathbf{c}) \in \{0, 1\}^{r_1} \times \{0, 1\}^{r_2}$, we have $\mathbf{f}_{\mathbf{b}, \mathbf{c}} \in S = \{0, 1\}$, and so

$$L_{r_1}(\mathbf{b}, \mathbf{c}) = p(\mathbf{z}_{\mathbf{b}}, \mathbf{f}_{\mathbf{b}, \mathbf{c}}) = \begin{cases} y_{\mathbf{b}}^{(1)} := p(\mathbf{z}_{\mathbf{b}}, 1) & \text{if } \mathbf{f}_{\mathbf{b}, \mathbf{c}} = 1 \\ y_{\mathbf{b}}^{(0)} := p(\mathbf{z}_{\mathbf{b}}, 0) & \text{if } \mathbf{f}_{\mathbf{b}, \mathbf{c}} = 0. \end{cases}$$

The values $\mathbf{y}_{\mathbf{b}}^{(0)}, \mathbf{y}_{\mathbf{b}}^{(1)}$ do not depend on $\mathbf{c}$. Hence the prover can set up the lookup table $T_{r_1} \subseteq (\mathbb{K} \times \mathbb{K})^{\{0, 1\}^{r_1}}$ defined as

$$T_{r_1}[\mathbf{b}] = (\mathbf{y}_{\mathbf{b}}^{(1)}, \mathbf{y}_{\mathbf{b}}^{(0)}) \quad \text{for all } \mathbf{b} \in \{0, 1\}^{r_1}$$

at the beginning of the protocol, and use it to obtain the values $L_{r_1}(\mathbf{b}, \mathbf{c})$. Concretely

$$L_{r_1}(\mathbf{b}, \mathbf{c}) = T_{r_1}[\mathbf{b}][\mathbf{f}_{\mathbf{b}, \mathbf{c}}] \quad \text{for all } (\mathbf{b}, \mathbf{c}) \in \{0, 1\}^{r_1} \times \{0, 1\}^{r_2} -$$

In our instantiation, $\mathbf{y}_{\mathbf{b}}^{(1)} = g^{\gamma_{\mathbf{b}}}$ and $\mathbf{y}_{\mathbf{b}}^{(0)} = 1$ (cf. ??), and so the prover precomputes the elements $g^{\gamma_{\mathbf{b}}}$ at the beginning of the protocol.

As we will explain later, the prover will also precompute similar tables for the values $L_{r_1-j}(\mathbf{b}, \mathbf{c})$ of the levels close to the leaves,

$$T_{r_1-j} \in \left(\mathbb{K}^{2^{2^j}} \right)^{\{0, 1\}^{r_1-j}}, \quad (17)$$

as well as other tables relevant for the different sumcheck rounds of [Construction 6.1](#). Concretely, for $\mathbf{b} \in \{0, 1\}^{r_1-j}$, $T_{r_1-j}[\mathbf{b}]$ consists of all possible products of 2^j consecutive factors of the grand product, namely the factors indexed by $(\mathbf{b}, \mathbf{b}')$ with $\mathbf{b}' \in \{0, 1\}^j$. Each factor can take two values, depending on whether $\mathbf{f}_{\mathbf{b}, \mathbf{b}', \mathbf{c}}$ is 1 or 0. Hence $T_{r_1-j}[\mathbf{b}]$ consists of 2^{2^j} elements. Hence the size of T_{r_1-j} in (17).

Because $|T_{r_1-j}|$ grows doubly-exponentially in j , these tables are viable only for the bottom few levels of the tree ($j = 0, 1, 2$ in our implementation). In higher levels we use standard sumchecks.

Importantly, all these tables have size independent of ℓ_2 . This (among others, cf. ??), is one of the reasons why we use the tensor decomposition structure, i.e. the split given by $\ell = \ell_1 \cdot \ell_2$, in [Construction 4.8](#).

Remark 6.2. All precomputed data in this section depends on the input vector $\mathbf{z}$ of $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, p)$, and possibly on further verifier challenges. Since, in a wider context where [Construction 6.1](#) is a sub-process of a larger proof system, $\mathbf{y}$ typically depends on verifier's challenges, these precomputations must be done at *runtime*.

First prover message in the last sumcheck in Step 2 of Construction 6.1 We start by analyzing Step 2 with $i = r_1 - 1$. In this step prover and verifier run a sumcheck for the claim

$$C^{(r_1-1)} = \sum_{(\mathbf{b}, \mathbf{c}) \in \{0,1\}^{r_1-1} \times \{0,1\}^{r_2}} L_{r_1}(\mathbf{b}, 0, \mathbf{c}) \cdot L_{r_1}(\mathbf{b}, 1, \mathbf{c}) \cdot \text{eq}(\mathbf{b}, \mathbf{c}; \boldsymbol{\alpha}^{(i)})$$

This is the most expensive step of Construction 6.1 for the prover (if done naively), since it is the sumcheck over the largest number of variables throughout Construction 6.1.

For $1 \leq j \leq r_1 - 1$, the j -th prover message in this sumcheck is, if $\boldsymbol{\rho}^{(j)} = (\rho_1, \dots, \rho_{j-1})$ is the vector of verifier challenges so far in this sumcheck,

$$\begin{aligned} h_j(X) &:= \sum_{(\mathbf{b}, \mathbf{c}) \in \{0,1\}^{r_1-1-j} \times \{0,1\}^{r_2}} L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 0, \mathbf{c}) \cdot L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 1, \mathbf{c}) \cdot \text{eq}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, \mathbf{c}; \boldsymbol{\alpha}^{(i)}) \\ &= \sum_{\mathbf{c} \in \{0,1\}^{r_2}} \left(\sum_{\mathbf{b} \in \{0,1\}^{r_1-1-j}} L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 0, \mathbf{c}) \cdot L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 1, \mathbf{c}) \cdot \text{eq}(\boldsymbol{\rho}^{(j)}, \mathbf{b}; \boldsymbol{\alpha}^{(i,1)}) \right) \cdot \text{eq}(X, \mathbf{c}; \boldsymbol{\alpha}^{(i,2)}) \\ &= \sum_{\mathbf{c} \in \{0,1\}^{r_2}} h_{j,\mathbf{c}}(X) \cdot \text{eq}(X, \mathbf{c}; \boldsymbol{\alpha}^{(i,2)}), \end{aligned}$$

for $\boldsymbol{\alpha}^{(i)} = (\boldsymbol{\alpha}^{(i,1)}, \boldsymbol{\alpha}^{(i,2)}) \in \{0,1\}^{r_1-j-1} \times \{0,1\}^{r_2+1}$. We claim that

$$L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 0, \mathbf{c}) = \left(L_{r_1}(\boldsymbol{\rho}^{(j)}, 1, \mathbf{b}, 0, \mathbf{c}) - L_{r_1}(\boldsymbol{\rho}^{(j)}, 0, \mathbf{b}, 0, \mathbf{c}) \right) \cdot X + L_{r_1}(\boldsymbol{\rho}^{(j)}, 0, \mathbf{b}, 0, \mathbf{c}) \quad (18)$$

Indeed, $L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 0, \mathbf{c})$ is a linear polynomial, and the right-hand side in the above expression is the line passing through the two evaluations of $L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 0, \mathbf{c})$ at $X = 0$ and $X = 1$. An analogous expression holds for $L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 1, \mathbf{c})$:

$$L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 1, \mathbf{c}) = \left(L_{r_1}(\boldsymbol{\rho}^{(j)}, 1, \mathbf{b}, 1, \mathbf{c}) - L_{r_1}(\boldsymbol{\rho}^{(j)}, 0, \mathbf{b}, 1, \mathbf{c}) \right) \cdot X + L_{r_1}(\boldsymbol{\rho}^{(j)}, 0, \mathbf{b}, 1, \mathbf{c}). \quad (19)$$

Renaming

$$E_{\varepsilon_1 \varepsilon_2}^{(j)}[\mathbf{b}, \mathbf{c}] := L_{r_1}(\boldsymbol{\rho}^{(j)}, \varepsilon_1, \mathbf{b}, \varepsilon_2, \mathbf{c}) \quad \varepsilon_1, \varepsilon_2 \in \{0,1\}, \quad \omega[\mathbf{b}] := \text{eq}(\boldsymbol{\rho}^{(j)}, \mathbf{b}; \boldsymbol{\alpha}^{(i,1)}),$$

we obtain

$$h_{j,\mathbf{c}}(X) = \sum_{\mathbf{b} \in \{0,1\}^{r_1-1-j}} X^2 \cdot A_2^{(j)}[\mathbf{b}, \mathbf{c}] + X \cdot A_1^{(j)}[\mathbf{b}, \mathbf{c}] + A_0^{(j)}[\mathbf{b}, \mathbf{c}]$$

where, [Albert: double check](#)

$$\begin{aligned} A_0^{(j)}[\mathbf{b}, \mathbf{c}] &:= E_{00}^{(j)}[\mathbf{b}, \mathbf{c}] \cdot E_{01}^{(j)}[\mathbf{b}, \mathbf{c}] \cdot \omega[\mathbf{b}], \\ A_1^{(j)}[\mathbf{b}, \mathbf{c}] &:= \left((E_{10}^{(j)}[\mathbf{b}, \mathbf{c}] - E_{00}^{(j)}[\mathbf{b}, \mathbf{c}]) \cdot E_{01}^{(j)}[\mathbf{b}, \mathbf{c}] + (E_{11}^{(j)}[\mathbf{b}, \mathbf{c}] - E_{01}^{(j)}[\mathbf{b}, \mathbf{c}]) \cdot E_{00}^{(j)}[\mathbf{b}, \mathbf{c}] \right) \cdot \omega[\mathbf{b}] \\ A_2^{(j)}[\mathbf{b}, \mathbf{c}] &:= (E_{10}^{(j)}[\mathbf{b}, \mathbf{c}] - E_{00}^{(j)}[\mathbf{b}, \mathbf{c}]) \cdot (E_{11}^{(j)}[\mathbf{b}, \mathbf{c}] - E_{01}^{(j)}[\mathbf{b}, \mathbf{c}]) \cdot \omega[\mathbf{b}]. \end{aligned}$$

Now, for $j = 1$, we have $\boldsymbol{\rho}^{(j)} = \perp$, and hence each value $E_{\varepsilon_1 \varepsilon_2}^{(1)}[\mathbf{b}, \mathbf{c}]$ can be obtained by querying $T_{r_1}[\mathbf{b}]$ and selecting $T_{r_1}[\mathbf{b}][0]$ or $T_{r_1}[\mathbf{b}][1]$ depending on whether $\mathbf{f}_{\mathbf{b}, \mathbf{c}} = 1$ or 0. A consequence is that

the coefficients themselves $A_k^{(1)}[\mathbf{b}, \mathbf{c}]$ ($k \in [3]$) can be obtained through lookup tables, as follows: we have

$$A_0^{(1)}[\mathbf{b}, \mathbf{c}] = T_{r_1}[0, \mathbf{b}, 0, \mathbf{c}][\mathbf{f}_{(0,\mathbf{b},0),\mathbf{c}}] \cdot T_{r_1}[0, \mathbf{b}, 1, \mathbf{c}][\mathbf{f}_{(0,\mathbf{b},1),\mathbf{c}}] \cdot \omega[\mathbf{b}]$$

Hence

$$A_0^{(1)}[\mathbf{b}, \mathbf{c}] = \begin{cases} \mathbf{y}_{0,\mathbf{b},0}^{(1)} \cdot \mathbf{y}_{0,\mathbf{b},1}^{(1)} \cdot \omega[\mathbf{b}] & \text{if } \mathbf{f}_{(0,\mathbf{b},0),\mathbf{c}} = 1, \mathbf{f}_{(0,\mathbf{b},1),\mathbf{c}} = 1 \\ \mathbf{y}_{0,\mathbf{b},0}^{(1)} \cdot \mathbf{y}_{0,\mathbf{b},1}^{(0)} \cdot \omega[\mathbf{b}] & \text{if } \mathbf{f}_{(0,\mathbf{b},0),\mathbf{c}} = 1, \mathbf{f}_{(0,\mathbf{b},1),\mathbf{c}} = 0 \\ \mathbf{y}_{0,\mathbf{b},0}^{(0)} \cdot \mathbf{y}_{0,\mathbf{b},1}^{(1)} \cdot \omega[\mathbf{b}] & \text{if } \mathbf{f}_{(0,\mathbf{b},0),\mathbf{c}} = 0, \mathbf{f}_{(0,\mathbf{b},1),\mathbf{c}} = 1 \\ \mathbf{y}_{0,\mathbf{b},0}^{(0)} \cdot \mathbf{y}_{0,\mathbf{b},1}^{(0)} \cdot \omega[\mathbf{b}] & \text{if } \mathbf{f}_{(0,\mathbf{b},0),\mathbf{c}} = 0, \mathbf{f}_{(0,\mathbf{b},1),\mathbf{c}} = 0 \end{cases}$$

Thus the prover can precompute the table

$$T_{A_0} \in (\mathbb{K}^4)^{\{0,1\}^{r_1-2}} = \mathbb{K}^{\ell_1}$$

given by

$$T_{A_0}[\mathbf{b}] = (\mathbf{y}_{0,\mathbf{b},0}^{(1)} \cdot \mathbf{y}_{0,\mathbf{b},1}^{(1)} \cdot \omega[\mathbf{b}], \mathbf{y}_{0,\mathbf{b},0}^{(1)} \cdot \mathbf{y}_{0,\mathbf{b},1}^{(0)} \cdot \omega[\mathbf{b}], \mathbf{y}_{0,\mathbf{b},0}^{(0)} \cdot \mathbf{y}_{0,\mathbf{b},1}^{(1)} \cdot \omega[\mathbf{b}], \mathbf{y}_{0,\mathbf{b},0}^{(0)} \cdot \mathbf{y}_{0,\mathbf{b},1}^{(0)} \cdot \omega[\mathbf{b}])$$

and use it to obtain any value $A_0^{(1)}[\mathbf{b}, \mathbf{c}]$ for $(\mathbf{b}, \mathbf{c}) \in \{0, 1\}^{r_1-2} \times \{0, 1\}^{r_2}$ via a single read into T_{A_0} . The size of T_{A_0} , again, does not depend on $\ell_2 = 2^{r_2}$. Building T_{A_0} requires at most $8 \cdot 2^{r_1-2} = 2^{r_1+1}$ multiplications and no additions.

We now explain how to obtain the values $A_1^{(1)}[\mathbf{b}, \mathbf{c}]$ and $A_2^{(1)}[\mathbf{b}, \mathbf{c}]$, again via the construction of a lookup table. Note that these depend on the *four* bits $\mathbf{f}_{(\varepsilon_1, \mathbf{b}, \varepsilon_2), \mathbf{c}}$, $\varepsilon_1, \varepsilon_2 \in \{0, 1\}$, so our tables will need $2^4 = 16$ entries (as opposed to 4 as in T_{A_0}) per index $\mathbf{b}$. The prover proceeds as follows:

- It precomputes a table $T_{S_1} \in (\mathbb{K}^4)^{\{0,1\}^{r_1-2}}$ analogous to T_{A_0} , which allows to obtain the values

$$S_1^{(1)}[\mathbf{b}, \mathbf{c}] := E_{10}^{(1)}[\mathbf{b}, \mathbf{c}] \cdot E_{11}^{(1)}[\mathbf{b}, \mathbf{c}] \cdot \omega[\mathbf{b}],$$

(observe that T_{A_0} allows to obtain the values $E_{00}^{(1)}[\mathbf{b}, \mathbf{c}] \cdot E_{01}^{(1)}[\mathbf{b}, \mathbf{c}] \cdot \omega[\mathbf{b}]$).

- It precomputes the table $T_{A_2} \in (\mathbb{K}^{16})^{\{0,1\}^{r_1-2}}$ given by

$$T_{A_2}[\mathbf{b}] = \left((\mathbf{y}_{1,\mathbf{b},0}^{(x_1)} - \mathbf{y}_{0,\mathbf{b},0}^{(x_0)}) \cdot (\mathbf{y}_{1,\mathbf{b},1}^{(x'_1)} - \mathbf{y}_{0,\mathbf{b},1}^{(x'_0)}) \cdot \omega[\mathbf{b}] \right)_{(x_0, x_1, x'_0, x'_1) \in \{0,1\}^4}, \quad (20)$$

so that $A_2^{(1)}[\mathbf{b}, \mathbf{c}]$ is the element

$$A_2^{(1)}[\mathbf{b}, \mathbf{c}] = T_{A_2}[\mathbf{b}][\mathbf{f}_{(0,\mathbf{b},0),\mathbf{c}}, \mathbf{f}_{(1,\mathbf{b},0),\mathbf{c}}, \mathbf{f}_{(0,\mathbf{b},1),\mathbf{c}}, \mathbf{f}_{(1,\mathbf{b},1),\mathbf{c}}].$$

Building T_{A_2} takes at most 20 multiplications and 8 additions per index $\mathbf{b}$ ⁴. I.e. at most $5 \cdot 2^{r_1}$ multiplications and 2^{r_1+1} additions in total.

- It obtains the values $A_1^{(1)}[\mathbf{b}, \mathbf{c}]$ via the identity

$$S_1^{(1)}[\mathbf{b}, \mathbf{c}] = A_0^{(1)}[\mathbf{b}, \mathbf{c}] + A_1^{(1)}[\mathbf{b}, \mathbf{c}] + A_2^{(1)}[\mathbf{b}, \mathbf{c}] \quad \text{for all } (\mathbf{b}, \mathbf{c}) \in \{0, 1\}^{r_1-2} \times \{0, 1\}^{r_2}.$$

⁴For the multiplications, there are 4 multiplications to compute the 4 elements $t_{x'_0, x'_1, \mathbf{b}} := (\mathbf{y}_{1,\mathbf{b},1}^{(x'_1)} - \mathbf{y}_{0,\mathbf{b},1}^{(x'_0)}) \cdot \omega[\mathbf{b}]$, and then 16 more multiplications to compute $(\mathbf{y}_{1,\mathbf{b},0}^{(x_1)} - \mathbf{y}_{0,\mathbf{b},0}^{(x_0)}) \cdot t_{x'_0, x'_1, \mathbf{b}}$

Once the tables have been computed, obtaining $h_{1,\mathbf{c}}(X)$ for one $\mathbf{c} \in \{0, 1\}^{r_2}$ costs $3 \cdot 2^{r_1-2}$ table reads (one read per table $T_{A_1}, T_{S_1}, T_{A_3}$ per index $\mathbf{b} \in \{0, 1\}^{r_1-2}$) and $3 \cdot 2^{r_1-2} + 2$ additions: this is the cost of adding together the values read at each table, providing $\sum_{\mathbf{b}} A_0[\mathbf{b}, \mathbf{c}]$, $\sum_{\mathbf{b}} S_1[\mathbf{b}, \mathbf{c}]$, $\sum_{\mathbf{b}} A_2[\mathbf{b}, \mathbf{c}]$, and then doing two additions to obtain $\sum_{\mathbf{b}} A_1[\mathbf{b}, \mathbf{c}] = \sum_{\mathbf{b}} S_1[\mathbf{b}, \mathbf{c}] - \sum_{\mathbf{b}} A_0[\mathbf{b}, \mathbf{c}] - \sum_{\mathbf{b}} A_2[\mathbf{b}, \mathbf{c}]$. In total, this amounts to at most $3\ell/4$ table reads and $\approx 3\ell/4 + 2 \cdot \ell_2$ additions to compute $h_{1,\mathbf{c}}$ for all $\mathbf{c}$, and no multiplications (in our instantiation, additions in $\mathbb{K}$ are 128-bit XORs).

Finally, the prover computes $h_1(X)$ from the polynomials $h_{1,\mathbf{c}}(X)$ with at most $5 \cdot 2^{r_2}$ additions and $3 \cdot 2^{r_2}$ multiplications (plus 2^{r_2} multiplications to precompute the values $\mathbf{eq}(\mathbf{c}; \alpha^{(i,2)})$, shared by all rounds of this sumcheck, and 2^{r_1-2} multiplications for the weights $\omega[\mathbf{b}]$).

In [Table 4](#) we record the total upper bounds for computing $h_1(X)$. In our application we obtain lower numbers since $\mathbf{y}_{\mathbf{b}}^{(0)} = 1$ and we work in characteristic two, hence constant terms cancel inside the differences from [Eq. \(20\)](#). Our implementation, in our use-case, builds all three tables with 8 multiplications and roughly 20 additions per index $\mathbf{b} \in \{0, 1\}^{r_1-2}$, i.e. $\approx 2\ell_1$ multiplications in total.

On the other hand, a standard generic approach to compute $h_1(X)$ would require $O(\ell) = O(\ell_1 \cdot \ell_2)$ multiplications and additions, even if using the few sumcheck optimizations that apply in our settings (to our knowledge) [Albert: which ones, exactly?](#). Crucially, we note that our sumcheck features summands of full field bit size $\log |\mathbb{K}|$, which voids some known powerful optimizations when the summands are small, e.g. the univariate skip [\[?\]](#) and some optimizations from [\[?\]](#) [Albert: This is a crucial point. It should be highlighted more. And it should be an important point of the introduction and technical overview](#)

	Our approach	Standard/generic
Multiplications	$\leq 10 \cdot \ell_1 + 4 \cdot \ell_2$	$\Theta(\ell_1 \cdot \ell_2)$
Additions	$\leq 3 \cdot \ell_1 \cdot \ell_2 / 4 + 2 \cdot \ell_1 + 5 \cdot \ell_2$	$\Theta(\ell_1 \cdot \ell_2)$
Table reads	$3 \cdot \ell_1 \cdot \ell_2 / 4$, into tables of $6 \cdot \ell_1$ field elements	–

Table 4: The cost in field operations of the first sumcheck round of the last sumcheck in [Construction 6.1](#) (Step 2 with $i = r_1 - 1$), where $\ell = \ell_1 \cdot \ell_2 = 2^{r_1} \cdot 2^{r_2}$ is the witness size. In our instantiation $\ell_1 \approx \ell^{0.6} = 2^{r_1}$ and $\ell_2 \approx \ell^{0.4} = 2^{r_2}$ ([Section 4.7](#)).

Subsequent prover messages in the last sumcheck in Step 2 of [Construction 6.1](#) Next we focus on computing $h_2(X)$. Substituting X by the verifier's first sumcheck challenge ρ_1 in [Eqs. \(18\)](#) and [\(19\)](#), we have, for every $\mathbf{b} \in \{0, 1\}^{r_1-1}, \mathbf{c} \in \{0, 1\}^{r_2}$,

$$\begin{aligned}
L_{r_1}(\rho_1, \mathbf{b}, \mathbf{c}) &= (L_{r_1}(1, \mathbf{b}, \mathbf{c}) - L_{r_1}(0, \mathbf{b}, \mathbf{c})) \cdot \rho_1 + L_{r_1}(0, \mathbf{b}, \mathbf{c}) \\
&= \begin{cases} \mathbf{y}_{\mathbf{b}}^{(11)} := \left(\mathbf{y}_{1,\mathbf{b}}^{(1)} - \mathbf{y}_{0,\mathbf{b}}^{(1)} \right) \cdot \rho_1 + \mathbf{y}_{0,\mathbf{b}}^{(1)} & \text{if } \mathbf{f}_{(0,\mathbf{b}),\mathbf{c}} = 1, \mathbf{f}_{(1,\mathbf{b}),\mathbf{c}} = 1 \\ \mathbf{y}_{\mathbf{b}}^{(10)} := \left(\mathbf{y}_{1,\mathbf{b}}^{(0)} - \mathbf{y}_{0,\mathbf{b}}^{(1)} \right) \cdot \rho_1 + \mathbf{y}_{0,\mathbf{b}}^{(1)} & \text{if } \mathbf{f}_{(0,\mathbf{b}),\mathbf{c}} = 1, \mathbf{f}_{(1,\mathbf{b}),\mathbf{c}} = 0 \\ \mathbf{y}_{\mathbf{b}}^{(01)} := \left(\mathbf{y}_{1,\mathbf{b}}^{(1)} - \mathbf{y}_{0,\mathbf{b}}^{(0)} \right) \cdot \rho_1 + \mathbf{y}_{0,\mathbf{b}}^{(0)} & \text{if } \mathbf{f}_{(0,\mathbf{b}),\mathbf{c}} = 0, \mathbf{f}_{(1,\mathbf{b}),\mathbf{c}} = 1 \\ \mathbf{y}_{\mathbf{b}}^{(00)} := \left(\mathbf{y}_{1,\mathbf{b}}^{(0)} - \mathbf{y}_{0,\mathbf{b}}^{(0)} \right) \cdot \rho_1 + \mathbf{y}_{0,\mathbf{b}}^{(0)} & \text{if } \mathbf{f}_{(0,\mathbf{b}),\mathbf{c}} = 0, \mathbf{f}_{(1,\mathbf{b}),\mathbf{c}} = 0 \end{cases}
\end{aligned}$$

The idea now is that the prover can compute $h_2(X)$ in essentially the same way as $h_1(X)$, but this time, instead of starting with the table

$$T_{r_1} \in (\mathbb{K}^2)^{\{0,1\}^{r_1}} \quad \text{such that } T_{r_1}[\mathbf{b}] = (\mathbf{y}_{\mathbf{b}}^{(0)}, \mathbf{y}_{\mathbf{b}}^{(1)}) \text{ and } L_{r_1}(\mathbf{b}, \mathbf{c}) = T_{r_1}[\mathbf{b}][\mathbf{f}_{\mathbf{b},\mathbf{c}}], \quad (\mathbf{b}, \mathbf{c}) \in \{0, 1\}^{r_1} \times \{0, 1\}^{r_2},$$

it starts with the table

$$T_{r_1}^{(2)} \in (\mathbb{K}^4)^{\{0,1\}^{r_1-1}} \quad \text{such that } T_{r_1}^{(2)}[\mathbf{b}] = (\mathbf{y}_{\mathbf{b}}^{(11)}, \mathbf{y}_{\mathbf{b}}^{(10)}, \mathbf{y}_{\mathbf{b}}^{(01)}, \mathbf{y}_{\mathbf{b}}^{(00)}) \quad \text{for } \mathbf{b} \in \{0,1\}^{r_1-1}$$

for which we have, indexing the entries of $T_{r_1}^{(2)}[\mathbf{b}]$ with the 4 values $(1,1), (1,0), (0,1), (0,0)$,

$$L_{r_1}(\rho_1, \mathbf{b}, \mathbf{c}) = T_{r_1}^{(2)}[\mathbf{b}][(\mathbf{f}_{0,\mathbf{b}}, \mathbf{f}_{1,\mathbf{b}})]$$

This allows the prover to obtain the values $L_{r_1}(\rho_1, \mathbf{b}, \mathbf{c})$ for all $(\mathbf{b}, \mathbf{c}) \in \{0,1\}^{r_1-1} \times \{0,1\}^{r_2}$ with a simple read in $T_{r_1}^{(2)}$.

We now have, for $(\mathbf{b}, \mathbf{c}) \in \{0,1\}^{r_1-3} \times \{0,1\}^{r_2}$,

$$\begin{aligned} A_0^{(2)}[\mathbf{b}, \mathbf{c}] &= L_{r_1}(\rho_1, 0, \mathbf{b}, 0, \mathbf{c}) \cdot L_{r_1}(\rho_1, 0, \mathbf{b}, 1, \mathbf{c}) \cdot \omega^{(2)}[\mathbf{b}] \\ &= T_{r_1}^{(2)}[0, \mathbf{b}, 0][(\mathbf{f}_{(0,0,\mathbf{b},0),\mathbf{c}}, \mathbf{f}_{(1,0,\mathbf{b},0),\mathbf{c}})] \cdot T_{r_1}^{(2)}[0, \mathbf{b}, 1][(\mathbf{f}_{(0,0,\mathbf{b},1),\mathbf{c}}, \mathbf{f}_{(1,0,\mathbf{b},1),\mathbf{c}})] \cdot \omega[\mathbf{b}] \end{aligned}$$

for some weights $\omega^{(2)}[\mathbf{b}]$. Hence the elements $A_0^{(2)}[\mathbf{b}, \mathbf{c}]$ can be obtained by querying a table

$$T_{A_0}^{(2)} \in (\mathbb{K}^{16})^{\{0,1\}^{r_1-3}}$$

with

$$T_{A_0}^{(2)}[\mathbf{b}] = (T_{r_1}^{(2)}[0, \mathbf{b}, 0][\mathbf{x}] \cdot T_{r_1}^{(2)}[0, \mathbf{b}, 1][\mathbf{x}'] \cdot \omega[\mathbf{b}])_{\mathbf{x}, \mathbf{x}' \in \{0,1\}^2}$$

The table $T_{S_1}^{(2)}$ is analogous. The coefficient $A_2^{(2)}$, however, now couples *eight* bits — each difference $E_{1\varepsilon}^{(2)} - E_{0\varepsilon}^{(2)}$ is selected by four bits — so tabulating it directly would require a $2^8 = 256$ -case table. Instead of precombining it, our implementation *factors* the term: it gathers the two differences from two 16-case tables and joins them with a single multiplication per slot.

This can be iterated in further sumcheck rounds, for computing $h_3(X), h_4(X)$, etc. The downside is a doubly-exponential growth of the table widths: each further round *squares* the number of cases per index of every table, but it only *halves* the number of indices. Hence, in practice one stops using our optimization after a few rounds of sumcheck, and falls back to continue the sumcheck protocol normally. Concretely, in our implementation we never compute a table with more than 16 elements per index (hence, for example, the table $T_{A_2}^{(2)}$ is not built in our implementation). This is done so that the tables can be kept in cache. [Table 5](#) displays the different tables built or not built in our implementation, the cost of doing so, and their size at $\ell_1 = 2^{r_1} = 2^{16}$ (corresponding to a total witness size of 2^{27} in our implementation).

Table	Dimension	Adds/mults. in $\mathbb{K}$	Size (MB), $\ell_1 = 2^{16}$	Built?
$T_{r_1}^{(1)}$	$(\mathbb{K}^2)^{2^{r_1}}$	0/0	2.1	yes
$T_{A_0}^{(1)}$	$(\mathbb{K}^{2^2})^{2^{r_1-2}}$	0 / 2^{r_1+1}	1.0	yes
$T_{S_1}^{(1)}$	$(\mathbb{K}^{2^2})^{2^{r_1-2}}$	0 / 2^{r_1+1}	1.0	yes
$T_{A_2}^{(1)}$	$(\mathbb{K}^{2^4})^{2^{r_1-2}}$	2^{r_1+1} / $5 \cdot 2^{r_1}$	4.2	yes
$T_{r_1}^{(2)}$	$(\mathbb{K}^{2^2})^{2^{r_1-1}}$	2^{r_1+2} / 2^{r_1+1}	2.1	yes
$T_{A_0}^{(2)}$	$(\mathbb{K}^{2^4})^{2^{r_1-3}}$	0 / 2^{r_1+2}	2.1	yes
$T_{S_1}^{(2)}$	$(\mathbb{K}^{2^4})^{2^{r_1-3}}$	0 / 2^{r_1+2}	2.1	yes
$T_{A_2}^{(2)}$	$(\mathbb{K}^{2^8})^{2^{r_1-3}}$	—	33.6	no
$T_{r_1}^{(3)}$	$(\mathbb{K}^{2^4})^{2^{r_1-2}}$	2^{r_1+3} / 2^{r_1+2}	4.2	yes
$T_{A_0}^{(3)}$	$(\mathbb{K}^{2^8})^{2^{r_1-4}}$	—	16.8	no
$T_{S_1}^{(3)}$	$(\mathbb{K}^{2^8})^{2^{r_1-4}}$	—	16.8	no
$T_{A_2}^{(3)}$	$(\mathbb{K}^{2^{16}})^{2^{r_1-4}}$	—	4295	no

Table 5: Dimensions, build costs, and sizes of the tables for computing $h_1(X), h_2(X), h_3(X)$, with $\log |\mathbb{K}| \approx 128$. We use the example size $\ell_1 = 2^{r_1} = 2^{16}$, which following our instantiation ($\ell_1 \approx \ell^{0.6}$, [Section 4.7](#)) this corresponds to a total witness size $\ell \approx 2^{27}$. **Albert:** It seems we do not always build $T_{A_0}^{(2)}$ and $T_{S_1}^{(2)}$?

Instead of building the table $T_{A_2}^{(2)}$ to obtain the values $A_2^{(2)}[\mathbf{b}, \mathbf{c}]$, the prover reads $T_{r_1}^{(2)}$ at appropriate values and computes $A_2^{(2)}[\mathbf{b}, \mathbf{c}]$ using some of the elements in the vectors read this way. In Round 3 we only use the table $T_{r_1}^{(3)}$, and otherwise perform sumcheck normally.

The rest of sumchecks in Step 2 of [Construction 6.1](#) So far we have discussed the last sumcheck from Step 2 of [Construction 6.1](#), corresponding to the lower layer of the binary tree of products of our GKR circuit for the grand products.

The sumchecks run for $i = r_1 - 2, r_1 - 3, \dots$ can be optimized in the exact same way as we did for the last sumcheck ($i = r_1 - 1$). In this case, instead of starting with the table $T_{r_1}^{(1)}$, which at each index contains the two values $(\mathbf{y}_{\mathbf{b}}^{(1)}, \mathbf{y}_{\mathbf{b}}^{(0)})$, one starts with the table

$$T_{r_1-j} \in (\mathbb{K}^{2^{2^j}})^{\{0,1\}^{r_1-j}}$$

described in [Eq. \(17\)](#). Since T_{r_1-j} has quadratically many more entries per index than T_{r_1-j+1} , all subsequent tables also have quadratically many more entries per index compared to the analogous tables from the $(i+1)$ -th sumcheck. And, further, they require quadratically more operations to comput.

In our implementation we want to keep all tables in cache. Because of this, the smaller i is (i.e. the further we get from the last sumcheck in the GKR protocol), the less number of sumcheck rounds we use tables in. Concretely, for $i = r_1 - 2$ ($j = 1$) we build and use tables $T_{r_1-1}^{(1)}, T_{A_0}^{(1)}, T_{S_1}^{(1)}, T_{r_1-1}^{(2)}$ (note the lack of $T_{A_2}^{(1)}$). For $i = r_1 - 3$ ($j = 3$) we only use the table $T_{r_1-2}^{(1)}$.

6.2 Other optimizations for [Construction 4.8](#)

Using 4 factor multiplication (instead of 2) in the last step of the GKR grand product In Step 2 we reduce the number of sumchecks by 1 (so now there are $r_1 - 2$ sumchecks) by making

the last sumcheck certify the claim

$$C^{(r_1-2)} = \sum_{(\mathbf{b}, \mathbf{c}) \in \{0,1\}^{r_1-2} \times \{0,1\}^{r_2}} L_{r_1}(\mathbf{b}, 0, 0, \mathbf{c}) \cdot L_{r_1}(\mathbf{b}, 0, 1, \mathbf{c}) \cdot L_{r_1}(\mathbf{b}, 1, 0, \mathbf{c}) \cdot L_{r_1}(\mathbf{b}, 1, 1, \mathbf{c}) \cdot \text{eq}(\mathbf{b}, \mathbf{c}; \boldsymbol{\alpha}^{(i)})$$

I.e.

Step 1 of Construction 4.8 via lookup tables In Step 1 of Construction 4.8, the prover computes the elements $\mu_1, \dots, \mu_{\ell_2}$ again by means of a lookup table T , which is indexed over $i \in [\ell_1/4]$ and contains the 16-length vectors

$$T[i] = \left(\sum_{k \in [4]} \varepsilon_k \cdot \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)})_{4*(i-1)+k} \right)_{\varepsilon_1, \dots, \varepsilon_4 \in \{0,1\}} \in \mathbb{Z}[X]^{16}.$$

6.3 Optimizing $\mathbb{F}_2\mathbb{Z}$ for virtualized inputs $\pi_2^{-1}(M \cdot \pi_2(\mathbf{f}))$

Albert: Leaving this section out (add to our open problems list)

In Construction 5.1, at Step 2, prover and verifier use $\Pi_{\text{red}}^{\ell_2}$ to reduce the claim

$$\prod_{i \in [\ell_1]} ((g^{\gamma_i} - 1) \cdot (M_0 \cdot \pi_2(\mathbf{f}_{*j}))_i + 1) = g^{\mu_j} \quad \in \mathbb{F}_{2^\nu}, \quad \text{for all } j \in [\ell_2] \quad (21)$$

to ℓ_2 inner product claims over $\mathbb{F}_{2^\nu}$ for the vectors $M_0 \cdot \pi_2(\mathbf{f}_{*j})$. In many applications of interest (see ??), the vectors $M_0 \cdot \pi_2(\mathbf{f}_{*j})$ are larger than $\mathbf{f}_{*j}$, but contain entries that are structured expressions of the entries in $\mathbf{f}_{*j}$. For example, in ??, $M \cdot \pi_2(\mathbf{f})$ is the concatenation of $\pi_2(\mathbf{f})$ and the vector with entries $\pi_2(\mathbf{f})_i \text{ XOR } \pi_2(\mathbf{f})_{i+1}$.

A natural question is whether, in such scenarios, the proof of (21) can be optimized so that it costs less than a full proof for ℓ_2 arbitrary input vectors of the same length as $M_0 \cdot \pi_2(\mathbf{f}_{*j})$. In this section we show that this is indeed possible.

7 $\mathbb{F}_2\mathbb{Z}$ for PIMSAT

7.1 Polynomial Ideal Membership Satisfiability (PIMSAT)

Throughout this section R denotes either $\mathbb{Z}$ or a finite field $\mathbb{F}$ with q^e elements. **Albert:** Actually it may be good to allow R to be $\mathbb{Q}$, so we can inline constraints more easily (even though we can write the constraints over $\mathbb{Q}$ and then turn them into $\mathbb{Z}$ constraints by multiplying by the LCM, so maybe it's not worth the added complexity

In this section we describe a variation of the PESAT relation (cf. Definition 3.2), called *Polynomial Ideal Membership Satisfiability (PIMSAT)*. It generalizes PESAT by allowing polynomial constraints over polynomial rings (or modules), and by replacing equality to zero by ideal membership. PIMSAT is a subset of the UCS relation from [Abd+26] (cf. Remark 7.2).

We fix a witness length ℓ_0 which we assume is a power of two; an input length k ; a bit size bound B ; a degree bound d ; a collection of ι distinct ideals $\mathcal{I}_1, \dots, \mathcal{I}_\iota$ of $R[X]$; a number of rows n ; and for each $i \in [\iota]$, a vector of n polynomials

$$\mathbf{p}^{(i)} = \left(\mathbf{p}_j^{(i)} \mid j \in [n] \right) \in ((R[X])[Y_1, \dots, Y_{k+\ell}])^n$$

with coefficients in $R[X]$ and on $k + \ell$ variables. We denote

$$\mathbf{p} = (\mathbf{p}^{(i)})_{i \in [\iota]}, \quad \mathcal{I} = (\mathcal{I}_i)_{i \in [\iota]}.$$

The PIMSAT relation with respect to these parameters is defined as

$$\mathcal{R}_{\text{PIMSAT}}(R_{\leq B}^{\leq d}, \mathbf{p}, \mathcal{I}) = \left\{ \begin{array}{l} \mathbf{x} = \mathbf{x} \in \left(R_{\leq B}^{\leq d}[X]\right)^k, \\ \mathbf{y} = \perp, \\ \mathbf{w} = \mathbf{f} \in \left(R_{\leq B}^{\leq d}[X]\right)^\ell \end{array} \left| \begin{array}{l} \mathbf{p}_j^{(i)}(\mathbf{x}, \mathbf{f}) \in \mathcal{I}_i \text{ for all } j \in [n], i \in [\iota] \end{array} \right. \right\}$$

The next remark explains that all our constructions for $\mathcal{R}_{\text{PIMSAT}}$ apply, in particular, to the usual R1CS, CCS [STW23], AIR [air], lookups, permutation constraints, etc. over finite fields.

Example 7.1 (R1CS constraints over $R[X]$ with ideal membership). Let $R, k, \ell, n, \iota, \mathcal{I}_1, \dots, \mathcal{I}_\iota, B, d$ be as before, and let $A^{(i)}, B^{(i)}, C^{(i)}$ be $n \times (k + \ell)$ matrices with entries in $R[X]$ (typically, these are sparse matrices with entries in a small subset of R), for all $i \in [\iota]$. Then the R1CS-PIMSAT relation is defined as:

$$\mathcal{R}_{\text{R1CS-PIMSAT}} := \left\{ \begin{array}{l} \mathbf{x} = \mathbf{x} \in \left(R_{\leq B}^{\leq d}[X]\right)^k, \\ \mathbf{y} = \perp, \\ \mathbf{w} = \mathbf{f} \in \left(R_{\leq B}^{\leq d}[X]\right)^\ell \end{array} \left| \begin{array}{l} \left(\left(A^{(i)} \cdot \mathbf{z} \right) \circ \left(B^{(i)} \cdot \mathbf{z} \right) - \left(C^{(i)} \cdot \mathbf{z} \right) \right)_j \in \mathcal{I}_i \\ \text{for all } j \in [n], i \in [\iota] \end{array} \right. \right\},$$

$\mathbf{z} = (\mathbf{x}, \mathbf{f})$

The relation $\mathcal{R}_{\text{R1CS-PIMSAT}}$ is precisely $\mathcal{R}_{\text{PIMSAT}}(R_{\leq B}^{\leq d}, \mathbf{p}, \mathcal{I})$ with

$$\mathbf{p}_j^{(i)}(\mathbf{x}, \mathbf{f}) = \left(\left(A^{(i)} \cdot \mathbf{z} \right) \circ \left(B^{(i)} \cdot \mathbf{z} \right) - \left(C^{(i)} \cdot \mathbf{z} \right) \right)_j.$$

Following Example 7.1 and [linear-accumulation; zinc; Abd+26] one can show that PIMSAT allows to express popular constraint systems such as CCS, AIR, and lookup and permutation constraints, over polynomial rings and with ideal membership.

Remark 7.2 (PIMSAT vs the UCS relation from Zinc+ [Abd+26]). PIMSAT is a particular subset of the widely general UCS relation from [Abd+26]. In this work, we will build a IOP for PIMSAT with witnesses being committed over binary fields. The same ideas can be used to construct an analogous IOP for UCS with witnesses with entries in a bounded subset of $\mathbb{Z}[X]$ (as opposed to $\mathbb{Q}[X]$ [Albert: committing to rational through their bit-decomposition is a bit strange, but it may still be possible?](#)). We choose to focus on PIMSAT to favor simplicity of exposition and of our IOP.

The main difference between PIMSAT and UCS is that the later allows for witnesses and constraints over arbitrary and multiple domains in the same instance, including rational polynomial rings $\mathbb{Q}[X]$, and it allows to reuse witness elements in $\mathbb{Q}[X]$ or $\mathbb{Z}[X]$ in other domains such as finite fields and polynomial rings over finite fields, by using homomorphic projections.

7.2 An IOP for PIMSAT with oracles in binary fields

We now describe an IOP for $\mathcal{R}_{\text{PIMSAT}}$. We assume $n = 2^{n'}$ is a power of two, and identify $[n]$ with $\{0, 1\}^{n'}$. Let $\mathcal{C}$ be a linear code over $\mathbb{F}_{2^\nu}$ of dimension $(\ell_0 \cdot d \cdot B)/2^\nu$ and blocklength m . Let ℓ_1, ℓ_2

be tensor decomposition parameters, to be used in [Construction 4.12](#), so that $\ell_0 \cdot d \cdot B = \ell_1 \cdot \ell_2$. Assume $\mathcal{C} = \mathcal{C}_1^{\equiv \ell_2}$ is the ℓ_2 -interleaving of a linear code $\mathcal{C}_1$ over $\mathbb{F}_{2^\nu}$ of dimension $\ell_1/2^\nu$.

For each prime, recall that $\pi_q : \mathbb{Z} \rightarrow \mathbb{F}_q$ denotes reduction modulo q ; and for every finite field $\mathbb{K}$ of characteristic q , we convene that π_q acts on $\mathbb{K}$ as the identity. For each finite field $\mathbb{K}$ let $\Pi_{\mathbb{K}}$ be an IOR that given $(\mathbf{x}, \perp, \mathbf{w})$ for $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, \mathbf{p})$ (for any collection of polynomials $\mathbf{p}$), outputs $(\mathbf{x}', \perp, \mathbf{w}) \in \mathcal{R}_{\text{Lin}}(\mathbb{K})$. We emphasize that the output of $\Pi_{\mathbb{K}}$ has the same witness $\mathbf{w}$ as the input.

In our instantiations, $\Pi_{\mathbb{K}}$ is either the usual Spartan [\[Set19\]](#) PIOP over $\mathbb{K}$, or a STARK-like PIOP [\[Sta21\]](#) over $\mathbb{K}$.

Construction 7.3. [IOP for $\mathcal{R}_{\text{PIMSAT}}$] **Parameters.** $(R, M_R, d, \ell_0, B, \ell_1, \ell_2, \mathbb{F}_{2^\nu}, \mathcal{C}_1^{\equiv \ell_2})$ comprised of: a ring $R \in \{\mathbb{Z}, \mathbb{F}_{q^e}\}$; an R -module M_R of dimension d ; a witness length ℓ_0 ; a bit-size bound B ; a binary field $\mathbb{F}_{2^\nu}$; tensor decomposition dimensions ℓ_1, ℓ_2 with $\ell_1 \cdot \ell_2 = \ell_0 \cdot d \cdot B$; a ℓ_2 -interleaving of a code $\mathcal{C}_1$ over $\mathbb{F}_{2^\nu}$ of dimension $\ell_1/2^\nu$.

Inputs. The prover and verifier receive $\mathbf{x} = \mathbf{x} \in R_{\leq B}^d[X]^k$. The prover additionally receives $\mathbf{w} = \mathbf{f} \in R_{\leq B}^d[X]^\ell$ such that $(\mathbf{x}, \perp, \mathbf{w}) \in \mathcal{R}_{\text{PIMSAT}}(R_{\leq B}^d, \mathbf{p}, \mathcal{I})$.

1. Round 1: oracles.

- The prover computes

$$f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\text{bits}(\mathbf{f})) \in \mathbb{F}_{2^\nu}^m$$

(cf. (1)) and sends the verifier oracle access to f .

- The prover forms the multilinear polynomial, for all $i \in [\ell]$,

$$Q^{(i)}(Z_1, \dots, Z_\nu) := \sum_{\mathbf{b} \in \{0,1\}^{n'}} \mathbf{p}_{\mathbf{b}}^{(i)}(\mathbf{x}, \mathbf{f}) \cdot \text{eq}(\mathbf{b}, Z_1, \dots, Z_\nu) \in (R[X])[Z_1, \dots, Z_\nu].$$

The claim that $\mathbf{p}_{\mathbf{b}}^{(i)}(\mathbf{x}, \mathbf{f}) \in \mathcal{I}_i$ for all $\mathbf{b} \in \{0,1\}^{n'}$ and $i \in [\ell]$ is equivalent to the claim

$$Q^{(i)}(\mathbf{z}) \in \mathcal{I}_i \quad \text{for all } \mathbf{z} \in \{0,1\}^{n'}, i \in [\ell]. \quad (22)$$

- If $R = \mathbb{F}$, we let $\mathbb{K}$ be a large enough (to be determined later) extension of $\mathbb{F}$. If $R = \mathbb{Z}$, verifier samples a prime from a set of prime $\mathcal{P}$ (cf. [Remark 7.4](#) for comments on how to do this in practice). Abusing the notation, we denote the prime by q and denote $\mathbb{F}_q$ by $\mathbb{K}$.

The claim (22) is replaced by

$$\pi_q(Q^{(i)}(\mathbf{z})) \in \pi_q(\mathcal{I}_i) \quad \text{for all } \mathbf{z} \in \{0,1\}^{n'}, i \in [\ell]. \quad (23)$$

Recall (cf. ??) that π_q is coefficient-wise reduction modulo q when $R = \mathbb{Z}$, and it is the identity map when R is a field $\mathbb{F}$ of characteristic q .

- The verifier samples and sends a random point $\mathbf{r} \in \mathbb{K}^{n'}$.

2. Round 2: Ideal check

- The prover responds with values $u_i \in \mathbb{K}[X]$ for each $i \in [\ell]$. Supposedly, $u_i = \pi_q(Q^{(i)})(\mathbf{r})$. These values can be computed as one would compute the corresponding values in a univariate skip, see [Section 7.2 Albert: todo](#). Alternatively, if $Q^{(i)}$ is linear on the multilinear extension

of $\mathbf{f}$, then u_i can easily be computed from $\widetilde{\pi_q(\mathbf{f})}(\mathbf{r})$, which in turn can be computed as d MLE evaluations over $\pi_q(R)$ ⁵.

- The verifier checks that $u_i \in \pi_q(\mathcal{I}_i)$ for all $i \in [\iota]$. Then it samples and sends batching elements $\theta_1, \dots, \theta_\iota \in \mathbb{K}$. Further, if $d > 0$, it samples and sends an element α **Albert: it could send a random $\mathbb{F}_q$ -module homomorphism $\mathbb{K}^{<d}[X] \rightarrow \mathbb{K}$.**

3. Using the IOR $\Pi_{\mathbb{K}}$ for the projected claim

- Let $\pi := \pi_{q,\alpha}$ (cf. ??). The prover and verifier replace the claim (23) with the claim

$$\sum_{i \in [\iota]} \sum_{\mathbf{b} \in \{0,1\}^{n'}} \theta_i \cdot \pi(\mathbf{p}_{\mathbf{b}}^{(i)})(\pi(\mathbf{x}), \pi(\mathbf{f})) \cdot \text{eq}(\mathbf{b}, \mathbf{r}) = \sum_{i \in [\iota]} \theta_i \cdot \pi(u_i) \in \mathbb{K}. \quad (24)$$

Observe that $(\pi(\mathbf{x}), \perp, \pi(\mathbf{f})) \in \mathcal{R}_{\text{PESAT}}(\mathbb{K}, \hat{\mathbf{p}})$ (cf. Definition 3.2) with $\hat{\mathbf{p}} = (\hat{\mathbf{p}}_1)$ given by

$$\hat{\mathbf{p}}_1 = \sum_{i \in [\iota]} \theta_i \cdot \sum_{\mathbf{b} \in \{0,1\}^{n'}} \pi(\mathbf{p}_{\mathbf{b}}^{(i)})(Y_1, \dots, Y_{k+\ell}) - \sum_{i \in [\iota]} \theta_i \cdot \pi(u_i).$$

Accordingly, prover and verifier use the IOR $\Pi_{\mathbb{K}}$ to reduce (24) to a claim of the form:

$$((\mathbf{v}, \mu), \perp, \pi(\mathbf{f})) \in \mathcal{R}_{\text{Lin}}(\mathbb{K}) \quad (25)$$

for some $\mathbf{v} \in \mathbb{K}^{\ell_0}$ and some $\mu \in \mathbb{K}$.

4. Using the $\mathbb{F}_2\mathbb{Z}$ IOR for $\mathcal{R}_{\text{Lin-BitsDual}}$

- The initial claim $(\mathbf{x}, \perp, \mathbf{w}) \in \mathcal{R}_{\text{PIMSAT}}$ has been replaced by $((\mathbf{v}, \mu), \perp, \pi(\mathbf{f})) \in \mathcal{R}_{\text{Lin}}(\mathbb{K})$. Notice that $\pi : R[X] \rightarrow \mathbb{K}$ induces a π_q -module homomorphism $\psi : R^{<d}[X] \rightarrow \mathbb{K}$ given by $\psi(h(X)) = \pi(h(X))$. Hence

$$((\mathbf{v}, \mu), f, \mathbf{f}) \in \mathcal{R}_{\text{Lin-BitsDual}}(R_{\leq B}^{<d}, \psi, \mathbb{K}, \mathcal{C}). \quad (26)$$

Prover and verifier now use Construction 4.12 to prove (26), with parameters

$$(R, q, R^{<d}[X], d, \mathbb{K}, \psi, (1, X, \dots, X^{d-1}), \ell_0, B, \ell_1, \ell_2, \mathbb{F}_{2^\nu}, \mathcal{C}_1^{\equiv \ell_2}).$$

Remark 7.4 (Randomly sampling a field). **Albert: Add some comment about the recursive cost and the mitigations we have**

Theorem 7.5 (Completeness and round-by-round knowledge soundness of Construction 7.3). *Let $(R, M_R, d, \ell_0, B, \ell_1, \ell_2, \mathbb{F}_{2^\nu}, \mathcal{C}_1^{\equiv \ell_2})$ be parameters for Construction 7.3. Assume the following:*

- *For every field $\mathbb{K}$ sampleable in Round 1 of Construction 7.3 (or for the fixed field $\mathbb{K}$ if $R = \mathbb{F}$), $\Pi_{\mathbb{K}}$ is a $t_{\Pi_{\mathbb{K}}}$ -round perfectly complete IOR from $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, \mathbf{p})$ to $\mathcal{R}_{\text{Lin}}(\mathbb{K})$ with round-by-round knowledge soundness (without relaxation) and errors $\kappa_{\Pi_{\mathbb{K}},1}, \dots, \kappa_{\Pi_{\mathbb{K}},t_{\Pi_{\mathbb{K}}}}$.*
- *Let $\delta \in (0, 1)$ be a proximity parameter.*

⁵This was already observed at least in [flock], when $Q^{(i)}$ is the multilinear extension of the vector $C \cdot \mathbf{z}$ in an R1CS constraint **Albert: check: is that linear on witness?**

- Assume $|\mathbb{F}_{2^\nu}| - 1 \geq (\ell_1 + 1) \cdot (q_{\max} - 1)$, where $q_{\max}$ is the largest prime in $\mathcal{P}$ in [Construction 4.12](#).
- Assume the notation and hypotheses of [Theorems 4.10](#) and [4.14](#), so that [Construction 4.12](#) is a perfectly complete $(\tau_{\text{ext}} + t_{\text{red}} + t_{\text{iopp}})$ -round IOP for $\mathcal{R}_{\text{Lin-BitsDual}}(R_{\leq B}^{\leq d}, \psi, \mathbb{K}, \mathcal{C})$, with $\tau_{\text{ext}} = 1$ if the field $\mathbb{K}$ in [Construction 7.3](#) is a proper extension field and $\tau_{\text{ext}} = 0$ if $\mathbb{K}$ is a prime field, with round-by-round knowledge with relaxation $(\mathcal{R}_{\text{Lin-BitsDual}}(R_{\leq B}^{\leq d}, \psi, \mathbb{K}, \mathcal{C})^\delta, \perp)$ and errors

$$\kappa_{\text{lift-proj}}^*, (\kappa_{\text{red}i})_{i \in [t_{\text{red}}]}, (\kappa_{\text{iopp}i})_{i \in [t_{\text{iopp}}]},$$

where

$$\kappa_{\text{lift-proj}}^* = \begin{cases} \frac{\deg(R)}{q_{\min}} + \frac{\log(\ell_1+1)+\log(q-1)}{|\mathcal{P}| \cdot \lceil \log(q_{\min}) \rceil} & \text{if } \mathbb{K} \text{ is a proper extension field,} \\ \perp & \text{if } \mathbb{K} \text{ is a prime field.} \end{cases}$$

and $q_{\min}$ is the smallest prime in the set of sampleable primes $\mathcal{P}$.

Then [Construction 7.3](#) is a perfectly complete $(2 + t_{\mathbb{K}} + \tau_{\text{ext}} + t_{\text{red}} + t_{\text{iopp}})$ -IOP for $\mathcal{R}_{\text{PIMSAT}}(R_{\leq B}^{\leq d}, \mathbf{p}, \mathcal{I})$ with round-by-round knowledge soundness without and errors

$$\kappa_1, \kappa_2, (\kappa_{\Pi_{\mathbb{K}}, i})_{i \in [t_{\Pi_{\mathbb{K}}}]}, \kappa_{\text{lift-proj}}^*, (\kappa_{\text{red}i})_{i \in [t_{\text{red}}]}, (\kappa_{\text{iopp}i})_{i \in [t_{\text{iopp}}]},$$

with

$$\begin{aligned} \kappa_1 &:= \left(\frac{B_{\max} \cdot (1 + \max(\nabla(\mathbf{p}) - d_{\min}, 0))}{|\mathcal{P}| \cdot \lceil \log(|\mathbb{K}_{\min}|) \rceil} \right) \cdot \tau_{\mathbb{Z}} + \frac{1}{|\mathbb{K}_{\min}|} \\ \kappa_2 &:= \frac{\nabla(\mathbf{p})}{|\mathbb{K}_{\min}|} + \frac{1}{|\mathbb{K}_{\min}|}, \end{aligned}$$

where:

- $\tau_{\mathbb{Z}} = 1$ if $R = \mathbb{Z}$ and $\tau_{\mathbb{Z}} = 0$ otherwise,
- $\mathbb{K}_{\min}$ is either $\mathbb{K}$ if $R = \mathbb{F}$, or the field $\mathbb{F}_q$ with $q \in \mathcal{P}$ with the smallest size if $R = \mathbb{Z}$,
- $d_{\min}$ is the smallest degree among the generators of $\mathcal{I}_1, \dots, \mathcal{I}_\ell$,
- $\nabla(\mathbf{p}) := \max \left\{ \max_{\mathbf{b} \in \{0,1\}^{n'}, i \in [\ell]} \{ \deg_X(\mathbf{p}_j^{(i)}) + (d-1) \cdot \max_{t \in [k+\ell]} \deg_{Y_t}(\mathbf{p}_j^i) \cdot (k+\ell) \}, d \right\}$
- $B_{\max} := \max_{\mathbf{b} \in \{0,1\}^{n'}, i \in [\ell]} \lceil \log(|\mathbf{p}_{\mathbf{b}}^{(i)}(\mathbf{x}, \mathbf{f})|_{\infty}) \rceil$.

In [Section 7.2](#) we describe the variations in number of rounds and errors of [Construction 7.3](#) depending on whether $R = \mathbb{Z}$ or $R = \mathbb{F}$, and on whether $\mathbb{K}$ is a prime field or a proper extension field.

We defer the proof of [Theorem 7.5](#) to [Section A.2 Albert: Errors after composition of IOR's cannot be the same, since they receive different inputs?](#). We state and prove a key lemma here:

	$R = \mathbb{Z}$	$R = \mathbb{F}, \mathbb{F} \subsetneq \mathbb{K}$	$R = \mathbb{F} = \mathbb{K}$ is a prime field
Rounds	$2 + t_{\mathbb{K}} + t_{\text{red}} + t_{\text{iopp}}$	$2 + t_{\mathbb{K}} + 1 + t_{\text{red}} + t_{\text{iopp}}$	$2 + t_{\mathbb{K}} + t_{\text{red}} + t_{\text{iopp}}$
$\kappa_{\text{lift-proj}}$	$\perp$	$\frac{\deg(R)}{q_{\min}} + \frac{\log(\ell_1+1) + \log(q-1)}{ \mathcal{P} \cdot \lfloor \log(q_{\min}) \rfloor}$	$\perp$
κ_1	$\frac{B_{\max} \cdot (1 + \max(\nabla(\mathbf{p}) - d_{\min}, 0))}{ \mathcal{P} \cdot \lfloor \log(\mathbb{K}_{\min}) \rfloor} + \frac{1}{ \mathbb{K}_{\min} }$	$\frac{1}{ \mathbb{K}_{\min} }$	$\frac{1}{ \mathbb{K}_{\min} }$

Table 6: Variations in number of rounds and errors of [Construction 7.3](#) depending on whether $R = \mathbb{Z}$ or $R = \mathbb{F}$, and on whether $\mathbb{K}$ is a prime field or a proper extension field

Lemma 7.6 (Ideal membership is preserved under modular reduction). *Let $d, B \geq 1$ and $f, h \in \mathbb{Z}_{\leq B}^{\leq d}[X]$, with h either 0 or a nonzero monic polynomial. Let $\mathcal{P}$ be a set of primes. Let $\mathcal{I}_h$ be the ideal generated by h in $\mathbb{Z}[X]$. Then*

$$\Pr_{p \leftarrow \mathcal{P}} \left[\pi_p(f) \in \pi_p(\mathcal{I}_h) \wedge f \notin \mathcal{I}_h \right] < \frac{B \cdot (1 + \max(\deg(f) - \deg(h) + 1, 0))}{|\mathcal{P}| \cdot \lfloor \log(q_{\min}) \rfloor}.$$

where $q_{\min} := \min \mathcal{P}$.

Proof. Assume $f \notin \mathcal{I}_h$. In particular $f \neq 0$. If $h = 0$, then $\pi_p(\mathcal{I}_h) = \{0\}$, and so $\pi_p(f) = 0$. Since $\|f\|_{\infty} \leq 2^B - 1$, there are at most $B / \lfloor \log(q_{\min}) \rfloor$ primes p such that $\pi_p(f) = 0$, as needed.

If $h \neq 0$ and $\deg(h) = 1$, then $h = 1$ (since h is monic) and so $f \in \mathcal{I}_h = \mathbb{Z}[X]$, a contradiction. Assume thus that $\deg(h) \geq 1$. Let $Q, R \in \mathbb{Z}[X]$ be such that $f = h \cdot Q + R$ with $\deg(R) < \deg(h)$ (such integer polynomials exist because h is monic). Since $f \notin \mathcal{I}_h$, we have $R \neq 0$. By ??,

$$\|R\|_{\infty} \leq \|f\|_{\infty} \cdot (1 + \|h\|_{\infty})^{\max(\deg(f) - \deg(h) + 1, 0)}.$$

Now since $\deg(\pi_p(h)) = \deg(h)$ (because h is monic) we have $\deg(\pi_q(h)) > \deg(R) \geq \deg(\pi_p(R))$. The condition $\pi_p(f) \in \pi_p(\mathcal{I}_h)$ thus implies that $\pi_p(R) = 0$. There are at most $\log_2(\|R\|_{\infty}) / \lfloor \log q_{\min} \rfloor$ primes dividing the coefficients of R . Hence

$$\Pr_{p \leftarrow \mathcal{P}} [f \notin \mathcal{I}_h \wedge \pi_p(f) \in \pi_p(\mathcal{I}_h)] \leq \frac{\log_2(\|R\|_{\infty})}{|\mathcal{P}| \cdot \lfloor \log q_{\min} \rfloor} \leq \frac{B + \max(\deg(f) - \deg(h) + 1, 0) \cdot B}{|\mathcal{P}| \cdot \lfloor \log q_{\min} \rfloor},$$

as needed. $\square$

Albert: Ideal membership as a generalization of univariate skip? **Albert:** Repeated computations through ideal membership?

7.3 The CM constraint system: composition of $\mathbb{Z}$ -linear with $\mathbb{F}_2$ -linear constraints

Albert: Not sure this is needed depending on how we write the technical overview.

Albert: Maybe we should have general R1CS here, not just linear R1CS. And then describe the CM relation a subset of the more general R1CS relation

In this section we focus on a specific subset of $\mathcal{R}_{\text{PIMSAT}}(\{0, 1\}, \mathbf{p}, \perp)$, and we extend it with the same virtualization mechanism introduced in [Section 5](#) for the relation $\mathcal{R}_{\text{Lin-BitsDual}}^{\text{virtual}}(\{0, 1\}, \pi_q, R, C)$.

As usual, let ℓ' be a vector length. Let $C \in \mathbb{Z}^{n \times \ell'}$ be an integer matrix, and let $\mathbf{p}_{\text{CM}} = (\mathbf{p}_{\text{CM}, i})_{i \in [n]}$ be such that for any $(\mathbf{x}, \mathbf{w}) \in \mathbb{Z}^{\ell'}$,

$$\mathbf{p}_i(\mathbf{x}, \mathbf{q}) = \langle C_{i*} \cdot (\mathbf{x}, \mathbf{w}) \rangle,$$

where C_{i*} denotes the i -th row of C . Let further k be a public input length, and let $A \in \mathbb{F}_2^{\ell' \times (\ell+k)}$ be a matrix with entries in $\mathbb{F}_2$. Let $\mathcal{C}$ be a linear code over $\mathbb{F}_{2^\nu}$ of dimension ℓ/ν and block length m .

Definition 7.7. The CM relation is defined as follows:

$$\mathcal{R}_{\text{CM}} := \left\{ \begin{array}{l} \mathbf{x} = (\mathbf{x}, C, A) \in \{0, 1\}^k \times \mathbb{Z}^{n \times \ell'} \times \mathbb{F}_2^{\ell' \times (k+\ell)} \\ \mathbf{y} = \perp \\ \mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell \end{array} \middle| \begin{array}{l} C \cdot \pi_2^{-1}(A \cdot \pi_2(\mathbf{z})) = 0 \text{ in } \mathbb{Z} \\ \mathbf{z} = (\mathbf{x}, \mathbf{f}) \in \mathbb{Z}^k \times \mathbb{Z}^\ell \end{array} \right\}$$

Remark 7.8. $\mathcal{R}_{\text{CM}}$ is related to PIMSAT and $\mathcal{R}_{\text{Lin-BitsDual}}^{\text{virtual}}$ in the sense that $\mathcal{R}_{\text{CM}}$ is the result of taking $\mathcal{R}_{\text{PIMSAT}}(\{0, 1\}, \mathbf{p}_{\text{CM}}, \perp)$ (i.e. the PIMSAT relation with $R = \mathbb{Z}, d = 0, B = 1$ and no ideals) and replacing the vector input-witness vector in PIMSAT by a vector of the form $\mathbf{h} = \pi_2^{-1}(A \cdot \pi_2(\mathbf{f}))$.

In our implementation and in ?? we arithmetize SHA-256 as an instance of $\mathcal{R}_{\text{CM}}$.

In ?? [Albert: Technical overview](#) we described an IOP for $\mathcal{R}_{\text{CM}}$. This IOP is simply the result of instantiating the IOP in [Construction 7.3](#) for $\mathcal{R}_{\text{PIMSAT}}(\{0, 1\}, \mathbf{p}_{\text{CM}}, \perp)$, and using the $\mathbb{F}2\mathbb{Z}$ IOP in [Construction 5.1](#) for the relation $\mathcal{R}_{\text{Lin-BitsDual}}^{\text{virtual}}$ (cf. [Section 5](#)) to prove the final inner product claims of [Construction 7.3](#) (cf. Step 4), instead of using the plain $\mathbb{F}2\mathbb{Z}$ version from [Construction 4.8](#).

Remark 7.9 (The $\mathcal{R}_{\text{CM}}$ relation is NP-complete). Any Boolean circuit can be arithmetized as an instance of $\mathcal{R}_{\text{CM}}$. Indeed, since any boolean circuit can be written as a system of equations using the operations XOR and AND, it suffices to prove that, given $x, y, z \in \{0, 1\}$, we can express both of the following constraints as an instance of $\mathcal{R}_{\text{CM}}$:

$$x \text{ XOR } y = z, \quad x \text{ AND } y = z.$$

Indeed, $x \text{ XOR } y = z$ if and only if $\pi_2^{-1}(\pi_2(x) + \pi_2(y)) = z$, where here $+$ means addition modulo 2. Further, $x \text{ AND } y = z$ if and only if

$$2 \cdot z = x + y - \pi_2^{-1}(\pi_2(x) + \pi_2(y)).$$

One can verify this last equivalence for example by directly inspecting that it holds for the four possible combinations of $x, y \in \{0, 1\}$.

[Albert: Todo: Expressiveness result of Remco](#)

8 Experiments

8.1 Instantiation

Remark 8.1 (The intermediate IOR $\Pi_{\mathbb{K}}$ as a batched sumcheck). [Albert: Scratch. Maybe remove](#) In many reasonable instances of $\mathcal{R}_{\text{PIMSAT}}$ (e.g. R1CS, CCS, or AIR-like instantiations), we have that

$$\mathbf{p}_{\mathbf{b}}^{(i)}(Y_1, \dots, Y_{k+\ell}) = \mathbf{p}'^{(i)}(\mathbf{b}, Y_1, \dots, Y_{k+\ell}) \quad \text{for all } i \in [\iota],$$

for some polynomials $\mathbf{p}'^{(i)}$ on $n' + k + \ell$ [Albert: \$\ell_0\$?](#) variables.

In this case, one can take the IOR $\Pi_{\mathbb{K}}$ from $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, \hat{\mathbf{p}})$ to $\mathcal{R}_{\text{Lin}}(\mathbb{K})$ from at Step 3 of [Construction 7.3](#) to consist simply in applying the batched sumcheck protocol over $\mathbb{K}$ for the claims

$$\sum_{\mathbf{b} \in \{0,1\}^{n'}} \pi(\mathbf{p}'^{(i)})(\mathbf{b}, \pi(\mathbf{x}), \pi(\mathbf{f})) \cdot \text{eq}(\mathbf{b}, \mathbf{r}) = \pi(u_i) \in \mathbb{K} \quad \text{for all } i \in [\iota].$$

At the end of batched sumcheck, they are left with a claim of the form

$$\pi(\mathbf{p}'^{(i)})(\mathbf{r}^{(1)}, \pi(\mathbf{x}), \pi(\mathbf{f})) = u'_i \in \mathbb{K} \quad \text{for all } i \in [\iota] \quad (27)$$

for some $\mathbf{r}^{(1)} \in \mathbb{K}^{\nu}$, $u'_i \in \mathbb{K}$.

....

...

The claim (27) forms a valid instance-witness for the PESAT relation (cf. [Definition 3.2](#)) over $\mathbb{K}$ with polynomial constraints given by $\pi(\mathbf{p}'^{(i)})(\mathbf{r}^{(1)}, Y_1, \dots, Y_{k+\ell}) - u'_i$, i.e. for such constraints and corresponding parameters

$$(\pi(\mathbf{x}), \perp, \pi(\mathbf{f})) \in \mathcal{R}_{\text{PESAT}}.$$

Accordingly, prover and verifier use the IOR $\Pi_{\mathbb{K}}$ to reduce (27) to a claim of the form

$$\widetilde{\pi(\mathbf{f})}(\mathbf{r}^{(2)}) = \mu \quad (28)$$

for some $\mathbf{r}^{(2)} \in \mathbb{K}^{\log \ell}$ and some $\mu \in \mathbb{K}$. In other words, $((\mathbf{r}^{(2)}, \mu), \perp, \pi(\mathbf{f})) \in \mathcal{R}_{\text{mle}}$ over $\mathbb{K}$.

8.2 Experimental results

8.2.1 Raw performance of $\mathbb{F}2\mathbb{Z}$

We measure the performance of [Construction 4.8](#) for committing to witnesses $\mathbf{f} \in \{0,1\}^{\ell} \subseteq \mathbb{Z}^{\ell}$, and proving claims of the form $\langle \mathbf{v}, \pi_q(\mathbf{f}) \rangle = \mu$ over a finite field $\mathbb{F}_{q^d}$, where π_q denotes reduction modulo a prime q , and $\mathbf{v} \in \mathbb{F}_{q^d}^{\ell}$. We instantiate [Construction 4.8](#) following the guideline in [Section 4.7](#). For WHIR we use a Reed-Solomon code of rate $1/8$ over $\mathbb{F}_{2^{128}}$.

In ?? we present performance for the case $q \leq 2^{105}$, $d = 1$, in which case so the optional step of Round 1 of [Construction 4.8](#) is omitted (cf. [Section 4.7](#)). In ?? we make the same measurement for the case $d = 5$ and $q \leq 2^{32}$.

8.2.2 Integer and modular multiplication

In ?? we measure the cost of proving 2^i integer multiplications of the form $x \cdot y = z$, for x, y random 32-bit integers, and z a 64-bit integer. We use our IOP for the PIMSAT relation [Construction 7.3](#). The specific form of the PIMSAT relation is as follows: the witness and the constraints are over $\mathbb{Z}$

8.2.3 SHA-256 hashing

SHA-256 compression overview We begin with a brief overview of SHA-256 [[Nat15](#)] to establish notation, then formalize the relation we seek to arithmetize.

For the purposes of SHA-256, a *word* is a ≤ 32 string of bits, which we represent as a binary integer polynomial, i.e. as an element of $\{0,1\}^{<32}[X]$. The SHA-256 compression function receives

as input a 512-bit string, represented as sixteen *message words* $(M_t)_{t \in [16]} \in \{0, 1\}^{<32}[X]^{16}$. The algorithm also uses 64 fixed *round constants* $(K_t)_{t \in [64]} \in \{0, 1\}^{<32}[X]^{64}$.

We let

$$\begin{aligned}\psi_2 : \mathbb{Z}[X] &\rightarrow \mathbb{Z} \\ a(X) &\mapsto a(2)\end{aligned}\tag{29}$$

be the ring homomorphism sending integer polynomials to their evaluation at $X = 2$. Notice that, given $a \in \{0, 1\}^{<32}[X]$, the integer $\psi_2(a) \in \mathbb{Z}$ is the integer whose bit-representation coincides with the coefficients of a . The restriction

$$\psi_2 : \{0, 1\}^{<32}[X] \longrightarrow \{0, \dots, 2^{32} - 1\}$$

is a bijection; throughout this subsection, ψ_2^{-1} denotes the inverse of this restricted map, not an inverse on all of $\mathbb{Z}[X]$.

SHA-256 is built from the following six auxiliary functions. For $x, y, z \in \{0, 1\}^{<32}[X]$, define:

$$\begin{aligned}\Sigma_0(x) &:= \text{ROTR}^2(x) \text{ XOR } \text{ROTR}^{13}(x) \text{ XOR } \text{ROTR}^{22}(x) && \in \{0, 1\}^{<32}[X] \\ \Sigma_1(x) &:= \text{ROTR}^6(x) \text{ XOR } \text{ROTR}^{11}(x) \text{ XOR } \text{ROTR}^{25}(x) && \in \{0, 1\}^{<32}[X] \\ \text{Ch}(x, y, z) &:= (x \text{ AND } y) \text{ XOR } (\neg x \text{ AND } z) && \in \{0, 1\}^{<32}[X] \\ \text{Maj}(x, y, z) &:= (x \text{ AND } y) \text{ XOR } (x \text{ AND } z) \text{ XOR } (y \text{ AND } z) && \in \{0, 1\}^{<32}[X] \\ \sigma_0(x) &:= \text{ROTR}^7(x) \text{ XOR } \text{ROTR}^{18}(x) \text{ XOR } \text{SHR}^3(x) && \in \{0, 1\}^{<32}[X] \\ \sigma_1(x) &:= \text{ROTR}^{17}(x) \text{ XOR } \text{ROTR}^{19}(x) \text{ XOR } \text{SHR}^{10}(x) && \in \{0, 1\}^{<32}[X]\end{aligned}\tag{30}$$

Here ROTR^r rotates a word right by r positions, SHR^r shifts it right by r positions and fills the high positions with zero, and XOR , AND , $\neg$ act coefficient-wise on bits.

Message schedule. At the beginning of the SHA-256 compression function, the sixteen input message words $(M_t)_{t \in [16]}$ are expanded into a so-called *message schedule*, which consists of 64 words $(W_t)_{t \in [64]} \in (\{0, 1\}^{<32}[X])^{64}$ defined as follows:

$$W_t := \begin{cases} M_t & t \in [16], \\ \psi_2^{-1}(\psi_2(W_{t-16}) + \psi_2(\sigma_0(W_{t-15})) + \psi_2(W_{t-7}) + \psi_2(\sigma_1(W_{t-2})) \bmod 2^{32}) & t \in \{17, \dots, 64\}. \end{cases}\tag{31}$$

Compression loop. After the message schedule is computed, SHA-256 performs the so-called *compression loop*. Throughout it, the function maintains an 8-word internal state, conventionally labeled as

$$(a, b, c, d, e, f, g, h) \in \{0, 1\}^{<32}[X]^8.$$

The compression loop iterates 64 rounds. At each round $t \in [64]$, the state entering that round is denoted $(a_t, b_t, c_t, d_t, e_t, f_t, g_t, h_t) \in \{0, 1\}^{<32}[X]^8$. The per-round update is given by, for $t \in [64]$,

$$\begin{aligned}T_{1,t} &:= \psi_2^{-1}(\psi_2(h_t) + \psi_2(\Sigma_1(e_t)) + \psi_2(\text{Ch}(e_t, f_t, g_t)) + \psi_2(K_t) + \psi_2(W_t) \bmod 2^{32}), \\ T_{2,t} &:= \psi_2^{-1}(\psi_2(\Sigma_0(a_t)) + \psi_2(\text{Maj}(a_t, b_t, c_t)) \bmod 2^{32}), \\ a_{t+1} &:= \psi_2^{-1}(\psi_2(T_{1,t}) + \psi_2(T_{2,t}) \bmod 2^{32}), \\ e_{t+1} &:= \psi_2^{-1}(\psi_2(d_t) + \psi_2(T_{1,t}) \bmod 2^{32}), \\ (b_{t+1}, c_{t+1}, d_{t+1}, f_{t+1}, g_{t+1}, h_{t+1}) &:= (a_t, b_t, c_t, e_t, f_t, g_t).\end{aligned}$$

For the feed-forward step, let

$$\begin{aligned} S^{\text{in}} &= (a_1, b_1, c_1, d_1, e_1, f_1, g_1, h_1), \\ S^{\text{fin}} &= (a_{65}, b_{65}, c_{65}, d_{65}, e_{65}, f_{65}, g_{65}, h_{65}). \end{aligned}$$

The compression output is the word vector H^{out} defined by

$$H_j^{\text{out}} = \psi_2^{-1}(\psi_2(S_j^{\text{in}}) + \psi_2(S_j^{\text{fin}}) \bmod 2^{32}) \quad (j \in \{0, \dots, 7\}).$$

8.2.4 Falcon signature verification

8.2.5 RSA and Poseidon

Albert: Let's compare to with the circuit of Limber

8.2.6 Hashing followed by Falcon signature verification

8.2.7 Comparison with Hyrax

Albert: Comparing with Hyrax may be more interesting than it sounds initially. Hyrax is pay-per-bit and opens on large prime fields: a profile that F2Z can adopt. Hence (maybe) projects using Hyrax could use F2Z as a drop-in replacement which is PQ secure and more efficient

References

- [Abd+26] A. Abdugafarov et al. “Zinc+: SNARKs for Polynomial Rings”. Manuscript, May 8, 2026. 2026 (cited on pp. 15, 35, 36).
- [ACFY25] G. Arnon, A. Chiesa, G. Fenzi, and E. Yogeve. “WHIR: Reed-Solomon Proximity Testing with Super-Fast Verification”. In: *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part IV*. Ed. by S. Fehr and P. Fouque. Vol. 15604. Lecture Notes in Computer Science. Springer, 2025, pp. 214–243. DOI: 10.1007/978-3-031-91134-7\8. URL: https://doi.org/10.1007/978-3-031-91134-7%5C_8 (cited on pp. 17, 22).
- [Nat15] National Institute of Standards and Technology. *Secure Hash Standard (SHS)*. Tech. rep. FIPS PUB 180-4. U.S. Department of Commerce, 2015. URL: <https://doi.org/10.6028/NIST.FIPS.180-4> (cited on p. 42).
- [Set19] S. Setty. *Spartan: Efficient and general-purpose zkSNARKs without trusted setup*. Cryptology ePrint Archive, Paper 2019/550. <https://eprint.iacr.org/2019/550>. 2019. URL: <https://eprint.iacr.org/2019/550> (cited on p. 37).
- [Sta21] StarkWare. *ethSTARK Documentation*. Online. Version 1.2. 2021. URL: <https://eprint.iacr.org/2021/582> (cited on p. 37).
- [STW23] S. Setty, J. Thaler, and R. Wahby. *Customizable constraint systems for succinct arguments*. Cryptology ePrint Archive, Paper 2023/552. 2023. URL: <https://eprint.iacr.org/2023/552> (cited on p. 36).

A Deferred proofs

Albert: Notation changed significantly recently. REVIEW

A.1 Completeness and soundness of Construction 4.12

Albert: I wrote this quite quickly. It may need polishing

The result follows from Theorem 4.10 and the fact that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{Lin-BitsDual}}(M_{R \leq B}, \psi, \mathbb{K}, \mathcal{C})$ if and only if (12) holds (due to Lemma 4.11). More concretely: perfect completeness is immediate. For round-by-round knowledge soundness, let KState' be a state function for Construction 4.8. Then we define a state function KState for Construction 4.12 as, when tr is neither an empty transcript or a transcript containing only a prover message (and no verifier messages),

$$\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = \text{KState}'(\mathbf{x}', \mathbf{y}, \text{tr}, \mathbf{w}),$$

where if $\mathbf{x} = (\mathbf{v}, \mu)$ then $\mathbf{x}' = (\mathbf{u}, \mu)$ is defined as in ???. For empty transcripts we let $\text{KState}(\mathbf{x}, \mathbf{y}, \mathbf{w}) = 1$ if and only if $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{Lin-BitsDual}}(M_{R \leq B}, \psi, \mathbb{K}, \mathcal{C}_1^{\equiv \ell_2})^\delta$. If tr is a transcript containing uniquely the first prover message, then we let $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = \text{KState}(\mathbf{x}, \mathbf{y}, \mathbf{w})$.

Further, let $\mathcal{E}'$ be an extractor for Construction 4.8. We define an extractor $\mathcal{E}$ for Construction 4.12 as follows: if tr is neither empty nor consisting in just one prover message, $\mathcal{E}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) := \mathcal{E}'(\mathbf{x}', \mathbf{y}, \text{tr}, \mathbf{w})$. For tr empty, we let $\mathcal{E}(\mathbf{x}, \mathbf{y}, \mathbf{w})$ proceeds as follows: first it computes

$$\mathbf{g} = \mathcal{E}'(\mathbf{x}', \mathbf{y}, \mathbf{w}).$$

If $\mathbf{g} \in \{0, 1\}^{\ell_0 \cdot B \cdot d} \subseteq \mathbb{Z}^{\ell_0 \cdot B \cdot d}$, then let $\mathbf{f} \in M_{R \leq B}^{\ell_0}$ such that $\text{bits}(\mathbf{f}) = \mathbf{g}$, concretely, index $\mathbf{f}$ as $\mathbf{g} = (\mathbf{g}_{ijk})_{i \in [\ell_0], j \in [d], k \in [B]}$. Let $\phi = \text{id}$ be the identity map if $R = \mathbb{Z}$, and $\phi = \pi_q$ if R is a field of characteristic q . Then output

$$\mathbf{f} := \sum_{j \in [d], k \in [B]} \gamma_j \cdot 2^{k-1} \cdot \phi(\mathbf{g}_{*jk}),$$

where $\mathbf{g}_{*jk} = (\mathbf{g}_{ijk})_{i \in [\ell_0]} \in \{0, 1\}^{\ell_0} \subseteq \mathbb{Z}^{\ell_0}$. Then $\text{bits}(\mathbf{f}) = \mathbf{g}$ Albert: more detail?.

If tr is a transcript consisting of only one prover message, we let $\mathcal{E}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = \mathcal{E}(\mathbf{x}, \mathbf{y}, \mathbf{w})$.

For the last two cases, we claim that

$$((\mathbf{u}, \mu), \mathbf{y}, \mathbf{g}) \in \mathcal{R}_{\text{Lin-BitsDual}}(\{0, 1\}, \pi_q, \mathbb{K}, \mathcal{C}_1^{\ell_2}) \Leftrightarrow ((\mathbf{v}, \mu), \mathbf{y}, \mathbf{f}) \in \mathcal{R}_{\text{Lin-BitsDual}}(M_{R \leq B}, \psi, \mathbb{K}, \mathcal{C}_1^{\equiv \ell_2}).$$

Indeed, this follows from ??, which yields

$$\langle \psi(\mathbf{f}), \mathbf{v} \rangle = \langle \pi_q(\text{bits}(\mathbf{f})), \mathbf{u} \rangle = \langle \pi_q(\mathbf{g}), \mathbf{u} \rangle.$$

Albert: increase detail?.

We now prove that, for any transcript tr where verifier is about to send its j -th message

$$P_j := \Pr_{\rho} \left[\exists \mathbf{w} \left| \begin{array}{l} \wedge \text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr}, \rho, \mathbf{w})) = 0 \\ \wedge \text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \rho, \mathbf{w}) = 1 \end{array} \right. \right] \leq \kappa'_j.$$

where κ'_j is the j -th error in ??.

Indeed, the inequality is immediate if tr does not consist of a single prover message, since then KState and $\mathcal{E}$ coincide with the state KState' and extractor $\mathcal{E}'$ of [Construction 4.8](#). For P_1 , namely if tr consists of a single prover message, then

$$P_1 = \Pr_{\rho} \left[\exists \mathbf{w} \mid \begin{array}{l} \wedge \text{KState}(\mathbf{x}, \mathbf{y}, \mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr}, \rho, \mathbf{w})) = 0 \\ \wedge \text{KState}'(\mathbf{x}', \mathbf{y}, \text{tr}, \rho, \mathbf{w}) = 1 \end{array} \right]$$

If $\text{KState}'(\mathbf{x}', \mathbf{y}, \text{tr}, \rho, \mathbf{w}) = 1$ then with probability $1 - \kappa_0^*$, $\mathcal{E}'(\mathbf{x}', \mathbf{y}, \text{tr}, \mathbf{w})$ outputs $\mathbf{g}$ such that $\text{KState}'(\mathbf{x}', \mathbf{y}, \text{tr}, \mathbf{g}) = 1$, hence, in this case, $\text{KState}'(\mathbf{x}', \mathbf{y}, \mathbf{g}) = 1$, which holds if and only if $(\mathbf{x}', \mathbf{y}, \mathbf{g}) \in \mathcal{R}_{\text{Lin-BitsDual}}(\{0, 1\}, \pi_q, \mathbb{K}, \mathcal{C}_1^{\equiv \ell_2})$. Hence, with probability at least $1 - \kappa_0^*$, $\mathcal{E}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w})$ outputs $\mathbf{f}$ such that $(\mathbf{x}, \mathbf{y}, \mathbf{f}) \in \mathcal{R}_{\text{Lin-BitsDual}}(M_{R_{\leq B}}, \psi, \mathbb{K}, \mathcal{C}_1^{\equiv \ell_2})$. It follows that $P_1 \leq \kappa_0^*$.

A.2 Completeness and soundness of [Construction 7.3](#)

To prove [Theorem 7.5](#), we will need to introduce an extension of the definition of $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, \mathbf{p})$ and $\mathcal{R}_{\text{Lin}}(\mathbb{K})$ so that these relations can hold witnesses in $R_{\leq B}^{\leq d}$ and their encodings over a binary field. Namely, given a field of characteristic q and a π_q -module homomorphism (cf. ??) $\psi_{\mathbb{K}} : R^{< d}[X] \rightarrow \mathbb{K}$, we define:

$$\mathcal{R}_{\text{PESAT}}(R_{\leq B}^{\leq d}, \mathbb{K}, \mathbf{p}, \psi_{\mathbb{K}}, \mathcal{C}) := \left\{ \begin{array}{l} \left(\begin{array}{l} \mathbf{x} = \mathbf{x} \in \mathbb{K}^k, \\ \mathbf{y} = f \in (\mathbb{F}_{2^\nu})^m, \\ \mathbf{w} = \mathbf{f} \in (R_{\leq B}^{\leq d})^{\ell_0} \end{array} \right) \mid \begin{array}{l} f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\text{bits}(\mathbf{f})) \\ \wedge \mathbf{p}_i(\mathbf{x}, \psi_{\mathbb{K}}(\mathbf{f})) = 0 \text{ for all } i \in [n] \end{array} \right\}$$

Note that when $\mathbf{p}$ contains a single linear polynomial, then

$$\mathcal{R}_{\text{PESAT}}(R_{\leq B}^{\leq d}, \mathbb{K}, \mathbf{p}, \psi, \mathcal{C}) = \mathcal{R}_{\text{Lin-BitsDual}}(R_{\leq B}^{\leq d}, \psi, \mathbb{K}, \mathcal{C}).$$

We will also need an encoded version of PIMSAT: we define $\mathcal{R}_{\text{PIMSAT}}(R_{\leq B}^{\leq d}, \mathbf{p}, \mathcal{I}, \mathcal{C})$ as

$$\mathcal{R}_{\text{PIMSAT}}(R_{\leq B}^{\leq d}, \mathbf{p}, \mathcal{I}, \mathcal{C}) = \left\{ \begin{array}{l} \left(\begin{array}{l} \mathbf{x} = \mathbf{x} \in (R_{\leq B}^{\leq d}[X])^k, \\ \mathbf{y} = f \in \mathbb{F}_{2^\nu}^m, \\ \mathbf{w} = \mathbf{f} \in (R_{\leq B}^{\leq d}[X])^\ell \end{array} \right) \mid \begin{array}{l} f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\text{bits}(\mathbf{f})), \\ \mathbf{p}_j^{(i)}(\mathbf{x}, \mathbf{f}) \in \mathcal{I}_i \text{ for all } j \in [n], i \in [\ell] \end{array} \right\}$$

As usual, for the relations $\mathcal{R}$ that contain the constraint $f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\text{bits}(\mathbf{f}))$, we also consider their *relaxed* version $\mathcal{R}^\delta$, where such constraint is replaced by $\Delta(f, \text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\text{bits}(\mathbf{f}))) < \delta$.

Lemma A.1. *Let $\mathbb{K}$ be a finite field of characteristic q and let $\psi_{\mathbb{K}} : R^{< d}[X] \rightarrow \mathbb{K}$ be a π_q -module homomorphism [Albert: correct terminology? And above?](#). Let $\Pi_{\mathbb{K}}$ be a $t_{\mathbb{K}}$ -round perfectly complete IOR from $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, \mathbf{p})$ to $\mathcal{R}_{\text{Lin}}(\mathbb{K})$ with round-by-round knowledge soundness (with no relaxation) with errors $(\kappa_{\Pi_{\mathbb{K}}, i})_{i \in [t_{\mathbb{K}}]}$.*

Let $\delta \in (0, 1)$. Then there is an IOR $\Pi'_{\mathbb{K}}$ from $\mathcal{R}_{\text{PESAT}}(R_{\leq B}^{\leq d}, \mathbb{K}, \mathbf{p}, \psi_{\mathbb{K}}, \mathcal{C})$ to $\mathcal{R}_{\text{Lin-BitsDual}}(R_{\leq B}^{\leq d}, \psi_{\mathbb{K}}, \mathbb{K}, \mathcal{C})$ with $t_{\mathbb{K}}$ rounds, perfectly complete, and with round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{PESAT}}(R_{\leq B}^{\leq d}, \mathbb{K}, \mathbf{p}, \psi_{\mathbb{K}}, \mathcal{C})^\delta, \mathcal{R}_{\text{Lin-BitsDual}}(R_{\leq B}^{\leq d}, \psi_{\mathbb{K}}, \mathbb{K}, \mathcal{C})^\delta)$ and with errors $(\kappa_{\Pi_{\mathbb{K}}, i})_{i \in [t_{\mathbb{K}}]}$.

Albert: Could there be an issue with extracting the correct bit size B ? No because prover outputs $\mathbf{f}$, not $\mathbf{f}'$ This IOR $\Pi'_{\mathbb{K}}$ is exactly $\Pi_{\mathbb{K}}$, except that prover and verifier receive an oracle f , which

they do not use throughout the IOR; and additionally the prover receives a witness $\mathbf{f}$ with entries in $R_{\leq B}^d$ instead of in $\mathbb{K}$. Then the prover of $\Pi'_{\mathbb{K}}$ proceeds exactly as the prover of $\Pi_{\mathbb{K}}$ after replacing $\mathbf{f}$ by $\psi_{\mathbb{K}}(\mathbf{f})$. At the end of the IOR, both prover and verifier of $\Pi'_{\mathbb{K}}$ output the same $\mathbf{x}$ output by $\Pi_{\mathbb{K}}$. Additionally, they output $\mathbf{y} = f$, and the prover of $\Pi'_{\mathbb{K}}$ outputs any vector in the preimage of $\psi_{\mathbb{K}}^{-1}(\mathbf{f}')$, where $\mathbf{f}'$ is the witness output by the prover of $\Pi_{\mathbb{K}}$.

Proof. **Albert:** TODO. Anywhere in the literature? □

We build three IORs Π_1, Π_2, Π_3 whose composition closely mimics [Construction 7.3](#).

Description of Π_1 . The IOR Π_1 is a two-round IOR from $\mathcal{R}_{\text{PIMSAT}}(R_{\leq B}^d, \mathbf{p}, \mathcal{I}, \mathcal{C})$ to $\mathcal{R}_{\text{PESAT}}(R_{\leq B}^d, \mathbb{K}, \hat{\mathbf{p}}, \pi, \mathcal{C})$, for some polynomials $\hat{\mathbf{p}}$ and π_q -module homomorphism $\pi : R_{\leq B}^d \rightarrow \mathbb{K}$. In Π_1 , verifier and prover receive $(\mathbf{x}, f)$ and $(\mathbf{x}, f, \mathbf{f}) \in \mathcal{R}_{\text{PIMSAT}}(R_{\leq B}^d, \mathbf{p}, \mathcal{I}, \mathcal{C})$ as inputs, respectively. In the first round, prover and verifier simulate Round 1 of [Construction 7.3](#), skipping the first item (computing and sending the encoding f of $\mathbf{f}$), since f is already in the input of the prover and verifier. The prover instead sends an empty message $\perp$. For the second round of Π_1 , prover and verifier simulate Round 2 of [Construction 7.3](#). At the end of Π_1 , prover and verifier output

$$(\pi(\mathbf{x}), f, \mathbf{f}) \in \mathcal{R}_{\text{PESAT}}(R_{\leq B}^d, \mathbb{K}, \hat{\mathbf{p}}, \pi, \mathcal{C}),$$

where $\hat{\mathbf{p}}$ and π are determined during the execution of Π_1 **Albert:** This may be formally incorrect: I don't think current definitions support relations in an IOR changing depending on the run of the IOR, so we'd need to add the parameters in $\mathbf{x}$.

Description of Π_2 . The IOR Π_2 is an IOR from $\mathcal{R}_{\text{PESAT}}(R_{\leq B}^d, \mathbb{K}, \hat{\mathbf{p}}, \pi, \mathcal{C})$ to $\mathcal{R}_{\text{Lin}}(R_{\leq B}^d, \mathbb{K}, \hat{\mathbf{p}}, \mathcal{C})$. Prover and verifier receive $(\pi(\mathbf{x}), f)$ and $(\pi(\mathbf{x}), f, \mathbf{f})$ as inputs. Then they apply the IOR $\Pi'_{\mathbb{K}}$ given by [Lemma A.1](#), where $\Pi_{\mathbb{K}}$ is the IOR applied in Item 3 of [Construction 7.3](#). They use as inputs $(\pi(\mathbf{x}), f)$ and $(\pi(\mathbf{x}), f, \mathbf{f})$, respectively. By [Lemma A.1](#), $\Pi'_{\mathbb{K}}$ is an IOR from $\mathcal{R}_{\text{PESAT}}(R_{\leq B}^d, \mathbb{K}, \hat{\mathbf{p}}, \pi, \mathcal{C})$ to $\mathcal{R}_{\text{Lin}}(R_{\leq B}^d, \mathbb{K}, \pi, \mathcal{C})$, perfectly complete, with $t_{\mathbb{K}}$ rounds, and with round-by-round knowledge with relaxation $(\mathcal{R}_{\text{PESAT}}(R_{\leq B}^d, \mathbb{K}, \hat{\mathbf{p}}, \pi, \mathcal{C})^\delta, \mathcal{R}_{\text{Lin}}(R_{\leq B}^d, \mathbb{K}, \pi, \mathcal{C})^\delta)$ and with errors $(\kappa_{\Pi_{\mathbb{K}}, i})_{i \in [t_{\mathbb{K}}]}$.

Description of Π_3 The IOR Π_3 is an IOP for $\mathcal{R}_{\text{Lin}}(R_{\leq B}^d, \mathbb{K}, \pi, \mathcal{C})$. It consists in applying Phase 4 of [Construction 7.3](#), i.e. simulating [Construction 4.12](#) with inputs $((\mathbf{v}, \mu), f, \mathbf{f})$. By [Theorem 4.14](#) and the hypothesis of [Theorem 7.5](#), [Construction 4.12](#) is a $(\tau_{\text{ext}} + t_{\text{red}} + t_{\text{iopp}})$ -round perfectly complete IOP for $\mathcal{R}_{\text{Lin}}(R_{\leq B}^d, \mathbb{K}, \pi, \mathcal{C})$ with round-by-round knowledge with relaxation $(\mathcal{R}_{\text{Lin}}(R_{\leq B}^d, \mathbb{K}, \pi, \mathcal{C})^\delta, \perp)$, with errors $\kappa_{\text{lift-proj}}^*, (\kappa_{\text{red}i})_{i \in [t_{\text{red}}]}, (\kappa_{\text{iopp}})_{i \in [t_{\text{iopp}}]}$.

The next lemma states that the composition of Π_1, Π_2 and Π_3 is an IOR whose completeness and soundness implies completeness and soundness of [Construction 7.3](#).

Lemma A.2. *Let Π_{123} be the composition of Π_1, Π_2 and Π_3 . Then Π_{123} is a $(2 + t_{\mathbb{K}} + \tau_{\text{ext}} + t_{\text{red}} + t_{\text{iopp}})$ -round IOP for $\mathcal{R}_{\text{PIMSAT}}(R_{\leq B}^d, \mathbf{p}, \mathcal{I}, \mathcal{C})$. Assume Π_{123} is perfectly complete and, for any $\delta \in (0, 1)$, has round-by-round knowledge with relaxation $(\mathcal{R}_{\text{PIMSAT}}(R_{\leq B}^d, \mathbf{p}, \mathcal{I}, \mathcal{C})^\delta, \perp)$ with errors $(\kappa'_j)_{j \in [2 + t_{\mathbb{K}} + \tau_{\text{ext}} + t_{\text{red}} + t_{\text{iopp}}]}$.*

Then [Construction 7.3](#) is a $(2 + t_{\mathbb{K}} + \tau_{\text{ext}} + t_{\text{red}} + t_{\text{iopp}})$ -round perfectly complete IOP for $\mathcal{R}_{\text{PIMSAT}}(R_{\leq B}^{\leq d}, \mathbf{p}, \mathcal{I})$, and has round-by-round knowledge without relaxation with errors $(\kappa_j)_{j \in [2+t_{\mathbb{K}}+\tau_{\text{ext}}+t_{\text{red}}+t_{\text{iopp}}]}$ defined as

$$\kappa_j(\mathbf{x}) := \max_{f \text{ s.t. } (\mathbf{x}, f) \notin \mathcal{R}_{\text{PIMSAT}}(R_{\leq B}^{\leq d}, \mathbf{p}, \mathcal{I}, \mathcal{C})^\delta} \kappa_j(\mathbf{x}, f)'.$$

Albert: Need to argue somewhere that these errors are negligible if κ'_j are – I'd prefer to just use the errors from κ'_j directly. This may be related with the issue about composition of IORs and having ill-defined errors

Proof. Deferred to [Section A.2.1](#). □

Due to [Lemma A.2](#), it suffices to show that Π_{123} is perfectly complete and has the desired soundness properties. We prove this by using the composition of IOR result [Remark 3.8](#). Both Π_2 and Π_3 already have the desired properties by definition and hypotheses of [Theorem 4.10](#). Thus it suffices to show the following:

Lemma A.3 (Completeness and soundness of Π_1). *Let $\delta \in (0, 1)$. The IOR Π_1 is a perfectly complete 2-round IOR from $\mathcal{R}_{\text{PIMSAT}}(R_{\leq B}^{\leq d}, \mathbf{p}, \mathcal{I}, \mathcal{C})$ to $\mathcal{R}_{\text{PESAT}}(R_{\leq B}^{\leq d}, \mathbb{K}, \hat{\mathbf{p}}, \pi, \mathcal{C})$ with round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{PIMSAT}}(R_{\leq B}^{\leq d}, \mathbf{p}, \mathcal{I}, \mathcal{C})^\delta, \mathcal{R}_{\text{PESAT}}(R_{\leq B}^{\leq d}, \mathbb{K}, \hat{\mathbf{p}}, \pi, \mathcal{C})^\delta)$ and with errors*

$$\begin{aligned} \kappa_1 &= \frac{B_{\max} \cdot (1 + \max(\nabla(\mathbf{p}) - \min_{i \in [\iota]} \deg(\mathcal{I}_i), 0))}{|\mathcal{P}| \cdot \lfloor \log(|\mathbb{K}_{\min}|) \rfloor} + \frac{1}{|\mathbb{K}_{\min}|} \\ \kappa_2 &= \frac{\nabla(\mathbf{p})}{|\mathbb{K}_{\min}|} + \frac{1}{|\mathbb{K}_{\min}|}, \end{aligned}$$

Proof. **Albert: Probably needs some polishing and review** Indeed, let $\mathbf{x} = \mathbf{x}, \mathbf{y} = f$ be an input and oracle for $\mathcal{R}_{\text{PIMSAT}}(R_{\leq B}^{\leq d}, \mathbf{p}, \mathcal{I}, \mathcal{C})$. For any $\mathbf{w}$ define $\text{KState}(\mathbf{x}, f, \mathbf{w}) = 1$ if and only if $(\mathbf{x}, f, \mathbf{w}) \in \mathcal{R}_{\text{PIMSAT}}(R_{\leq B}^{\leq d}, \mathbf{p}, \mathcal{I}, \mathcal{C})^\delta$. Given a full transcript $\text{tr} = (\perp, (\mathbb{K}, \mathbf{r}), (u_1, \dots, u_\iota), (\theta_1, \dots, \theta_\iota, \alpha))$ for Π_1 , define $\text{KState}(\mathbf{x}, f, \text{tr}, \mathbf{w}) = 1$ if and only if $(\pi(\mathbf{x}), f, \pi(\mathbf{f})) \in \mathcal{R}_{\text{PESAT}}(R_{\leq B}^{\leq d}, \mathbb{K}, \hat{\mathbf{p}}, \pi, \mathcal{C})^\delta$, where $\pi = \pi_{q, \alpha}$, q being the characteristic of $\mathbb{K}$ and α one of the verifier's messages in tr . For partial transcripts where the prover tr is about to send a message m , let $\text{KState}(\mathbf{x}, f, \text{tr}, m, \mathbf{w}) = \text{KState}(\mathbf{x}, f, \text{tr}, \mathbf{w})$.

We now define $\text{KState}(\mathbf{x}, f, \text{tr}, \mathbf{w})$ where tr is nonempty and such that prover is about to move. The only undefined setting is when $\text{tr} = (\perp, (\mathbb{K}, \mathbf{r}))$. In this case we define $\text{KState}(\mathbf{x}, f, \text{tr}, \mathbf{w}) = 1$ if and only if

$$\Delta(f, \text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\text{bits}(\mathbf{w}))) < \delta$$

and $\pi_q(Q_{\mathbf{w}}^{(i)})(\mathbf{r}) \in \pi_q(\mathcal{I}_i)$ for all $i \in [\iota]$, where

$$Q_{\mathbf{w}}^{(i)}(\mathbf{Z}) := \sum_{\mathbf{b} \in \{0,1\}^{n'}} \mathbf{p}_{\mathbf{b}}^{(i)}(\mathbf{x}, \mathbf{w}) \cdot \text{eq}(\mathbf{b}, \mathbf{Z}) \in R[X].$$

Observe that KState satisfies the three conditions of [Definition 3.6](#).

We define the extractor $\mathcal{E}_{\text{rbr}}$ as $\mathcal{E}_{\text{rbr}}(\mathbf{x}, f, \text{tr}, \mathbf{w}) = \mathbf{w}$ for all $\mathbf{x}, f, \text{tr}, \mathbf{w}$. Let tr be a transcript for Π_1 where the verifier is about to move, so that either $\text{tr} = (\perp)$ or $\text{tr} = (\perp, (\mathbb{K}, \mathbf{r}), (u_i)_{i \in [\iota]})$. For

a transcript $\mathbf{tr} = (\perp)$, we need to bound the following probability, corresponding to the round-by-round knowledge soundness of Round 1 of Π_1 :

$$P_1 := \Pr_{\mathbb{K}, \mathbf{r}} \left[\exists \mathbf{w} \mid \begin{array}{l} \wedge \text{KState}(\mathbb{x}, f, \perp, \mathbf{w}) = 0 \\ \wedge \text{KState}(\mathbb{x}, f, \perp, (\mathbb{K}, \mathbf{r}), \mathbf{w}) = 1 \end{array} \right]$$

By our definition of KState , P_1 is the probability that

$$\pi_q(Q_{\mathbf{w}}^{(i)})(\mathbf{r}) \in \pi_q(\mathcal{I}_i) \text{ for all } i \in [\iota] \text{ and } \Delta(f, \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\text{bits}(\mathbf{w}))) < \delta \quad (32)$$

while at the same time

$$\mathbf{p}_{\mathbf{b}}^{(i)}(\mathbf{x}, \mathbf{w}) \notin \mathcal{I}_i \text{ for some } \mathbf{b} \in \{0, 1\}^{n'} \text{ and } i \in [\iota] \quad (33)$$

or $\Delta(f, \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\text{bits}(\mathbf{w}))) \geq \delta$. Clearly, the latter condition cannot hold if (32) holds; hence we will study the probability that (32) and (33) hold simultaneously. We ultimately want to prove that $P_1 \leq \kappa_1$.

Let E_1 be the event that (33) holds and $\pi_q(\mathbf{p}_{\mathbf{b}}^{(i)}(\mathbf{x}, \mathbf{w})) \in \pi_q(\mathcal{I}_i)$ for all $\mathbf{b}, i$. By Lemma 7.6,

$$\Pr_{\mathbb{K}, \mathbf{r}}[E_1] \leq \max_{\mathbf{b} \in \{0, 1\}^{n'}, \iota \in [\iota]} \left\{ \frac{B' \cdot (1 + \max(\deg(\mathbf{p}_{\mathbf{b}}^i(\mathbf{x}, \mathbf{f})) - \deg(\mathcal{I}_i) + 1, 0))}{|\mathcal{P}| \cdot \lfloor \log(q_{\min}) \rfloor} \right\},$$

where $\deg(\mathcal{I}_i)$ is the degree of a generator of $\mathcal{I}_i$; and B' is $\lceil \log(\|\mathbf{p}_{\mathbf{b}}^{(i)}(\mathbf{x}, \mathbf{f})\|_\infty) \rceil$. We have, letting $d_{\min} = \min_{i \in [\iota]} \deg(\mathcal{I}_i)$, [Albert: more detail?](#)

$$\begin{aligned} & \max_{\mathbf{b} \in \{0, 1\}^{n'}, i \in [\iota]} \{ \deg(\mathbf{p}_{\mathbf{b}}^i(\mathbf{x}, \mathbf{f})) - \deg(\mathcal{I}_i) \} \\ & \leq \nabla(\mathbf{p}) - d_{\min} = \max_{\mathbf{b} \in \{0, 1\}^{n'}, i \in [\iota]} \{ \deg_X(\mathbf{p}_{\mathbf{b}}^i) + (d - 1) \cdot \max_{t \in [k + \ell]} \deg_{Y_t}(\mathbf{p}_{\mathbf{b}}^i) \} + d - d_{\min}, \\ & \max_{\mathbf{b} \in \{0, 1\}^{n'}, i \in [\iota]} B' = \max_{\mathbf{b} \in \{0, 1\}^{n'}, i \in [\iota]} \lceil \log(\|\mathbf{p}_{\mathbf{b}}^{(i)}(\mathbf{x}, \mathbf{f})\|_\infty) \rceil = B_{\max}. \end{aligned}$$

Now, let E be the event that both (32) and (33) hold. Then

$$P_1 = \Pr_{\mathbb{K}, \mathbf{r}}[E] \leq \Pr_{\mathbb{K}, \mathbf{r}}[E \mid \neg E_1] + \Pr_{\mathbb{K}, \mathbf{r}}[E_1].$$

We next analyze $\Pr_{\mathbb{K}, \mathbf{r}}[E \mid \neg E_1]$. If E_1 does not occur, then the multilinear polynomial $\pi_q(Q_{\mathbf{w}}^{(i)})(\mathbf{Z})$ is not the zero polynomial in $\mathbb{K}[X]$. On the other hand, for E to occur, we need $\pi_q(Q_{\mathbf{w}}^{(i)})(\mathbf{r}) = 0$ in $\mathbb{K}$. Hence, by Schwartz-Zippel lemma, $\Pr_{\mathbb{K}, \mathbf{r}}[E \mid \neg E_1] \leq 1/|\mathbb{K}_{\min}|$. It follows that $P_1 \leq \kappa_1$, as needed.

Finally, we bound the following probability, corresponding to the round-by-round knowledge soundness of the second round of Π_1 .

$$P_2 := \Pr_{(\theta_i)_{i \in [\iota], \alpha}} \left[\exists \mathbf{w} \mid \begin{array}{l} \wedge \text{KState}(\mathbb{x}, f, \perp, (\mathbb{K}, \mathbf{r}), (u_i)_{i \in [\iota]}, \mathbf{w}) = 0 \\ \wedge \text{KState}(\mathbb{x}, f, \perp, (\mathbb{K}, \mathbf{r}), (u_i)_{i \in [\iota]}, ((\theta_i)_{i \in [\iota]}, \alpha), \mathbf{w}) = 1 \end{array} \right]$$

By definition,

$$\text{KState}(\mathbb{x}, f, \perp, (\mathbb{K}, \mathbf{r}), (u_i)_{i \in [\iota]}, ((\theta_i)_{i \in [\iota]}, \alpha), \mathbf{w}) = 1 \Leftrightarrow (\pi(\mathbb{x}), f, \pi(f)) \in \mathcal{R}_{\text{PESAT}}(R_{\leq B}^{\leq d}, \mathbb{K}, \hat{\mathbf{p}}, \mathcal{C})^\delta \quad (34)$$

and verifier does not abort [Albert: It's not completely transparent how verifier accepts or aborts in the definition of RBR KS](#); while

$$\text{KState}(\mathbb{x}, f, \perp, (\mathbb{K}, \mathbf{r}), (u_i)_{i \in [\iota]}, \mathbf{w}) = \text{KState}(\mathbb{x}, f, \perp, (\mathbb{K}, \mathbf{r}), \mathbf{w}) = 0$$

if and only if $\Delta(f, \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\text{bits}(\mathbf{w}))) \geq \delta$ or

$$\pi_q(Q_{\mathbf{w}}^{(i)})(\mathbf{r}) \notin \pi_q(\mathcal{I}_i) \text{ for some } i \in [\iota]. \quad (35)$$

Let E be the event that (34) and (35) hold. Again, $P_2 = \Pr_{(\theta_i), \alpha}[E]$, since (34) forces $\Delta(f, \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\text{bits}(\mathbf{w}))) < \delta$.

Now, if E occurs then

$$\sum_{i \in [\iota]} \sum_{\mathbf{b} \in \{0,1\}^{n'}} \theta_i \cdot \pi(\mathbf{p}_{\mathbf{b}}^{(i)})(\pi(\mathbf{x}), \pi(\mathbf{w})) \cdot \text{eq}(\mathbf{b}, \mathbf{r}) = \sum_{i \in [\iota]} \theta_i \cdot \pi(u_i) \in \mathbb{K},$$

and at the same time

$$\pi_q(Q_{\mathbf{w}}^{(i)})(\mathbf{r}) = \sum_{\mathbf{b} \in \{0,1\}^{n'}} \pi_q(\mathbf{p}_{\mathbf{b}}^{(i)}(\mathbf{x}, \mathbf{f})) \cdot \text{eq}(\mathbf{b}, \mathbf{r}) \notin \pi_q(\mathcal{I}_i) \subseteq \mathbb{K}[X]$$

for some $i \in [\iota]$. Since the verifier does not reject, $u_i \in \pi_q(\mathcal{I}_i)$ for all $i \in [\iota]$. Hence, if E holds, then for some i , we have $\pi_q(Q_{\mathbf{w}}^{(i)})(\mathbf{r}) \neq u_i$. In particular $h_i := \pi_q(Q_{\mathbf{w}}^{(i)})(\mathbf{r}) - u_i \in \mathbb{K}[X]$ is a nonzero polynomial with coefficients in $\mathbb{K}$.

Let E_1 be the event that E holds and $\pi(Q_{\mathbf{w}}^{(i)})(\mathbf{r}) = \pi(u_i)$ for all $i \in [\iota]$ (recall that $\pi = \pi_{q, \alpha}$ acts on $\mathbb{K}[X]$ by replacing X by α). Now, since $h_i(X)$ is a nonzero polynomial from $\mathbb{K}[X]$, we have that $\pi(h_i(X)) = h_i(\alpha) = 0$ with probability at most $\deg(h_i)/|\mathbb{K}_{\min}|$ by Schwartz-Zippel lemma. It follows that [Albert: Again, I'm not missing a union bound right?](#)

$$\begin{aligned} \Pr_{(\theta_i), \alpha}[E_1] &\leq \max_{i \in [\iota]} \frac{\deg(\pi_q(Q_{\mathbf{w}}^{(i)})(\mathbf{r}) - u_i)}{|\mathbb{K}_{\min}|} \leq \max_{i \in [\iota]} \frac{\max\{\deg(\pi_q(Q_{\mathbf{w}}^{(i)})(\mathbf{r})), d\}}{|\mathbb{K}_{\min}|} \\ &\leq \max_{\mathbf{b} \in \{0,1\}^{n'}, i \in [\iota]} \frac{\max\{\deg(\mathbf{p}_{\mathbf{b}}^{(i)}(\mathbf{x}, \mathbf{f})), d\}}{|\mathbb{K}_{\min}|} \leq \frac{\nabla(\mathbf{p})}{|\mathbb{K}_{\min}|}, \end{aligned}$$

where we used that $\deg_X(u_i) < d$; that $\pi_q(Q_{\mathbf{w}}^{(i)})(\mathbf{r}) \in \mathbb{K}[X]$ is a $\mathbb{K}$ -linear combination of the polynomials $\pi_q(\mathbf{p}_{\mathbf{b}}^{(i)}(\mathbf{x}, \mathbf{f})) \in \mathbb{K}[X]$ [Albert: More details about the last inequality?](#). Now,

$$\Pr[E] \leq \Pr[E \mid \neg E_1] + \Pr[E_1].$$

If E_1 does not hold, but E holds, then $\pi(Q_{\mathbf{w}}^{(i)})(\mathbf{r}) \neq \pi(u_i)$ for some $i \in [\iota]$. Then the polynomial on variables $U_1, \dots, U_\iota$,

$$Q'(U_1, \dots, U_\iota) := \sum_{i \in [\iota]} U_i \cdot \pi(Q_{\mathbf{w}}^{(i)})(\mathbf{r}) - \sum_{i \in [\iota]} U_i \cdot \pi(u_i)$$

is a nonzero multilinear polynomial with coefficients in $\mathbb{K}$. Now, $[E \mid \neg E_1]$ holds if and only if $Q'(\theta_1, \dots, \theta_\iota) = 0 \in \mathbb{K}$. By Schwartz-Zippel lemma, this probability is at most $1/|\mathbb{K}_{\min}|$. It follows that $P_2 \leq \kappa_2$, as required. $\square$

This completes the proof of [Theorem 7.5](#).

A.2.1 Proof of Lemma A.2

We prove Lemma A.2. Observe that the prover and verifier of Π_{123} proceed in exactly the same way as the prover and verifier of Construction 7.3 after the prover sends f in Item 1 of Round 1 (except that in Π_{123} the prover sends an initial empty message).

If Π_{123} is perfectly complete, then clearly Construction 7.3 is as well [Albert: more details?](#). Further, transcripts between Π_{123} and Construction 7.3 are the same, except that the prover in Π_{123} sends an empty first message when the prover in Construction 7.3 sends the encoding f . More precisely, a (partial) nonempty transcript of Construction 4.8 has the form (f, tr') for some tr' and some first prover message f , and the “mirror” transcript of Π_{123} is $(\perp, \text{tr}')$, with f being now part of the instance oracle.

From any knowledge state function with relaxation $\text{KState}_{\Pi_{123}}$ for Π_{123} , we define a state function KState for Construction 7.3 as follows: let tr be a partial or full transcript of Construction 7.3. If tr is not empty, write $\text{tr} = (f, \text{tr}')$ as above. Then we define

$$\text{KState}(\mathbb{x}, \perp, (f, \text{tr}'), \mathbf{w}) = \text{KState}_{\Pi_{123}}(\mathbb{x}, f, (\perp, \text{tr}'), \mathbf{w}).$$

If tr is empty, we define

$$\text{KState}(\mathbb{x}, \perp, \mathbf{w}) = 1 \Leftrightarrow (\mathbb{x}, \perp, \mathbf{w}) \in \mathcal{R}_{\text{PIMSAT}}.$$

Hence KState satisfies Condition 1 of Definition 3.6 (Empty transcript). We now show it satisfies Conditions 2 (Prover moves) and 3 (Full transcript), assuming $\text{KState}_{\Pi_{123}}$ satisfies Definition 3.6.

Let tr be a transcript of Construction 7.3 where the prover is about to move and $\text{KState}(\mathbb{x}, \perp, \text{tr}, \mathbf{w}) = 0$. If tr is nonempty, then $\text{tr} = (f, \text{tr}')$ as above, and for any next prover message m of Construction 7.3, $\text{KState}(\mathbb{x}, \perp, (\text{tr}, m), \mathbf{w}) = \text{KState}_{\Pi_{123}}(\mathbb{x}, f, (\perp, \text{tr}', m), \mathbf{w}) = 0$ because, since $\text{KState}_{\Pi_{123}}$ satisfies Definition 3.6, $\text{KState}_{\Pi_{123}}(\mathbb{x}, f, (\perp, \text{tr}'), \mathbf{w}) = \text{KState}(\mathbb{x}, \perp, (f, \text{tr}'), \mathbf{w}) = 0$. If tr is empty, then the next prover’s message is f . If $\text{KState}(\mathbb{x}, \perp, \mathbf{w}) = 0$, then $(\mathbb{x}, \perp, \mathbf{w}) \notin \mathcal{R}_{\text{PIMSAT}}$. Now $\text{KState}(\mathbb{x}, \perp, f, \mathbf{w}) = \text{KState}_{\Pi_{123}}(\mathbb{x}, f, \perp, \mathbf{w})$. By the “Prover moves” condition of Definition 3.6, if $\text{KState}_{\Pi_{123}}(\mathbb{x}, f, \mathbf{w}) = 0$ (empty transcript where the prover is about to move), then $\text{KState}_{\Pi_{123}}(\mathbb{x}, f, \perp, \mathbf{w}) = 0$ ($\perp$ is always the first message of the prover in Π_{123}). Now $\text{KState}_{\Pi_{123}}(\mathbb{x}, f, \mathbf{w}) = 0$ indeed because by the condition “Empty transcript” of Definition 3.6, $\text{KState}_{\Pi_{123}}(\mathbb{x}, f, \mathbf{w}) = 0$ if and only if $(\mathbb{x}, f, \mathbf{w}) \notin \mathcal{R}_{\text{PIMSAT}}(R_{\leq B}^{\leq d}, \mathbf{p}, \mathcal{I}, \mathcal{C})^\delta$, and since $(\mathbb{x}, \perp, \mathbf{w}) \notin \mathcal{R}_{\text{PIMSAT}}$, it follows that $(\mathbb{x}, f, \mathbf{w}) \notin \mathcal{R}_{\text{PIMSAT}}(R_{\leq B}^{\leq d}, \mathbf{p}, \mathcal{I}, \mathcal{C})^\delta$. Hence $\text{KState}(\mathbb{x}, \perp, f, \mathbf{w}) = 0$ and KState satisfies the condition “Prover moves” of Construction 7.3.

Regarding the third condition, “Full transcript”, let (f, tr') be a full transcript for Construction 7.3. Then for all $\mathbb{x}, \mathbf{w}$,

$$\text{KState}(\mathbb{x}, \perp, (f, \text{tr}'), \mathbf{w}) = \text{KState}_{\Pi_{123}}(\mathbb{x}, f, (\perp, \text{tr}'), \mathbf{w}) = 1 \Leftrightarrow V_{\Pi_{123}}(\mathbb{x}, f, (\perp, \text{tr}')) = 1,$$

where $V_{\Pi_{123}}$ is the verifier of Π_{123} .

Notice that the verifier of Construction 7.3 given $(\mathbb{x}, \perp, (f, \text{tr}'))$ makes exactly the same checks as $V_{\Pi_{123}}(\mathbb{x}, f, (\perp, \text{tr}'))$. Hence the verifier of Construction 7.3 accepts $(\mathbb{x}, \perp, (f, \text{tr}'))$ if and only if $\text{KState}(\mathbb{x}, \perp, (f, \text{tr}'), \mathbf{w}) = 1$, as needed. This shows that KState satisfies the three conditions of Definition 3.6.

Now assume Π_{123} has round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{PIMSAT}}(R_{\leq B}^{\leq d}, \mathbf{p}, \mathcal{I}, \mathcal{C})^\delta, \perp)$ with errors $(\kappa'_j)_{j \in [2+t_{\mathbb{K}}, 1+t_{\text{red}}+t_{\text{iopp}}]}$, with respect to the state function $\text{KState}_{\Pi_{123}}$. We claim that

then [Construction 7.3](#) has round-by-round knowledge soundness without relaxation, with errors $(\kappa'_j)_{j \in [2+t_{\mathbb{K}}, 1+t_{\text{red}}+t_{\text{iopp}}]}$, with respect to the state function KState .

We start by defining an extractor $\mathcal{E}$ for [Construction 7.3](#) simply as

$$\mathcal{E}(\mathbb{x}, \perp, (f, \text{tr}'), \mathbf{w}) = \mathcal{E}_{\Pi_{123}}(\mathbb{x}, f, (\perp, \text{tr}'), \mathbf{w})$$

for every (partial) transcript (f, tr') where the verifier is about to move. We need not define an extractor for, say $(\mathbb{x}, \perp, \mathbf{w})$, since the verifier is not about to move after an empty transcript $\perp$ (the prover is about to send f).

Now, let $\text{tr} = (f, \text{tr}')$ be a (partial) transcript of [Construction 7.3](#) where the verifier is about to send its j -th message ρ . Then we have, using the definitions of KState , $\mathcal{E}$ and our assumption on the round-by-round knowledge soundness of Π_{123} ,

$$\begin{aligned} & \Pr_{\rho} \left[\exists \mathbf{w} \mid \begin{array}{l} \wedge \text{KState}(\mathbb{x}, \perp, (f, \text{tr}'), \mathcal{E}(\mathbb{x}, \perp, (f, \text{tr}'), \rho, \mathbf{w})) = 0 \\ \wedge \text{KState}(\mathbb{x}, \perp, (f, \text{tr}'), \rho, \mathbf{w}) = 1 \end{array} \right] \\ &= \Pr_{\rho} \left[\exists \mathbf{w} \mid \begin{array}{l} \wedge \text{KState}_{\Pi_{123}}(\mathbb{x}, f, (\perp, \text{tr}'), \mathcal{E}_{\Pi_{123}}(\mathbb{x}, f, (\perp, \text{tr}'), \rho, \mathbf{w})) = 0 \\ \wedge \text{KState}_{\Pi_{123}}(\mathbb{x}, f, (\perp, \text{tr}'), \rho, \mathbf{w}) = 1 \end{array} \right] \\ &\leq \kappa_j(\mathbb{x}, f)' \leq \max_{f \text{ s.t. } (\mathbb{x}, f) \notin \mathcal{R}_{\text{PIMSAT}}(R_{\leq B}^{\leq d}, \mathbf{p}, \mathcal{I}, \mathcal{C})^{\delta}} = \kappa_j(\mathbb{x}). \end{aligned}$$

This concludes the proof of [Lemma A.2](#).

B Ring switching via Galois orbits

Construction B.1. [Galois ring-switch: an IOR from $\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C})$ to $\mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^\nu}, \text{id}}^{\text{mle}}$] [Albert: TODO: formulate as a reduction from \$\mathcal{R}_{\text{Lin}}\(\mathbb{F}_{2^\nu}, \mathcal{C}\)\$ to \$\mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^\nu}, \text{id}}^{\text{mle}}\$](#) .

Let $b_1, \dots, b_m \in \mathbb{F}_2^{2^n}$. Using an $\mathbb{F}_2$ -basis $\beta_1, \dots, \beta_m$ of $\mathbb{F}_{2^m}$ we pack m inner products

$$\langle b_i, \mathbf{v} \rangle = \sum_{j \in \{0,1\}^n} b_{i,j} \cdot \mathbf{v}_j, \quad i = 1, \dots, m,$$

coefficient-wise into a single inner product

$$\begin{aligned} b &:= b_1 \cdot \beta_1 + \dots b_m \cdot \beta_m \in \mathbb{F}_{2^m}^{2^n} \\ \langle b, \mathbf{v} \rangle &= \langle b_1, \mathbf{v} \rangle \cdot \beta_1 + \dots + \langle b_m, \mathbf{v} \rangle \cdot \beta_m \\ &= \sum_{j \in \{0,1\}^n} (b_{1,j} \cdot \beta_1 + \dots + b_{m,j} \cdot \beta_m) \cdot \mathbf{v}_j. \end{aligned}$$

The Frobenius automorphism $\phi(z) = z^2$ keeps $\mathbb{F}_2 \subset \mathbb{F}_{2^m}$ fixed, and has no other fixed points. We extract the value of $\langle b_i, \mathbf{v} \rangle$ from the values of $\langle b, \mathbf{v} \rangle$ along the Galois orbit

$$\mathbf{v}, \mathbf{v}^2, \mathbf{v}^{2^2}, \dots, \mathbf{v}^{2^{m-1}},$$

as follows, where by $\mathbf{v}^t$ we mean that each component of $\mathbf{v}$ is raised to the power t . The Moore matrix

$$M = \begin{pmatrix} \beta_1 & \beta_2 & \dots & \beta_m \\ \beta_1^2 & \beta_2^2 & \dots & \beta_m^2 \\ \beta_1^{2^2} & \beta_2^{2^2} & \dots & \beta_m^{2^2} \\ \vdots & \vdots & \ddots & \vdots \\ \beta_1^{2^{m-1}} & \beta_2^{2^{m-1}} & \dots & \beta_m^{2^{m-1}} \end{pmatrix}$$

consists of the Galois orbits of the basis vectors, and is invertible. (See below.) Using that the component polynomials are over $\mathbb{F}_2$, we see that

$$\phi^k \left(\langle b, \phi^{-k}(\mathbf{v}) \rangle \right) = \langle b_1, \mathbf{v} \rangle \cdot \phi^k \beta_1 + \dots + \langle b_m, \mathbf{v} \rangle \cdot \phi^k \beta_m,$$

for any k . To this end, observe that

$$M \cdot \begin{pmatrix} \langle b_1, \mathbf{v} \rangle \\ \langle b_2, \mathbf{v} \rangle \\ \langle b_3, \mathbf{v} \rangle \\ \vdots \\ \langle b_m, \mathbf{v} \rangle \end{pmatrix} = \begin{pmatrix} \langle b, \mathbf{v} \rangle \\ \phi \left(\langle b, \phi^{-1}(\mathbf{v}) \rangle \right) \\ \phi^2 \left(\langle b, \phi^{-2}(\mathbf{v}) \rangle \right) \\ \vdots \\ \phi^{m-1} \left(\langle b, \phi^{-(m-1)}(\mathbf{v}) \rangle \right) \end{pmatrix}.$$

Multiplying both sides by M^{-1} we obtain an expression of the values $\langle b_i, \mathbf{v} \rangle$ as a linear expression of the values $\phi^k \left(\langle b, \phi^{-k}(\mathbf{v}) \rangle \right)$. **Ulrich:** Invertibility of M follows from that of M^t : any linear functional $(c_1, \dots, c_m)$ in the kernel of M^t gives rise to the linearizable polynomial

$$p(X) = \sum_{i=0}^{m-1} c_i \cdot X^{2^i}$$

which has $\beta_1, \dots, \beta_m$, and hence the whole space F as roots. By dimension, $p(X)$ must be trivial, meaning that all its coefficients are zero.

C Bilinear Embeddings

Given a field $\mathbb{F}$ and a degree- d extension $\mathbb{E}$. Our goal is to commit to a vector $\mathbf{w} \in \mathbb{F}^{nd}$ so that we can later use it to prove an inner product claim of the form $\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} = h$ for some $\mathbf{a} \in \mathbb{F}^{nd}$ and $h \in \mathbb{F}$. However, we only have an efficient inner product commitment scheme to commit to vectors $\tilde{\mathbf{w}} \in \mathbb{E}^n$ and open as $\langle \tilde{\mathbf{w}}, \tilde{\mathbf{a}} \rangle_{\mathbb{E}} = \tilde{h}$ for some $\tilde{\mathbf{a}} \in \mathbb{E}^n$ and $\tilde{h} \in \mathbb{E}$.

To do this, we seek a triple of maps (W, A, H) that satisfy:

$$\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} = H \left(\langle W(\mathbf{w}), A(\mathbf{a}) \rangle_{\mathbb{E}} \right).$$

C.1 Background

C.1.1 Perfect pairings

In the following let $\mathbb{E}/\mathbb{F}$ be a finite field extension of degree d . If we disregard multiplication in $\mathbb{E}$, then $\mathbb{E}$ is a d -dimensional vector space over $\mathbb{F}$, hence isomorphic to $\mathbb{F}^d$. Let $\mathbf{e}_1, \dots, \mathbf{e}_d$ denote the

standard basis of $\mathbb{F}^d$. Recall that a function $\mu : \mathbb{E} \times \mathbb{E} \rightarrow \mathbb{F}$ is a *bilinear form* if it is linear in each argument separately. It is *symmetric* if $\mu(x, y) = \mu(y, x)$ for all $x, y \in \mathbb{E}$. For each $x \in \mathbb{E}$, the partial application in the first argument defines a linear form

$$\mu_x : \mathbb{E} \rightarrow \mathbb{F} \quad y \mapsto \mu(x, y).$$

Thus $\mu_x \in \mathbb{E}^*$ where $\mathbb{E}^* = \text{Hom}_{\mathbb{F}}(\mathbb{E}, \mathbb{F})$ denotes the dual vector space. The bilinear form therefore induces an $\mathbb{F}$ -linear map

$$\Phi_\mu : \mathbb{E} \rightarrow \mathbb{E}^* \quad x \mapsto \mu_x.$$

We call μ *nondegenerate*, or equivalently a *perfect pairing*, if Φ_μ is an isomorphism. Or equivalently for finite spaces, [Albert: add a comment that injectivity implies surjectivity when the vector spaces have finite dimension?](#)

$$\mu(x, y) = 0 \text{ for all } y \in \mathbb{E} \implies x = 0.$$

Since μ is a perfect pairing, every basis $(\beta_1, \dots, \beta_d)$ of $\mathbb{E}$ has a unique dual basis $(\beta_1^\vee, \dots, \beta_d^\vee)$ satisfying $\mu(\beta_i, \beta_j^\vee) = \delta_{ij}$.

John: It would be good to explain how you would typically derive $\beta_j^\vee$

C.1.2 Succinct vectors

Given a field $\mathbb{F}$, a vector $\mathbf{a} \in \mathbb{F}^n$ for $n = 2^k$ is called *succinct* if it admits a tensor decomposition

$$\begin{bmatrix} m_{00} \\ m_{01} \end{bmatrix} \otimes \begin{bmatrix} m_{10} \\ m_{11} \end{bmatrix} \otimes \dots \otimes \begin{bmatrix} m_{(k-1)0} \\ m_{(k-1)1} \end{bmatrix} \quad (36)$$

where typically $m_{ij} \in \mathbb{F}$. The multilinear extension of $\mathbf{a}$, viewed as a function on $\{0, 1\}^k$, then becomes

$$\tilde{a}(\mathbf{r}) = \prod_{i=0}^{k-1} ((1 - r_i)m_{i0} + r_i m_{i1}). \quad (37)$$

which can be evaluated in $O(k)$ field operations.

More generally, the coefficients m_{ij} need not themselves be field elements. It suffices that they belong to an associative $\mathbb{F}$ -algebra in which every complete product $m_{0b_0}m_{1b_1} \dots m_{(k-1)b_{k-1}}$ evaluates to an element of $\mathbb{F}$. Consequently, every vector entry and every value of the multilinear extension remains an element of $\mathbb{F}$. As the m_{ij} no longer need to be commutative we must therefore also fix a permutation of the variables $\mathbf{r}$.

Since every finite-dimensional associative algebra admits a matrix representation, we may, without loss of generality, assume that the objects m_{ij} are matrices. In fact, it suffices to take $m_{ij} \in \mathbb{F}^{w_i \times w_{i+1}}$ with boundary dimensions $w_0 = w_k = 1$. We call $\max w_i$ the *width* of the representation, which is known as a *matrix branching program* and is closely related to matrix product states and weighted finite automata.

The canonical example of a succinct vector is the multilinear evaluation vector $a_i = \text{eq}(\mathbf{r}, \text{bits}(i))$ obtained by taking $m_{i0} = 1 - r_i$ and $m_{i1} = r_i$. Another classical example is the geometric progression $(1, x, x^2, \dots)$ obtained by taking $m_{i0} = 1$ and $m_{i1} = x^{2^i}$. An illustrative non-scalar example is the counting vector $(0, 1, 2, \dots)$ represented by

$$m_{0b} = \begin{bmatrix} 2 & b \end{bmatrix} \quad m_{ib} = \begin{bmatrix} 2 & b \\ 0 & 1 \end{bmatrix} \quad m_{(k-1)b} = \begin{bmatrix} b \\ 1 \end{bmatrix}.$$

C.2 Structure

We are now equipped to prove a theorem on the structure of embeddings. Under the assumption that the output map is $\mathbb{F}$ -linear, we show that for $n = 1$ the space of possible embeddings is very structured.

Theorem C.1 (Structure of bilinear embeddings). *Let $W, A : \mathbb{F}^d \rightarrow \mathbb{E}, H : \mathbb{E} \rightarrow \mathbb{F}$ be maps such that for all $\mathbf{w}, \mathbf{a} \in \mathbb{F}^d$,*

$$\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} = H(W(\mathbf{w})A(\mathbf{a})).$$

Suppose that H is $\mathbb{F}$ -linear. Then:

1. *W and A are $\mathbb{F}$ -linear isomorphisms,*
2. *$\mu_H(x, y) = H(xy)$ is a perfect pairing,*
3. *$(W(\mathbf{e}_1), \dots, W(\mathbf{e}_d))$ and $(A(\mathbf{e}_1), \dots, A(\mathbf{e}_d))$ are dual bases with respect to μ_H .*

Conversely, if W and A , and H are $\mathbb{F}$ -linear, μ_H is a perfect pairing and the images of the standard basis under W and A are dual bases with respect to μ_H , then the identity

$$\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} = H(W(\mathbf{w})A(\mathbf{a})).$$

holds for all $\mathbf{w}, \mathbf{a} \in \mathbb{F}^d$.

Proof of the forward direction. First, observe that $\mu_H(W(\mathbf{e}_i), A(\mathbf{e}_j)) = \langle \mathbf{e}_i, \mathbf{e}_j \rangle_{\mathbb{F}} = \delta_{ij}$. Taking $i = j$ shows H is nonzero. Since H is $\mathbb{F}$ -linear, μ_H is a symmetric bilinear form, we show that it is nondegenerate. Let $x \in \mathbb{E}$ be nonzero. Since multiplication by x is a bijection of $\mathbb{E}$ and H is nonzero, there exists $y \in \mathbb{E}$ such that $\mu_H(x, y) = H(xy) \neq 0$. Thus μ_H is nondegenerate, and hence a perfect pairing.

Next, we show that W and A are bijective. Suppose $W(\mathbf{w}_1) = W(\mathbf{w}_2)$. Then for every $\mathbf{a} \in \mathbb{F}^d$,

$$\langle \mathbf{w}_1 - \mathbf{w}_2, \mathbf{a} \rangle_{\mathbb{F}} = \mu_H(W(\mathbf{w}_1) - W(\mathbf{w}_2), A(\mathbf{a})) = 0.$$

By nondegeneracy of the standard inner product on $\mathbb{F}^d$, this implies $\mathbf{w}_1 = \mathbf{w}_2$. Hence W is injective. Since $\mathbb{F}^d$ and $\mathbb{E}$ have the same finite cardinality, W is bijective. The same argument shows that A is bijective.

We now prove that W is $\mathbb{F}$ -linear. For $\mathbf{w}_1, \mathbf{w}_2, \mathbf{a} \in \mathbb{F}^d$, linearity of H gives

$$\begin{aligned} & \mu_H(W(\mathbf{w}_1 + \mathbf{w}_2) - W(\mathbf{w}_1) - W(\mathbf{w}_2), A(\mathbf{a})) \\ &= \langle \mathbf{w}_1 + \mathbf{w}_2, \mathbf{a} \rangle_{\mathbb{F}} - \langle \mathbf{w}_1, \mathbf{a} \rangle_{\mathbb{F}} - \langle \mathbf{w}_2, \mathbf{a} \rangle_{\mathbb{F}} \\ &= 0. \end{aligned}$$

Since A is surjective, the second argument ranges over all of $\mathbb{E}$. Nondegeneracy of μ_H therefore implies $W(\mathbf{w}_1 + \mathbf{w}_2) = W(\mathbf{w}_1) + W(\mathbf{w}_2)$. Similarly, for $\lambda \in \mathbb{F}$,

$$\begin{aligned} & \mu_H(W(\lambda \mathbf{w}) - \lambda W(\mathbf{w}), A(\mathbf{a})) \\ &= \langle \lambda \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} - \lambda \langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} \\ &= 0. \end{aligned}$$

Again using surjectivity of A and nondegeneracy of μ_H , we obtain $W(\lambda \mathbf{w}) = \lambda W(\mathbf{w})$. Hence W is $\mathbb{F}$ -linear. By symmetry, A is also $\mathbb{F}$ -linear. Since they are also bijective, W and A are linear isomorphisms. From this it follows that $(W(\mathbf{e}_1), \dots, W(\mathbf{e}_d))$ and $(A(\mathbf{e}_1), \dots, A(\mathbf{e}_d))$ are bases of $\mathbb{E}$. Finally, from $\mu_H(W(\mathbf{e}_i), A(\mathbf{e}_j)) = \delta_{ij}$ it follows they are dual with respect to μ_H . $\square$

Proof of the converse direction. Can be directly shown using a basis expansion. Write $\mathbf{w} = \sum_{i=1}^d w_i \mathbf{e}_i$ and $\mathbf{a} = \sum_{j=1}^d a_j \mathbf{e}_j$. Since W and A are $\mathbb{F}$ -linear, $W(\mathbf{w}) = \sum_{i=1}^d w_i W(\mathbf{e}_i)$ and $A(\mathbf{a}) = \sum_{j=1}^d a_j A(\mathbf{e}_j)$. Therefore,

$$\begin{aligned} H(W(\mathbf{w})A(\mathbf{a})) &= \mu_H(W(\mathbf{w}), A(\mathbf{a})) \\ &= \sum_{i,j} w_i a_j \mu_H(W(\mathbf{e}_i), A(\mathbf{e}_j)) \\ &= \sum_{i,j} w_i a_j \delta_{ij} \\ &= \sum_i w_i a_i \\ &= \langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}}. \end{aligned}$$

□

The $n = 1$ case extends naturally to $n > 1$ by doing blockwise packing, i.e. we embed using $W' = I_n \otimes W$ and $A' = I_n \otimes A$ and keep the same H . Let $\mathbf{w} = (\mathbf{w}_1, \dots, \mathbf{w}_n)$ and $\mathbf{a} = (\mathbf{a}_1, \dots, \mathbf{a}_n)$ then

$$\begin{aligned} H(\langle W'(\mathbf{w}), A'(\mathbf{a}) \rangle_{\mathbb{E}}) &= H\left(\sum_{i=1}^n W(\mathbf{w}_i)A(\mathbf{a}_i)\right) \\ &= \sum_{i=1}^n H(W(\mathbf{w}_i)A(\mathbf{a}_i)) \\ &= \sum_{i=1}^n \langle \mathbf{w}_i, \mathbf{a}_i \rangle_{\mathbb{F}} \\ &= \langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}}. \end{aligned}$$

Going forward, we use W and A also for their blockwise extensions when the meaning is clear. We do not consider embeddings that mix coordinates across different blocks of size d , as such embeddings are unlikely to be useful in practice.

We have shown that every $n = 1$ embedding with a linear output map is determined by a choice of a nonzero linear map H and linear isomorphism W . The map W then determines the first basis $\beta_i = W(\mathbf{e}_i)$ and the map A is then forced to map to the μ_H -dual basis.

The linearity assumption on the output map is necessary. Indeed, composing a linear embedding with a multiplicative bijection $g : \mathbb{E} \rightarrow \mathbb{E}$ generally produces an embedding that is nonlinear. Whether every nonlinear embedding arises from such a multiplicative reparameterization is a separate question, which we do not need here.

An important property of bilinear embeddings is that they preserve succinctness if the block size aligns with the tensor decomposition:

Theorem C.2 (Preservation of succinctness). *Given $d = 2^k$, $n = 2^l$ and a $\mathbb{F}$ -linear blockwise map $A = I_n \otimes A_0$ for some isomorphism $A_0 : \mathbb{F}^d \rightarrow \mathbb{E}$, then if $\mathbf{a}$ is succinct then so is $A(\mathbf{a})$. In particular if*

$$\mathbf{a} = \bigotimes_{i=0}^{k+l-1} \begin{bmatrix} m_{i0} \\ m_{i1} \end{bmatrix}$$

where the m_{ij} can be scalars or $w_i \times w_{i+1}$ matrices, then

$$A(\mathbf{a}) = \left(\bigotimes_{i=0}^{l-1} \begin{bmatrix} m_{i0} \\ m_{i1} \end{bmatrix} \right) \cdot (\mathbf{I}_{w_l} \otimes A_0) \left(\bigotimes_{i=l}^{l+k-1} \begin{bmatrix} m_{i0} \\ m_{i1} \end{bmatrix} \right).$$

where the second factor is column vector in $\mathbb{E}^{w_l}$ which can be absorbed into the matrices $m_{(l-1)b}$, bringing the representation back into canonical form.

C.3 Constructions

The set of nonzero linear output maps H is $\mathbb{E}^* \setminus \{0\}$ which is isomorphic to $\mathbb{E}^\times$. The set of linear isomorphism W is isomorphic to $\mathrm{GL}_d(\mathbb{F})$, hence the total number of bilinear embeddings is $|\mathbb{E}^\times| \cdot |\mathrm{GL}_d(\mathbb{F})|$. We'll now look at specific choices and their properties.

C.3.1 Trace embedding

Let $q = |\mathbb{F}|$, the q -power Frobenius map is $\varphi : \mathbb{E} \rightarrow \mathbb{E}$ given by $x \mapsto x^q$. It is an $\mathbb{F}$ -linear field automorphism of order d , and its fixed field is precisely $\mathbb{F}$:

$$\mathbb{E}^\varphi = \{x \in \mathbb{E} : \varphi(x) = x\} = \mathbb{F}.$$

The field trace is the $\mathbb{F}$ -linear map $\mathrm{Tr}_{\mathbb{E}/\mathbb{F}} : \mathbb{E} \rightarrow \mathbb{F}$ defined by

$$\mathrm{Tr}_{\mathbb{E}/\mathbb{F}}(x) = \sum_{i=0}^{d-1} \varphi^i(x) = \sum_{i=0}^{d-1} x^{q^i}.$$

To see that this sum is indeed in $\mathbb{F} \subseteq \mathbb{E}$ observe the sum is fixed by φ , since

$$\varphi \left(\sum_{i=0}^{d-1} \varphi^i(x) \right) = \sum_{i=1}^d \varphi^i(x) = \sum_{i=0}^{d-1} \varphi^i(x),$$

where $\varphi^d(x) = \varphi^0(x)$. Hence the trace lies in the fixed field $\mathbb{F}$.

This makes $\mathrm{Tr}_{\mathbb{E}/\mathbb{F}}$ a natural choice of output map H . More generally, every $\mathbb{F}$ -linear map $H : \mathbb{E} \rightarrow \mathbb{F}$ is uniquely of the form $H(x) = \mathrm{Tr}_{\mathbb{E}/\mathbb{F}}(\alpha x)$ for some $\alpha \in \mathbb{E}$. Consequently, the nonzero output maps are parameterized by $\mathbb{E}^\times$.

The Galois orbit embedding chooses the basis $\beta_i = \varphi^i(\gamma)$ which has dual $\beta_i^\vee = \varphi^i(\gamma^\vee)$.

Ulrich: This sounds to me very much related to Galois Fribinius on an inner product with a general covector. You would end up with the multi-claim of the inner products of the packed poly with the Galois orbit of the covector. This is how I view it: Given $\mathbf{p}_1, \dots, \mathbf{p}_d \in \mathbb{F}^n$, and $\mathbf{q}_1, \dots, \mathbf{q}_d \in \mathbb{F}^n$, we are interested in

$$\sum_{k=1}^d \langle \mathbf{p}_k, \mathbf{q}_k \rangle,$$

right?

The packing map sends the coeff vectors $\mathbf{p}_1, \dots, \mathbf{p}_d$ to $\mathbf{p} = \beta_1 \cdot \mathbf{p}_1 + \dots + \beta_d \cdot \mathbf{p}_d$, where $\beta_1, \dots, \beta_d$ is an $\mathbb{F}$ -basis of $\mathbb{E}$. Choose the element a_k in $\mathbb{E}$ which extracts the k -th coordinate

$$\mathbf{p}_k = \Phi_k(\mathbf{p}) = \mathrm{Tr}_{\mathbb{E}/\mathbb{F}}(a_k \cdot \mathbf{p}).$$

Then

$$\begin{aligned}\langle \mathbf{p}_k, \mathbf{q}_k \rangle &= \left\langle \sum_{i=0}^d \varphi^i(a_k \cdot \mathbf{p}), \mathbf{q}_k \right\rangle \\ &= \sum_{i=0}^d \varphi^i(a_k) \cdot \varphi^i(\langle \mathbf{p}, \mathbf{q}_k \rangle),\end{aligned}$$

since $\mathbf{q}_k$ is over $\mathbb{F}$. This reduces our inner product claim to that of the packed poly $\mathbf{p}$ with the unpacked polys $\mathbf{q}_1, \dots, \mathbf{q}_k$.

But does that match our use case? We are rather interested in $\langle \mathbf{p}_k, \mathbf{v} \rangle$ for some $\mathbf{v} \in \mathbb{E}^n$. In this case,

$$\begin{aligned}\langle \mathbf{p}_k, \mathbf{v} \rangle &= \left\langle \sum_{i=0}^d \varphi^i(a_k \cdot \mathbf{p}), \mathbf{q}_k \right\rangle \\ &= \sum_{i=0}^d \varphi^i(a_k) \cdot \varphi^i(\langle \mathbf{p}, \varphi^{-i}(\mathbf{v}) \rangle),\end{aligned}$$

and we end up with inner products of the packed poly with the Galois orbit of $\mathbf{v}$.

C.3.2 Coefficient projection

In the coefficient projection we assume a concrete representation of $\mathbb{E}$ as $\mathbb{F}[X]/(f(X))$ for some monic degree d polynomial f irreducible over $\mathbb{F}$. We assume f is chosen for implementation efficiency: it is sparse with mostly low order terms. Further we represent members of $\mathbb{E}$ in the monomial basis $(1, X, X^2, \dots, X^{d-1})$. We can then represent W and A by invertible matrices in $\mathbb{F}^{d \times d}$ and H by a vector in $\mathbb{F}^d$. For implementation efficiency we seek an embedding with sparse W and A . It turns out taking the most efficient choices for H and W also gives efficient A :

Theorem C.3. *Write $f(X) = f_d X^d + f_{d-1} X^{d-1} + \dots + f_1 X + f_0$ so that $f_d = 1$ and $f_0 \neq 0$. Take $W = \mathbf{I}_d$ and $H = (1, 0, \dots, 0)$. Then the dual-basis embedding A is given by*

$$A_{ij} = \begin{cases} 1, & i = j = 0, \\ -f_{i+j}/f_0, & i, j \geq 1 \text{ and } i + j \leq d, \\ 0, & \text{otherwise.} \end{cases}$$

In particular, A is a bordered upper anti-triangular Hankel matrix. For $d = 5$ it looks like

$$A = \frac{1}{f_0} \begin{pmatrix} f_0 & 0 & 0 & 0 & 0 \\ 0 & -f_2 & -f_3 & -f_4 & -1 \\ 0 & -f_3 & -f_4 & -1 & 0 \\ 0 & -f_4 & -1 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 \end{pmatrix}.$$

Notably f_1 does not appear and for $k > 1$ the f_k coefficient appears $k - 1$ times. This perfectly matches the usual implementation preference for sparse reduction polynomials with mostly low-order terms.

When doing inner product openings, the $1/f_0$ factor can be moved out of the inner product.

For the 127-bit field $\mathbb{F}_2[X]/(X^{127} + X + 1)$ the A map is simply a reversal of the $i > 0$ coordinates. For the GHASH field $\mathbb{F}_2[X]/(X^{128} + X^7 + X^2 + X + 1)$ the map additionally needs seven XOR corrections.

C.4 Extension openings

Assume now that we wish to open a base-field witness $\mathbf{w} \in \mathbb{F}^{nd}$ against extension-field weights $\mathbf{a} \in \mathbb{E}^{nd}$, proving the claim $\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{E}} = h$ for some $h \in \mathbb{E}$. We present a one-round batching protocol and a $\log d$ -round folding protocol.

C.4.1 Batching protocol

Given a bilinear embedding (W, A, H) and a basis $(\beta_1, \dots, \beta_d)$ for $\mathbb{E}/\mathbb{F}$, we reduce the extension-field claim by decomposing into basis coordinates and batching the resulting claims with a random linear combination.

Construction C.4 (Batching protocol for extension opening). Given the claim $\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{E}} = h$, define $\mathbf{a}_i \in \mathbb{F}^{nd}$ such that

$$\mathbf{a} = \sum_{i=1}^d \mathbf{a}_i \beta_i.$$

The parties proceed as follows.

1. The prover sends $h_i = \langle W(\mathbf{w}), A(\mathbf{a}_i) \rangle_{\mathbb{E}}$ for $i = 1, \dots, d$.
2. The verifier checks $h = \sum_{i=1}^d H(h_i) \beta_i$.
3. The verifier samples an ϵ -zero-evader $\boldsymbol{\rho} \leftarrow_{\$} \mathbb{E}^d$.
4. The parties reduce the claim to $\langle W(\mathbf{w}), \mathbf{a}' \rangle_{\mathbb{E}} = h'$ where

$$h' = \sum_{i=1}^d \rho_i h_i, \quad \mathbf{a}' = \sum_{i=1}^d \rho_i A(\mathbf{a}_i).$$

This interactive oracle reduction is always complete, has round-by-round soundness error ϵ , and proof size $d \log |\mathbb{E}| = d^2 \log |\mathbb{F}|$.

Theorem C.5 (Succinctness of coordinate decomposition). *Given a succinct vector $\mathbf{a} \in \mathbb{E}^{nd}$ for $d = 2^k$ and a basis (β_i) of $\mathbb{E}/\mathbb{F}$ then the β -coordinate decomposed vectors are also succinct. In particular if*

$$\mathbf{a} = \bigotimes_{i=0}^{k+l-1} \begin{bmatrix} m_{i0} \\ m_{i1} \end{bmatrix}$$

where the m_{ij} can be scalars or $w_i \times w_{i+1}$ matrices, then

$$\mathbf{a}_i = \mathbf{u}_i \cdot \left(\bigotimes_{j=0}^{k+l-1} \begin{bmatrix} \Phi(m_{j0}) \\ \Phi(m_{j1}) \end{bmatrix} \right) \cdot \mathbf{v}$$

where $\Phi : \mathbb{E} \rightarrow \mathbb{F}^{d \times d}$ is the map that represents an $\mathbb{E}$ -element m by the matrix representing multiplication by m in basis (β_i) , $\mathbf{u}_i$ is the one-hot vector with $u_{ij} = \delta_{ij}$, and $\mathbf{v}$ is the base (β_i) representation of $1 \in \mathbb{E}$. Φ is applied elementwise to $w_i \times w_{i+1}$ matrices over $\mathbb{E}$ to produce $dw_i \times dw_{i+1}$ block matrices over $\mathbb{F}$. Absorbing the outer vectors in the matrices brings the representation back into canonical form, with the width increased by at most a factor d .

For practical implementations it makes sense to evaluate the branching program over $\mathbb{E}$ directly and then extract the coordinates in the basis (β_i) . This is equivalent to omitting the output selector $\mathbf{t}_i$ to keep the output dimension $w_0 = d$.

If $A = I_n \otimes A_0$ is blockwise we can also apply theorem C.2 and compose it with the combination weights $\boldsymbol{\rho} \in \mathbb{E}^d$ to get a succinct representation for $\mathbf{a}'$:

$$\mathbf{a}' = \boldsymbol{\rho} \cdot \left(\bigotimes_{i=0}^{l-1} \begin{bmatrix} \Phi(m_{i0}) \\ \Phi(m_{i1}) \end{bmatrix} \right) \cdot (I_{dw_l} \otimes A_0) \left(\bigotimes_{i=l}^{l+k-1} \begin{bmatrix} \Phi(m_{i0}) \\ \Phi(m_{i1}) \end{bmatrix} \right) \cdot \mathbf{v}. \quad (38)$$

The map Φ is an algebra embedding, so replacing each scalar $m_{ij} \in \mathbb{E}$ by $\Phi(m_{ij})$ leaves the branching-program computation unchanged, except that the values are now represented in the basis (β_i) representation of $\mathbb{E}$. The only operations outside the embedded $\mathbb{E}$ computation are the blockwise embedding A_0 and the random linear combination $\boldsymbol{\rho}$. Thus, apart from the applications of A_0 and $\boldsymbol{\rho}$, evaluating $\tilde{\mathbf{a}}'$ is the same computation as evaluating $\tilde{\mathbf{a}}$, performed in the regular representation of $\mathbb{E}$. **Remco: Something is off with the complexity. Should really check with a Sage implementation. Probably needs a factor d .**

The batch protocol is quadratic in d in terms of $\mathbb{F}$ -element operations and proof size. This is acceptable when d is small, but binary fields where $d = 128$ the communication would be 2,048 bytes. Avoiding this quadratic dependence on d appears to require further rounds of interaction and more sophisticated decomposition.

C.4.2 Folding protocol

In the folding protocol we interactively reduce the claim $\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{E}} = h$ through a sequence of embeddings W_i where at round i the claim is

$$\langle W_i(\mathbf{w}), A_i(\mathbf{a}) \rangle_{\mathbb{E}} = h_i \quad W_i : \mathbb{F}^{nd} \rightarrow \mathbb{E}^{nd_i} \quad A_i : \mathbb{E}^{nd} \rightarrow \mathbb{E}^{nd_i}$$

Here $d = d_0 \geq \dots \geq d_k = 1$ is a series of folding dimensions. We start with W_0 and A_0 the identity and $h_0 = h$ so that the initial claim is indeed $\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{E}} = h$. We also stipulate that the embeddings are blockwise such that $W_i = I_n \otimes W'_i$ and $A_i = I_n \otimes A'_i$. For simplicity we will only consider the $n = 1$ case from here on as $n > 1$ follows by blockwise repetition.

The final map is $W_k : \mathbb{F}^d \rightarrow \mathbb{E}$ which is an F -linear isomorphism. Let (β_i) be the implied basis such that

$$W_k(\mathbf{x}) = \sum_{i=1}^d x_i \beta_i.$$

Working backwards from the final embedding we create a sequence

$$R_i \circ W_{i+1} = W_i$$

where each map R_i has only $\left\lceil \frac{d_i}{d_{i+1}} \right\rceil$ Frobenius components.

Every $\mathbb{F}$ -linear map $T : \mathbb{E}^m \rightarrow \mathbb{E}^n$ can be viewed as being built from $\mathbb{E}$ -linear maps and $\mathbb{F}$ -linear scalar operations on $\mathbb{E}$. To recursively fold such maps, the scalar operations must be compatible with arbitrary $\mathbb{E}$ -linear coordinate transformations. Concretely, we require an intertwining

$$\beta(R\mathbf{x}) = \alpha(R)\beta(\mathbf{x}),$$

where α acts entrywise on R . This property singles out semilinear scalar operators. Over finite fields, these are precisely the Frobenius maps σ^i up to scalar multiples, and the Frobenius powers therefore provide the canonical scalar decomposition.

Theorem C.6 (Frobenius decomposition). *Let $\mathbb{E}/\mathbb{F}$ be a finite extension of degree d , and let $\sigma : \mathbb{E} \rightarrow \mathbb{E}$ be the Frobenius map $x \mapsto x^{|\mathbb{F}|}$. Then every $\mathbb{F}$ -linear map $T : \mathbb{E}^m \rightarrow \mathbb{E}^n$ admits a unique decomposition*

$$T(\mathbf{x}) = \sum_{i=0}^{d-1} M_i \cdot (\mathbf{I}_m \otimes \sigma^i)(\mathbf{x}), \quad M_i \in \mathbb{E}^{n \times m}.$$

We define $\text{supp}_\sigma(T) := \{i : M_i \neq 0\}$.

Single folding round. Assume the current claim is

$$\langle W_i(\mathbf{w}), A_i(\mathbf{a}) \rangle_{\mathbb{E}} = h_i.$$

Let

$$R_i : \mathbb{E}^{nd_{i+1}} \rightarrow \mathbb{E}^{nd_i}$$

be a fixed $\mathbb{F}$ -linear map satisfying

$$R_i \circ W_{i+1} = W_i.$$

By the Frobenius decomposition there exist unique matrices

$$M_{i,0}, \dots, M_{i,d-1} \in \mathbb{E}^{nd_i \times nd_{i+1}}$$

such that

$$R_i = \sum_{j=0}^{d-1} M_{i,j} \circ (\mathbf{I}_{nd_{i+1}} \otimes \sigma^j).$$

For every j , define the $\mathbb{F}$ -linear map

$$C_{i,j} = (\mathbf{I}_{nd_{i+1}} \otimes \sigma^{-j}) \circ M_{i,j}^\top.$$

Then, for every $\mathbf{x} \in \mathbb{E}^{nd_{i+1}}$ and $\mathbf{b} \in \mathbb{E}^{nd_i}$,

$$\langle R_i(\mathbf{x}), \mathbf{b} \rangle_{\mathbb{E}} = \sum_{j=0}^{d-1} \sigma^j(\langle \mathbf{x}, C_{i,j}(\mathbf{b}) \rangle_{\mathbb{E}}).$$

Applying this identity to

$$\mathbf{x} = W_{i+1}(\mathbf{w}), \quad \mathbf{b} = A_i(\mathbf{a}),$$

gives

$$\langle W_i(\mathbf{w}), A_i(\mathbf{a}) \rangle_{\mathbb{E}} = \sum_{j=0}^{d-1} \sigma^j(\langle W_{i+1}(\mathbf{w}), C_{i,j}(A_i(\mathbf{a})) \rangle_{\mathbb{E}}).$$

The prover sends the values

$$g_{i,j} = \langle W_{i+1}(\mathbf{w}), C_{i,j}(A_i(\mathbf{a})) \rangle_{\mathbb{E}}, \quad j = 0, \dots, d-1.$$

The verifier checks

$$h_i = \sum_{j=0}^{d-1} \sigma^j(g_{i,j}),$$

samples a random challenge $\rho_i \in \mathbb{E}$, and defines

$$A_{i+1} = \left(\sum_{j=0}^{d-1} \rho_i^j C_{i,j} \right) \circ A_i.$$

Finally, letting

$$h_{i+1} = \sum_{j=0}^{d-1} \rho_i^j g_{i,j},$$

the next claim is

$$\langle W_{i+1}(\mathbf{w}), A_{i+1}(\mathbf{a}) \rangle_{\mathbb{E}} = h_{i+1}.$$

We therefore seek an interactive reduction from a mixed claim to a packed claim

$$\begin{array}{ccc} \langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{E}} = h & \mathbf{w} \in \mathbb{F}^{nd} & \mathbf{a} \in \mathbb{E}^{nd} \\ \Downarrow & & \\ \langle W(\mathbf{w}), \mathbf{a}' \rangle_{\mathbb{E}} = h' & W(\mathbf{w}) \in \mathbb{E}^n & \mathbf{a}' \in \mathbb{E}^n \end{array}$$

where $W : \mathbb{F}^{nd} \rightarrow \mathbb{E}^n$ is a fixed $\mathbb{F}$ -linear embedding and $\mathbf{a}' \in \mathbb{E}^n$, $h' \in \mathbb{E}$ are determined by $\mathbf{a}$, h and the transcript. Ideally $\mathbf{a} \mapsto \mathbf{a}'$ should also preserve useful properties like efficient computation of its multilinear extension.

Above we decomposed $\mathbf{a}$ with respect to a basis of $\mathbb{E}/\mathbb{F}$. Here we instead decompose the induced $\mathbb{F}$ -linear functional on packed witnesses. Define

$$T_{\mathbf{a}} : \mathbb{E}^n \rightarrow \mathbb{E} \quad \mathbf{x} \mapsto \langle W^{-1}(\mathbf{x}), \mathbf{a} \rangle_{\mathbb{E}}.$$

This is an $\mathbb{F}$ -linear functional on $\mathbb{E}^n$, but is not generally $\mathbb{E}$ -linear. We'll first prove a representation lemma for $\mathbb{F}$ -linear $\mathbb{E} \rightarrow \mathbb{E}$ maps and then a decomposition theorem.

Theorem C.7. *Let $\mathbb{F} \subseteq \mathbb{E}$ be finite fields with $q = |\mathbb{F}|$ and $[\mathbb{E} : \mathbb{F}] = d$, then every $\mathbb{F}$ -linear map $T : \mathbb{E} \rightarrow \mathbb{E}$ is uniquely represented by a q -linearized polynomial*

$$P(X) = \sum_{j=0}^{d-1} c_j X^{q^j} \quad c_j \in \mathbb{E}.$$

Proof. By interpolation every map $T : \mathbb{E} \rightarrow \mathbb{E}$ can be represented uniquely by a polynomial $P \in \mathbb{E}[X]$ of degree less than $|\mathbb{E}| = q^d$

$$P(X) = \sum_{j=0}^{q^d-1} a_j X^j \quad a_j \in \mathbb{E}.$$

From $\mathbb{F}$ -linearity follows $P(0) = 0$ and hence $a_0 = 0$, further we have $P(X + Y) = P(X) + P(Y)$ and hence

$$\begin{aligned} 0 &= P(X + Y) - P(X) - P(Y) \\ &= \sum_{j=1}^{q^d-1} a_j (X + Y)^j - \sum_{j=1}^{q^d-1} a_j (X^j + Y^j) \\ &= \sum_{j=1}^{q^d-1} a_j \sum_{k=0}^j \binom{j}{k} X^k Y^{j-k} - \sum_{j=1}^{q^d-1} a_j (X^j + Y^j) \\ &= \sum_{j=1}^{q^d-1} a_j \sum_{k=1}^{j-1} \binom{j}{k} X^k Y^{j-k} \end{aligned}$$

Write $q = p^r$ with p the characteristic of $\mathbb{F}$. For every j with $a_j \neq 0$ we must have $\binom{j}{k} = 0 \pmod{p}$ for all $0 < k < j$, by Lucas's theorem this holds iff $j = p^s$ for some s . Hence we can reindex P keeping only the non-zero coefficients

$$P(X) = \sum_{j=0}^{rd-1} b_j X^{p^j} \quad b_j \in \mathbb{E}.$$

From $\mathbb{F}$ -linearity also follows $P(\lambda x) = \lambda P(x)$ for all $x \in \mathbb{E}$ and $\lambda \in \mathbb{F}$. For every $b_j \neq 0$ this requires $\lambda^{p^j} = \lambda$ for every $\lambda \in \mathbb{F}$. This holds exactly when $j = rs$ for some s . Reindexing P again we get our claimed result

$$P(X) = \sum_{j=0}^{d-1} c_j X^{q^j} \quad c_j \in \mathbb{E}.$$

□

Theorem C.8 (Frobenius decomposition). *Let $\mathbb{F} \subseteq \mathbb{K} \subseteq \mathbb{E}$ be finite fields with $[\mathbb{K} : \mathbb{F}] = m$, and let $\sigma(x) = x^{|\mathbb{F}|}$ denote the $\mathbb{F}$ -Frobenius automorphism. Then every $\mathbb{F}$ -linear map $T : \mathbb{E} \rightarrow \mathbb{E}$ admits a unique decomposition*

$$T(x) = \sum_{i=0}^{m-1} \sigma^i \circ T_i$$

where each T_i is $\mathbb{K}$ -linear.

Proof. Write $q = |\mathbb{F}|$ and $d = [\mathbb{E} : \mathbb{F}]$, by the preceding theorem T has a unique representation

$$T(X) = \sum_{j=0}^{d-1} c_j X^{q^j} \quad c_j \in \mathbb{E}.$$

Since $\mathbb{K} \subseteq \mathbb{E}$, we have $m \mid d$. Write each index uniquely as $j = ms + i$, with $0 \leq i < m$ and $0 \leq s < d/m$. Grouping the terms by their residue modulo m gives

$$T(X) = \sum_{i=0}^{m-1} \sum_{s=0}^{d/m-1} c_{ms+i} X^{q^{ms+i}}.$$

For each residue class i , define

$$T_i(X) = \sum_{s=0}^{d/m-1} c_{ms+i}^{q^{-i}} X^{q^{ms}},$$

where $z \mapsto z^{q^{-i}}$ denotes the inverse of the automorphism $z \mapsto z^{q^i}$ of $\mathbb{E}$. Each T_i is now a q^m -linearized polynomial hence applying the preceding theorem to the extension $\mathbb{K} \subseteq \mathbb{E}$, each T_i is $\mathbb{K}$ -linear. Then

$$(\sigma^i \circ T_i)(X) = T_i(X)^{q^i} = \sum_{s=0}^{d/m-1} c_{ms+i} X^{q^{ms+i}}.$$

Therefore T represented as

$$T = \sum_{i=0}^{m-1} \sigma^i \circ T_i.$$

Uniqueness follows immediately from the uniqueness of the q -linear polynomial representation. $\square$

The decomposition theorem extends coordinatewise to maps $\mathbb{E}^n \rightarrow \mathbb{E}$. In particular, the functional $T_{\mathbf{a}}(\mathbf{x}) = \langle W^{-1}(\mathbf{x}), \mathbf{a} \rangle_{\mathbb{E}}$ admits a unique decomposition

$$T_{\mathbf{a}} = \sum_{i=0}^{m-1} \sigma^i \circ T_i, \tag{39}$$

where each $T_i : \mathbb{E}^n \rightarrow \mathbb{E}$ is $\mathbb{K}$ -linear. Thus the initial claim may be written as

$$h = T_{\mathbf{a}}(W(\mathbf{w})) = \sum_{i=0}^{m-1} \sigma^i(T_i(W(\mathbf{w}))).$$

Construction C.9 (Frobenius fold). Given the claim $T_{\mathbf{a}}(W(\mathbf{w})) = h$, the parties proceed as follows.

1. The prover sends $v_i = T_i(W(\mathbf{w}))$ for $i = 1, \dots, m-1$.
2. The verifier derives $v_0 = h - \sum_{i=1}^{m-1} \sigma^i(v_i)$.
3. The verifier samples independent challenges $\rho_1, \dots, \rho_{m-1} \leftarrow \$ \mathbb{E}$.
4. The parties reduce to the claim $T'(W(\mathbf{w})) = h'$, where

$$T' = T_0 + \sum_{i=1}^{m-1} \rho_i T_i \quad \text{and} \quad h' = v_0 + \sum_{i=1}^{m-1} \rho_i v_i.$$

Construction C.10 (Folding protocol for extension opening). Choose a tower of intermediate fields $\mathbb{F} = \mathbb{K}_0 \subseteq \mathbb{K}_1 \subseteq \dots \subseteq \mathbb{K}_t = \mathbb{E}$, and write $m_i = [\mathbb{K}_{i+1} : \mathbb{K}_i]$. Initially, $T_{\mathbf{a}}$ is $\mathbb{K}_0$ -linear. At round i , apply the Frobenius fold with $\mathbb{F} = \mathbb{K}_i$, and $\mathbb{K} = \mathbb{K}_{i+1}$. This reduces the current $\mathbb{K}_i$ -linear functional to a $\mathbb{K}_{i+1}$ -linear functional.

After t rounds, the residual functional $T' : \mathbb{E}^n \rightarrow \mathbb{E}$ is $\mathbb{E}$ -linear. Consequently, there exists $\mathbf{a}' \in \mathbb{E}^n$ such that $T'(\mathbf{x}) = \langle \mathbf{x}, \mathbf{a}' \rangle_{\mathbb{E}}$. The final claim is therefore $\langle W(\mathbf{w}), \mathbf{a}' \rangle_{\mathbb{E}} = h'$, as required.

Each round requires sending $m_i - 1$ elements of $\mathbb{E}$. For a binary tower where $m_i = 2$, $\mathbb{F} = \mathbb{F}_2$, and $\mathbb{E} = \mathbb{F}_{2^{128}}$ we have 7 rounds of one $\mathbb{E}$ element message each, for a total of 112 bytes. Compare to the non-interactive protocol this reduces the communication complexity from $O(d^2)$ to $O(d \log d)$ elements of $\mathbb{F}_2$.

Towerless folding. Say $\mathbb{F} \subseteq \mathbb{E}$ admits no tower of subfields, for example $\mathbb{E} = \mathbb{F}_{2^{127}}$. Take again $d = [\mathbb{E} : \mathbb{F}]$ and decompose $T_{\mathbf{a}}$ as

$$T_{\mathbf{a}} = \sum_{i=0}^{d-1} \sigma^i \circ T_i,$$

We can still recursively fold using the protocol above with $m = 2$ by grouping even and odd exponents of σ^i (zero-padding as necessary), as there is no actual dependence on the subfield.

This folding progressively halves d (rounding up) in the $\mathbb{E}^{nd}$ inner product. This induces a sequence of perfect pairings

$$(\mathbb{F}^{nd}, \mathbb{E}^{nd}) \rightsquigarrow (\mathbb{E}^{nd/m}, \mathbb{E}^{nd/m}) \rightsquigarrow (\mathbb{K}_i^{nd/m}, \mathbb{E}^{nd/m})$$

$$\langle W_i(\mathbf{w}), A_i(\mathbf{a}) \rangle = \langle \mathbf{w}, \mathbf{a} \rangle.$$

The final result is a transcript dependent map $A : \mathbb{E}^{dn} \rightarrow \mathbb{E}^n$. If W is blockwise with $I_n \otimes W_0$ for some $W_0 : \mathbb{F}^d \rightarrow \mathbb{E}$ then so is A with $A_0 : \mathbb{E}^d \rightarrow \mathbb{E}$.

D Scratch

D.1 Adding Zero Knowledge

D.1.1 PIOP

D.1.2 F2Z-PCS

Albert: Sketch. Zk for F2Z is all known territory except Step 1 of [Construction 4.8](#) where P sends integer (polynomials) $\mu_1, \dots, \mu_{\ell_2}$. Even though $\pi_{\text{Canon}}^R{}^{-1}(\mathbf{v}^{(1)})$ is uniformly random in $[0, q-1]$, the elements μ_i do not follow a uniform distribution over any interval in $\mathbb{Z}$.

One way to address this is to have P commit to $\mu_1, \dots, \mu_{\ell_2}$ over $\mathbb{F}_{2^\nu}$ in zk (it commits to the bits), and to an element $u \in \mathbb{Z}$ such that $\langle \mathbf{f}_{*j}, \mathbb{F}_{2^\nu}^{-1}(\mathbf{v}^{(1)}) \rangle = \mu_j$ and $\sum_{j \in [\ell_2]} \mu_j \cdot \mathbf{v}_j^{(2)} = q \cdot u$ in $\mathbb{Z}$ (**)
Albert: note that in general we could be working on $\mathbb{Z}[X]$ here, so here we would add $+q \cdot u + h(X) \cdot w$ where $h(X)$ is the irreducible polynomial defining R . Now P proves (**) arithmetized over $\mathbb{F}_{2^\nu}$, using say Binius64 with zk.

Assume $\deg(R) = 0$ for now. Then P and V continue with [Construction 4.8](#), using zk-GKR and zk-Ligerito. At some point P needs to prove that some binary field elements equal $g^{\mu_1}, \dots, g^{\mu_{\ell_2}}$. It does this with one zk-GKR more grand product (this time much smaller).

If $\deg(R) > 0$, we proceed in the same way, except that the elements $g^{\mu_1}, \dots, g^{\mu_{\ell_2}}$ are instead $g^{\pi_q^{-1}(\pi_{q',\alpha}(\mu_j))}$, and so P must prove that some elements equal $g^{\mu'_j}$, and that $\mu_j = \pi_q^{-1}(\pi_{q',\alpha}(\mu_j))$ in $\mathbb{Z}$. The latter equality consists in evaluating the polynomial μ_j (whose bits we committed to over a binary field) on $X = \alpha'$, reducing mod q' , and lifting to integers. Thus we commit to $\mu'_j \in [0, q' - 1]$ and w'_j and prove that $\mu'_j = \mu_j(\alpha) + q' \cdot w'_j$ (***) .

When $\deg(R) > 0$, it is important to prove (**) and (***) together, since, at least for Binius64, integer multiplication proofs have a floor of about 40-50ms, but scale well. Typical ℓ_2 are 2^8 to 2^{11} . Claims (**) and (***) amount to $2 \cdot \ell_2 \cdot \deg(R) + 2$ 128-bit multiplications. For $\deg(R) = 2$ (occurs if we arithmetize our constraints over Goldilocks) and $\ell_2 = 2^{10}$ this is $\sim 2^{12}$ 128-bit integer mults.

Table 7: End-to-end Binius64 proving cost for N independent $128 \times 128 \rightarrow 256$ multiplications, without and with zero knowledge. Each multiplication is composed schoolbook-style from four native $64 \times 64 \rightarrow 128$ `imul` gates (4 MUL and 11 AND constraints). The ZK path is *recursive*: it symbolically executes the inner verifier to derive an outer Spartan constraint system, pads it with blinding, and commits through the BaseFold ZK compiler. Its overhead is therefore close to *additive* rather than multiplicative — roughly +430 KiB of proof, +2.4 ms of verification and +20–30 ms of proving, independent of N — so the relative cost falls as N grows. Measured on arm64 macOS (NEON+AES, all cores), `LOG_INV_RATE = 1`, Criterion mid-estimates over 10 samples.

N	Prove (ms)		Verify (ms)		Proof (KiB)		Peak mem (MB)	
	plain	ZK	plain	ZK	plain	ZK	plain	ZK
2^8	41.74	60.09	1.55	3.75	152.22	546.27	38.89	46.04
2^9	46.62	66.03	1.80	4.13	161.58	577.38	47.25	43.55
2^{10}	53.43	74.71	2.12	4.41	172.44	614.23	55.97	51.58
2^{11}	65.69	85.07	2.56	4.91	186.30	643.12	74.20	79.91
2^{12}	78.79	102.35	3.22	5.61	256.94	681.48	148.39	159.35
2^{13}	103.26	134.27	4.29	6.99	273.55	718.34	296.76	318.22

D.2 Open questions for us

Albert: Regarding [Section 6.3](#), I’ve only found a way to cheapen the following settings: say first and two vectors in the grand product are $\mathbf{g}_1, \mathbf{g}_2$, and third is $\mathbf{g}_1 \text{ XOR } \mathbf{g}_2$. This is a tricky problem. We should look more into it

Albert: Should we allow for $W > 1$? Currently just setting $W = 1$ and thinking of all bits as unrelated leads to better performance, but that sounds suboptimal.

Albert: Folding instances of $\mathcal{R}_{\text{Lin-BitsDual}}$